

第九部分 Python中的类

对象和类
类的创建
初始化方法__init__
检测缺失值的方法
成员私有化
类的继承
类存在的意义

面向对象编程语言

- Python语言是一种面向对象编程语言（Object-Oriented Programming Language），简称为**OOP语言**
- 无论是变量还是函数，它们都是属于某一个特定的类（class）。类中的对象叫做该类的**实例**（instance）
- 特定类的所有对象都与**方法**（method）相关联，这些方法类似于一个函数，我们编写一次之后，可以重复的调用
- 一旦某个对象属于某个特定类之后，该对象就可以使用该类的所有方法

面向对象编程语言

- Python是面向对象的高级动态编程语言，完全支持面向对象的基本功能，如封装、继承、多态以及对基类方法的覆盖或重写。
- Python中对象的概念很广泛，Python中的一切内容都是对象，除了数值、字符串、列表、元组、字典、集合、range对象等等，函数也是对象，类也是对象。

面向对象编程语言



- 一个人（篮球运动员）
  - 人的组成元素
    - 头、五官、四肢
    - 性别、血型、身高、名字
    - 所在球队、球衣号码
  - 人的行为
    - 吃喝
    - 跑步、跳跃
    - 运球、投篮、传球、抢断、传球

类（class）

高中生类



属性：姓名、年龄、性别、分数

方法：对属性进行查询或修改、与其他同类发生一些关系、运算等等

对象（object）

高中生类





高中生1

对象的属性
姓名：王耀华
年龄：18
性别：男
成绩：85
……

共有的方法：
改变姓名、
改变成绩等
等



高中生2

对象的属性
姓名：刘美丽
年龄：19
性别：女
成绩：95
……

内置类（类型）和对象

例如：字符串类和字符串对象

```
>>> s1=str(123)
>>> s2='abc'
>>> print(s1+'  '+s2)
123  abc
>>> s1.islower()
False
>>> s2.isnumeric()
False
```

已学的类（类型）

类（类型）：

int、 float 、bool 、string

list 、tuple 、dict 、set

对象举例：

3，[5.7,’a’]，{1:4,2:5}

type函数判断对象的类

类的名字空间

- 每个类有自己的名字空间，类名是空间名
- 每个对象也有自己的名字空间
- 存储在类名称空间中的名称是类的成员
- 类成员包含
- 类属性
- 类方法
- 右边以list类为例

- >>> dir(list)　　#显示类成员
- ['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
- 。。。。。。
- 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']

- #使用list作为名字空间访问
- >>> list.sort
- <method 'sort' of 'list' objects>

类方法是类名字空间中定义的函数

- lst=[4,97,7,34,-1]
- lst.sort()
- 相当于：
- list.sort(lst)
- list是lst的类名

- 字典 d.keys()
- 相当于：
- dict.keys(d)

- instance.method(...)
- 相当于：
- class.method(instance,...)

- class是instance的类型

对象继承类的方法

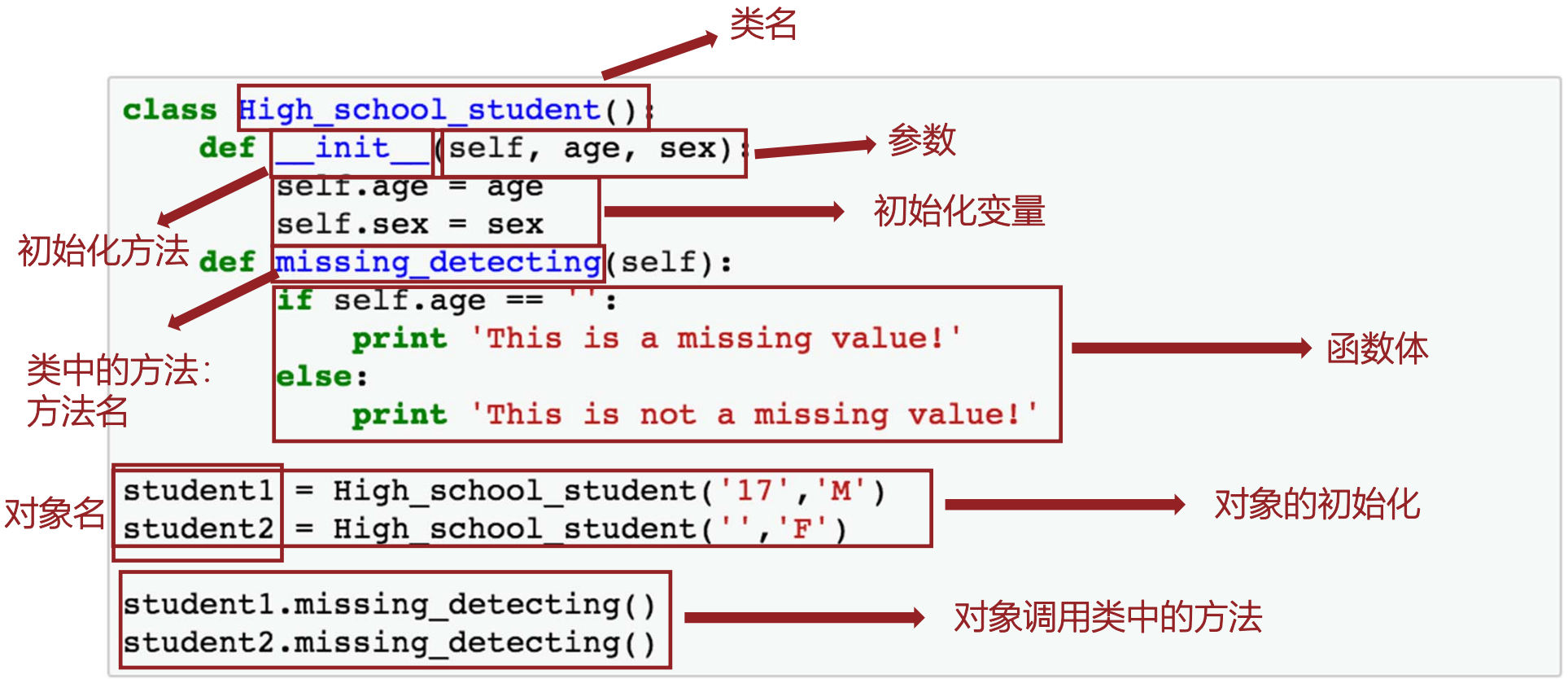
- >>> dir(list)
- ['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
- >>> lst=[4,97,7,34,-1]　　#lst是list的对象
- >>> dir(lst)
- ['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
- 不同对象一般有相同的方法，但属性可以不同

类和对象概念小结

- 类(class)：定义属性(数据)和行为（方法）的模板 ，
- 实例(instance)：是用类产生的对象，术语对象(object)和实例(instance)经常是可以互换的。
- 使用圆点运算符(.)引用方法和属性
- 类有自己名字空间
- 每个对象也有自己的名字空间
- 同义词：

对象 实例 实例对象
类 类型 类对象
属性 变量
方法 操作 函数

代码中的类和对象




```
This is not a missing value!
This is a missing value!
```

类的创建

- 用 class 类名 即可以创建一个类
- 在类名的程序块中可以定义这个类的属性、方法等等

```
class High_school_student():
    def __init__(self):
        self.age = 18.0
        self.sex = 'M'
        self.gradyear = 2006
```

创建对象

- 创建相应的实例

```
student_a = High_school_student()
print student_a
print High_school_student
```

```
<__main__.High_school_student instance at 0x104191680>
__main__.High_school_student
```

- 查看实例属性

```
student_a.age
```

```
18.0
```

初始化__init__方法

- __init__是一种特殊的方法，使得实例一开始就拥有类模板所具有的属性
- self参数是类中函数定义时必须使用的参数，并且永远是第一个参数，该参数表示创建的实例本身

```
class High_school_student():
    def __init__(self, age, sex):
        self.age = age
        self.sex = sex
```

正确地初始化对象

- 一旦"__init__"存在除了"self"之外的参数，那么在创建实例"student_a"时，我们就需要传入相应的参数值（不过"self"不需要传入数值，Python解释器会自动将实例变量传入）

```
student_a = High_school_student()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)

<ipython-input-6-2078c4b86214> in <module>()
----> 1 student_a = High_school_student()


TypeError: __init__() takes exactly 3 arguments (1 given)
```

正确地初始化对象

- 如果我们传入了两个参数——18 与 M，则我们可以成功创建一个高中生对象，这个高中生的年龄是18，性别是M
- 我们可以不断创建拥有不同年龄与性别的高中生对象实例

```
student_a = High_school_student(18.0, 'M')
print student_a.age
print student_a.sex
```

```
18.0
M
```

自定义检测缺失值的方法

- 检测实例对象中的 age 属性是否为缺省值，自定义相应的missing_detecting 方法

```
class High_school_student():
    def __init__(self, age, sex):
        self.age = age
        self.sex = sex
    def missing_detecting(self):
        if self.age == '':
            print 'This is a missing value!'
        else:
            print 'This is not a missing value!'
```

使用自定义的类方法

- 类中的方法的第一个参数必须为 self 。 下面我们首先创建实例 student_a ， 然后调用"missing_detecting"方法：

```
student_a = High_school_student('','F')
student_a.missing_detecting()
```

```
This is a missing value!
```

在__init__方法中导入数据

- 在每一次创建实例的时候，自动导入数据，并且定义与数据有关的变量

```
import csv
class High_school_student():
    def __init__(self):
        f = open('teenager_sns.csv')
        csvreader = csv.reader(f)
        teenager_sns = list(csvreader)
        # 删除第一个元素，即变量名列表元素
        del teenager_sns[0]
        self.teenager_sns = teenager_sns
    def missing_detecting(self, age, sex):
        if age == '' or sex == 'NA':
            missing = True
        else:
            missing = False
        return missing
```

例2：Student类和对象（1）

```
class Student:          #学生类：包含成员变量和成员方法
        def __init__ (self,mname,mnumber):  #构造方法
                self.name = mname#成员变量
                self.number = mnumber
                self.Course_Grade = {}
                self.GPA = 0
        def getInfo(self):          #成员方法
            print(self.name,self.number)
s1 = Student("wang","317000010")   #创建s1对象
s1.getInfo()
#Student.getInfo(s1）
s2=Student("zhang","317000011")        #创建s2对象
s2.getInfo()
```

Student类和s1对象的名字空间

- >>> dir(Student)
- ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'getInfo']
- >>> dir(s1)
- ['Course_Grade', 'GPA', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'getInfo', 'name', 'number']

Student类和s1对象

- 上述程序定义Student类:
  - 两个方法：__init__，getInfo。
  - 实例化了s1与s2两个对象。
  - 该类对象包括四个成员变量: name, number, Course_Grade, GPA;

__init__方法

- __init__方法是Python类中的一种特殊方法，方法名的开始和结束都是双下划线，该方法称为构造方法，当创建类的对象时，它被自动调用。
- __init__方法中可以声明类所产生的对象属性，并可为其赋初始值。该方法有一个特点，不能有返回值，因为它是用来构造对象的，调用后实例化了一个该类型的对象。

Self参数

- 类的实例方法按惯例有一个名为self的参数，并且必须是方法的第一个形参（如果有多个形参的话），self参数代表将来要创建的对象本身。
- 在类的方法中访问实例变量（数据成员）时需要以self为前缀。
- 在外部通过对象调用对象方法时并不需要传递这个参数，如果在外部通过类调用对象方法则需要显式为self参数传值。

创建对象

■定义了类之后，可以用来实例化对象，并通过“对象名.成员”的方式来访问其中的数据成员或方法成员。

```
s1 = Student(“ji”,”100001”)
s1.getInfo()
```

- 在Python中，可以使用内置方法isinstance()来测试一个对象是否为某个类的实例。

```
>>> isinstance(s1, Student)
True
>>> isinstance(s1, str)
False
```

成员的私有化

- 面向对象的语言一般都对对象有一些统一的要求。例如封装性。
- 封装性的意思是指，一个对象的成员属性要得到一定程度的保护。例如，要对一个对象的成员属性进行修改或访问，就必须通过对象允许的方法来进行（例如要求输入密码以确认拥有此权限等）。这样可以保护对象，使程序不易出错。
- C++与Java语言中，严格地实现了对象的封装性。

对象的封装性

- 例如在上面的 High_school_student 类中，虽然数据已经被封装在类里面，但是我们还是可以通过外部访问其中的变量。我们可以在外部对 age 进行修改：

```
class High_school_student():
    def __init__(self):
        self.age = 18.0
        self.sex = 'M'

student_a = High_school_student()
print student_a.age
student_a.age = 18.8
print student_a.age
```

说明这个High_school_student类没有进行有效的封装！

```
18.0
18.8
```



Python对于封装性的看法

- 设计封装的逻辑往往要耗费一定的精力，并且会导致代码更加冗长、不易读等等
- Python的语言风格没有特别强调封装性。它主张程序员自己通过更加严谨的方式组织程序以避免出错，而不是通过在语言层面实现封装达到这一点
- 但是，在Python中仍然可以实现对象的（非严格意义上的）封装

- 如果我们希望某些内部属性不被外部访问，我们可以在属性名称前加上两个下划线"__"，表示将该属性成员私有化，该成员在内部可以被访问，但是在外部是不能够访问的。

```
class High_school_student():
    def __init__(self):
        self.__age = 18.0
        self.__sex = 'M'

student_a = High_school_student()
student_a.__age
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-44-5796a68eb152> in <module>()
      5
      6 student_a = High_school_student()
----> 7 student_a.__age


AttributeError: High_school_student instance has no attribute '__age'
```

- 成员私有化不仅包括属性的私有化，也包括了方法的私有化，在方法名称前加上两个下划线"_"也可以使得函数只能被内部访问，不能够被外部访问
- 成员私有化并不是代表完全不能够从外部访问成员，而是提高了访问的门槛，防止意外或者随意改变成员，引发错误。我们仍然可以通过_类名+私有变量，对变量进行访问

```python
class student():
    # 对象初始化方法，在创建对象的时候调用
    def __init__(self, name, age, gender):
        # 属性名称前加上两个下划线"_",表示将该属性成员私有化
        self.__name = name
        self.age = age
        self.gender = gender

    def print_info(self):
        print(self.__name, self.age, self.gender)


s = student("dd", 22, "男")
# _类名+私有变量,访问私有变量
print(s._student__name)

输出结果:
dd
```

- 这也是我们说Python没有严格意义上的封装性的原因！

- 成员私有化不仅包括属性的私有化，也包括了方法的私有化，在方法名称前加上__也可以使得函数只能被内部访问，不能够被外部访问：

```
class High_school_student():
    def __init__(self):
        self.age = 18.0
        self.sex = 'M'
    def __missing_detecting(self):
        if self.age == '' or self.sex == 'NA':
            missing = True
        else:
            missing = False
        return missing

student_a = High_school_student()
student_a.__missing_detecting
```

对missing_detecting方法的访问将被阻止！

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-50-39ff3ef81fac> in <module>()
     11
     12 student_a = High_school_student()
---> 13 student_a.__missing_detecting

AttributeError: High_school_student instance has no attribute '__missing_detecting'
```

类的继承

- 与封装性一样，继承性是面向对象程序设计的另一种重要的属性。
- 类的继承好比孩子与父母之间的继承关系一样，孩子拥有父母所拥有的许多特性。在编程语言中，如果一个新的类继承另外一个类，那么这个新的类成为子类（Subclass），被子类继承的类称为父类
- 我们可以把更加一般、范围大的类的属性在父类定义，把更加具体、范围小的特点在子类定义。

定义子类

在定义好High_school_student后，可以定义两个子类Male_student与female_student继承它

```
class High_school_student():
    def __init__(self):
        f = open('teenager_sns.csv')
        csvreader = csv.reader(f)
        teenager_sns = list(csvreader)
        # 删除第一个元素，即变量名列表元素
        del teenager_sns[0]
        self.teenager_sns = teenager_sns

    def missing_detecting(self, age, sex):
        if age == '' or sex == 'NA':
            missing = True
        else:
            missing = False
        return missing
```

父类的定义

子类继承父类
没有定义专属于子类的属性/方法

```
class Male_student(High_school_student):
        pass

class Female_student(High_school_student):
        pass
```

子类继承父类属性及实例化方法

- 子类 Male_student 的实例 male_student 继承了父类的初始化属性 self.teenager_sns

```
male_student = Male_student()
print male_student.teenager_sns[0]
```

```
['2006', 'M', '18.982', '7', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0',
```

- 同样的，子类也继承 missing_detecting 和 outlier_detecting 两个实例方法：

```
print male_student.missing_detecting(male_student.teenager_sns[0][2],male_student.t
print male_student.outlier_detecting(male_student.teenager_sns[0][2])
```

```
False
False
```

- 比如，我们试试在子类中也定义一个 missing_detecting 方法：

```
class Male_student(High_school_student):
        def missing_detecting(self, age, sex):
            print 'This is a subclass method!'
```

- 然后试试创建子类实例并调用 missing_detecting 方法：

```
male_student = Male_student()
male_student.missing_detecting(male_student.teenager_sns[0][2],male_student.teenager_sns[0][1])
```

```
This is a subclass method!
```

可以看出这里我们用的是子类的missing_detecting，而不是父类的missing_detecting!

子类中定义__init__方法

- 如果不注意继承性的这个特点，有可能导致错误！

```
class Male_student(High_school_student):
    def __init__(self):
        self.male_info = 'This is a subclass attribute!'

    def missing_detecting(self, age, sex):
        print 'This is a subclass method!'
```

```
male_student = Male_student()
male_student.teenager_sns
```

```
---------------------------------------------------------------------------

AttributeError                            Traceback (most recent call last)

<ipython-input-194-6d181630c160> in <module>()
      1 male_student = Male_student()
----> 2 male_student.teenager_sns


AttributeError: Male_student instance has no attribute 'teenager_sns'
```

例：计算身体质量指数BMI

BMI是根据体重和身高来衡量健康一种方法。通过以千克为单位的体重除以以米为单位的身高的平方计算出BMI。下面是中国16岁以上人群的BMI图表：

构造函数

```
class BMI:
    def __init__(self, name, age, weight, height):
 #构造方法，创建对象时调用
        self.__name = name
        self.__age = age
        self.__weight = weight
        self.__height = height

    def getBMI(self):  #计算BMI
        bmi = self.__weight  /  (self.__height *
                    self.__height )
        return round(bmi * 100) / 100
```

getStatus(self)方法

```
def getStatus(self):
        bmi = self.getBMI()
        if bmi < 18.5:
            return "偏瘦"
        elif bmi < 24:
            return "正常"
        elif bmi < 30:
            return "偏胖"
        else:
            return "肥胖"
```

其他方法

```
def getName(self):
        return self.__name

def getAge(self):
        return self.__age

def getWeight(self):
        return self.__weight

def getHeight(self):
        return self.__height
```

创建对象

```
def main():
    bmi1 = BMI("赵四", 18, 70, 1.75)
    print( bmi1.getName(), "的BMI是",
        bmi1.getBMI(), bmi1.getStatus())

    bmi2 = BMI("王超", 38, 75, 1.70)
    print( bmi2.getName(), "的BMI是",
        bmi2.getBMI(), bmi2.getStatus())

main() # Call the main function
```

设计向量类

- 设计一个三维向量类，并实现向量的**加法**、**减法**以及向量与标量的**乘法**和**除法**运算

向量类方法（1）

```
class Vecter3:
  def __init__(self, x=0, y=0, z=0):#构造方法
    self.X = x
    self.Y = y
    self.Z = z

  def __add__(self, n): #加法
    r = Vecter3()
    r.X = self.X + n.X
    r.Y = self.Y + n.Y
    r.Z = self.Z + n.Z
    return r
```

向量类方法（2）

```
def __sub__(self, n):   #减法
    r = Vecter3()
    r.X = self.X - n.X
    r.Y = self.Y - n.Y
    r.Z = self.Z - n.Z
    return r

def __mul__(self, n):  #乘法
    r = Vecter3()
    r.X = self.X * n
    r.Y = self.Y * n
    r.Z = self.Z * n
    return r
```

实例

```
def __truediv__(self, n):  #除
    r = Vecter3()
    r.X = self.X / n
    r.Y = self.Y / n
    r.Z = self.Z / n
    return r

def __floordiv__(self, n):  #整除
    r = Vecter3()
    r.X = self.X // n
    r.Y = self.Y // n
    r.Z = self.Z // n
   return r

def show(self):  #打印向量值
     print((self.X,self.Y,self.Z))
```

```
def add3(self):
        self.X = self.X + 3
        self.Y = self.Y + 3
        self.Z = self.Z + 3
v1 = Vecter3(1,2,3)
v2 = Vecter3(4,5,6)
v3 = v1+v2
v3.show()
v4 = v1-v2
v4.show()
v5 = v1*3
v5.show()
v6 = v1/2
v6.show()
v1.add3()
v1.show()
```

封装

- 封装 ：
  - 将数据和对数据的操作组合起来构成类，类是一个不可分割的独立单位
  - 类中既要提供与外部联系的接口，同时又要尽可能隐藏类的实现细节。
  - Python类中成员分为数据（变量、属性）成员和方法（函数）成员。

方法和属性类型

- Python类中成员：
  - 数据成员（变量、属性）
    - 类数据成员
    - 实例数据成员
    - 公有
    - 内部：以下划线'__'开头
  - 方法（函数）
    - 实例方法：公有
      私有：方法名以下划线'__'开头
    - 类方法：　@classmethod
    - 静态方法：@staticmethod

Python社团约定

- 在Python中，以下划线开头的方法名和变量名有特殊的含义，尤其是在类的定义中。
  - _xxx：当做内部名，不应该在外部使用，
  - 不能用'from module import *'导入；
  - __xxx：解释器会换名，不能使用对象直接访问到这个成员。
  - __xxx__：系统定义的特殊成员；不要创建这种标识符

类成员变量与实例成员变量

- 实例的成员变量一般是指在构造方法__init__()中定义的，定义和使用时以self作为前缀
- 类的成员变量是在类中所有方法之外定义的

- 特性：在主程序中（或类的外部），实例变量属于实例(对象)，只能通过对象名访问；而类变量属于类，可以通过类名或对象名都可以访问。

类成员与实例成员

```
class Car:
    price = 100000              #定义类成员变量
    def __init__(self, c):
        self.color = c          #定义实例成员变量

car1 = Car("Red")               #实例化对象
car2 = Car("Blue")
print(car1.color, Car.price)    #查看实例变量和类变量的值
Car.price = 110000              #修改类变量
Car.name = 'QQ'                 #动态增加类变量
car1.color = "Yellow"           #修改实例变量
print(car2.color, Car.price, Car.name)
print(car1.color, Car.price, Car.name)
```

Car类和car1对象的名字空间

- >>> dir(Car)
- ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'price']
- >>> dir(car1)
- ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', 'color', 'price']

练习

```
class Car:
    price = 300000              #定义类变量

    def __init__(self, name):
        self.name=name          #定义实例变量
        self.color = ""     #存储汽车颜色

    def setColor(self,color):     #设置汽车的颜色
        self. color = color

car1 = Car("奥迪")                #创建car1对象
car2 = Car("宝马")                #创建car2对象
print(car1.name, Car.price)         #打印实例变量和类变量的值
Car.price = 310000                #修改类变量
car1.setColor ( '黑色')           #car1的汽车型号为QQ
car1.name = "新奥迪"                #修改实例变量
print(car1.name, Car.price, car1.color)
print(car2.name, Car.price, car2.color)
```

类方法（class method）

- 类方法会作用于整个类，对类作出的任何改变会对它的所有实例对象产生影响。
- #打印对象的个数
- class A():
- 　　count = 0
- 　　def __init__(self):
- 　　　　A. count += 1
- 　　def exclaim(self):
- 　　　　print("我是A!")
- 　　@classmethod
- 　　def kids(cls):　　　　#cls是class缩写
- 　　　print("A有",cls.count,"个对象")
- one = A()
- two = A()
- three= A()
- A.kids()

静态方法

- 即不影响类，也不影响对象
- 不用创建对象
- class Advise():
- 　　@staticmethod
- 　　def commercial():
- 　　　　print("这是广告")
- Advise.commercial()

Pass语句作用

- Python提供了一个关键字“pass”，类似于空语句，可以用在类和函数的定义中或者选择结构中。当暂时没有确定如何实现功能，或者为以后的软件升级预留空间，或者其他类型功能时，可以使用该关键字来“占位”。

```
>>> class A:
        pass
>>> def demo():
        pass
>>> if 5>3:
        pass
```

成员的输入显示

- 在IDLE环境中，在对象或类名后面加上一个圆点“.”，稍等则会自动列出其所有公开成员，模块也具有同样的用法。

- 如果在圆点“.”后面再加一个**下划线**“_”，则会列出该对象、类或模块的所有成员，包括私有成员。

类的继承和多态

- 一个类通过继承成新的类
- 扩展list类，增加随机选取功能
- import random
- class Mylist(list): #list是父类名
- 	def choice(self):
- 		return random.choice(self)
-
- lst=Mylist("good moring")
- print(lst.choice())

super函数

```
class Car():
    price = 300000                #定义类变量
    def __init__(self, name):
            self.name=name          #定义实例变量
            self.color = ""         #存储汽车颜色
    def setColor(self,color):           #设置汽车的颜色
            self. color = color
class ECar(Car):
     def __init__(self,name):
            super().__init__(name)   #初始化父类的属性
            self.battery_size = 300
     def getEcar(self):
            print("我是电动汽车"+self.name+"电瓶容量为"+ str(self.battery_size)
                  +"公里")

car=ECar("曹操专车")
car.getEcar()
```

多态

- python的多态性是指具有不同功能的运算符（函数）可以使用相同的符号，这样就可以用一个符号调用不同的功能

```
>>> 12+6
18
>>> '12'+'6'
'126'
>>>
```

运算符是方法，也是函数

- >>>2+3
- 5
- >>> int(3).__add__(2)
- 5
- >>> int.__add__(3,2)
- 5

*运算符重载（1）

- 运算符重载是Python设计理念的核心之一

运算符	方法	数字	字符串和列表
a+b	a.__add__(b)	加法	拼接
a-b	a.__sub__(b)	减法	---------
a*b	a.__mul__(b)	乘法	自连接
a/b	a.__truediv__(b)	除法	------
a//b	a.__floordiv__(b)	整除	------
a&b	a.__mod__(b)	余数	------
a==b	a.__eq__(b)	等于	
a!=b	a.__ne__(b)	不等于	
a>b	a.__gt__(b)	大于	
a>=b	a.__ge__(b)	大于或等于	

*运算符重载（2）

运算符	方法	数字	字符串
a<b	a.__lt__(b)	小于	
a<=b	a.__le__(b)	小于等于	
str(a)	a.__str__()	字符串表示形式	
len(a)	a.__len__()	-----	容器大小
type(a)	type.__init__(a)	构造函数	
[] (取值）	__getitem__()		
[]（赋值）	__setitem__()		

总结

- 对象与类
- 类的初始化：__init__函数创建对象的属性
- Python的封装性：提高访问门槛以防出错
- Python的继承性：子类继承父类，子类会覆盖父类
- 类存在的意义



第十部分 变量作用域

局部变量
全局变量
局部变量转为全局变量
同名变量引用

命名空间和作用域

1.一个名称在不同的使用情况下可能指代不同的事物。

2.Python程序用命名空间区分不同情况下的名字。

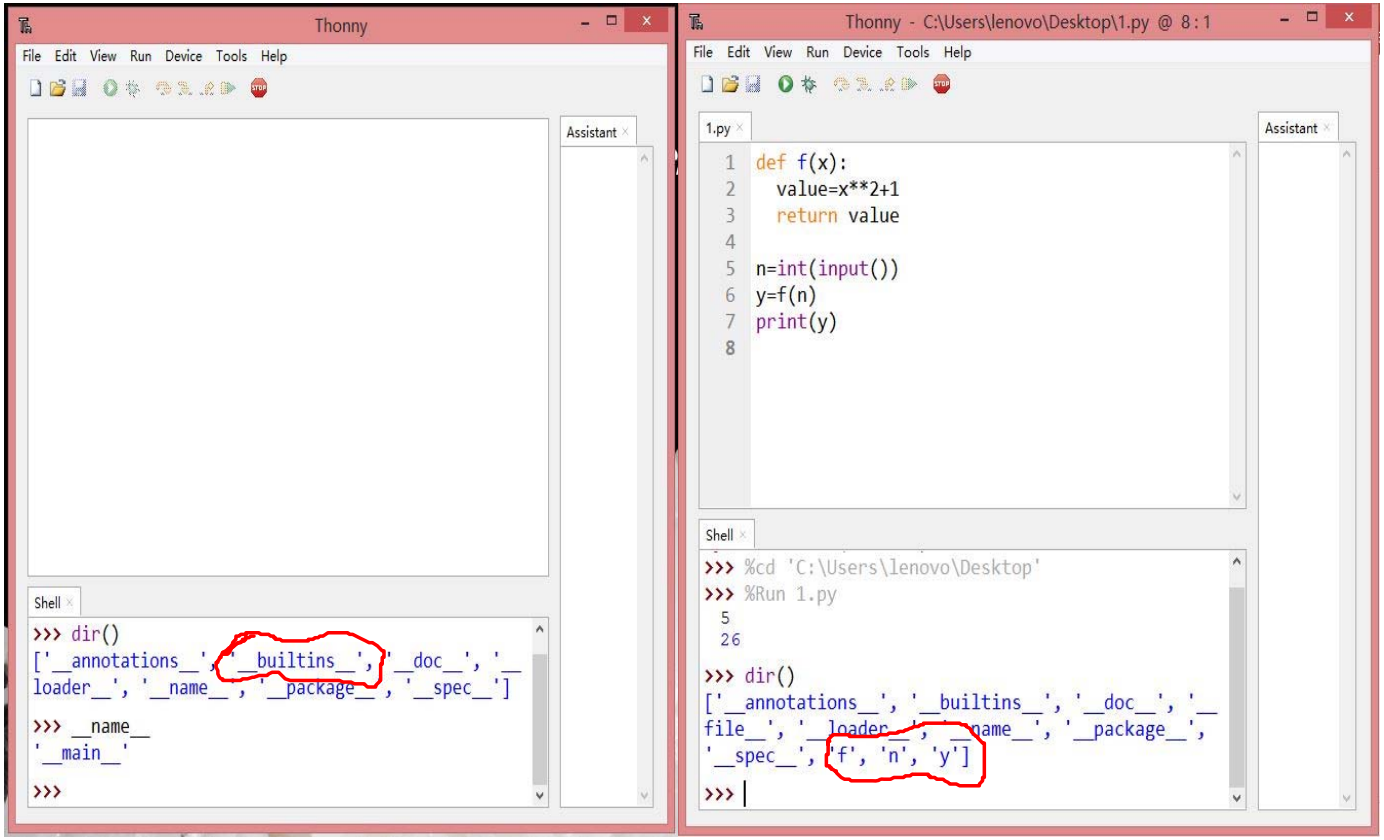
Python解释器启动时建立的初始环境里有一个内置命名空间(built-in namespace),记录所有标准常量名、标准函数名等。

3.程序运行时建立一个全局命名空间，全局变量就放在这个空间中。

4.每一个函数定义自己的命名空间，函数内部定义的变量是局部变量。如果在一个函数中定义一个变量x，在另外一个函数中也定义x变量，因为是在不同的命名空间，所以两者指代的是不同的变量。可以通过多种方式获取其他命名空间的变量。

5.每个程序在函数外是全局命名空间，在全局命名空间中的变量是全局变量。

全局命名空间


Thonny
File Edit View Run Device Tools Help
Assistant
Shell
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__']
>>> __name__
'__main__'
>>>
Thonny - C:\Users\lenovo\Desktop\1.py @ 8 : 1
File Edit View Run Device Tools Help
1.py
Assistant
1 def f(x):
2 value=x**2+1
3 return value
4
5 n=int(input())
6 y=f(n)
7 print(y)
8
Shell
>>> %cd 'C:\Users\lenovo\Desktop'
>>> %Run 1.py
5
26
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'f', 'n', 'y']
>>>

局部变量与全局变量

- Python语言规定赋值即定义。“var=1”赋值语句定义了变量“var”并赋值为1
- 全局变量：定义在函数外，存在于整个程序。
- 局部变量：定义在函数内，存在于该函数内部。
-

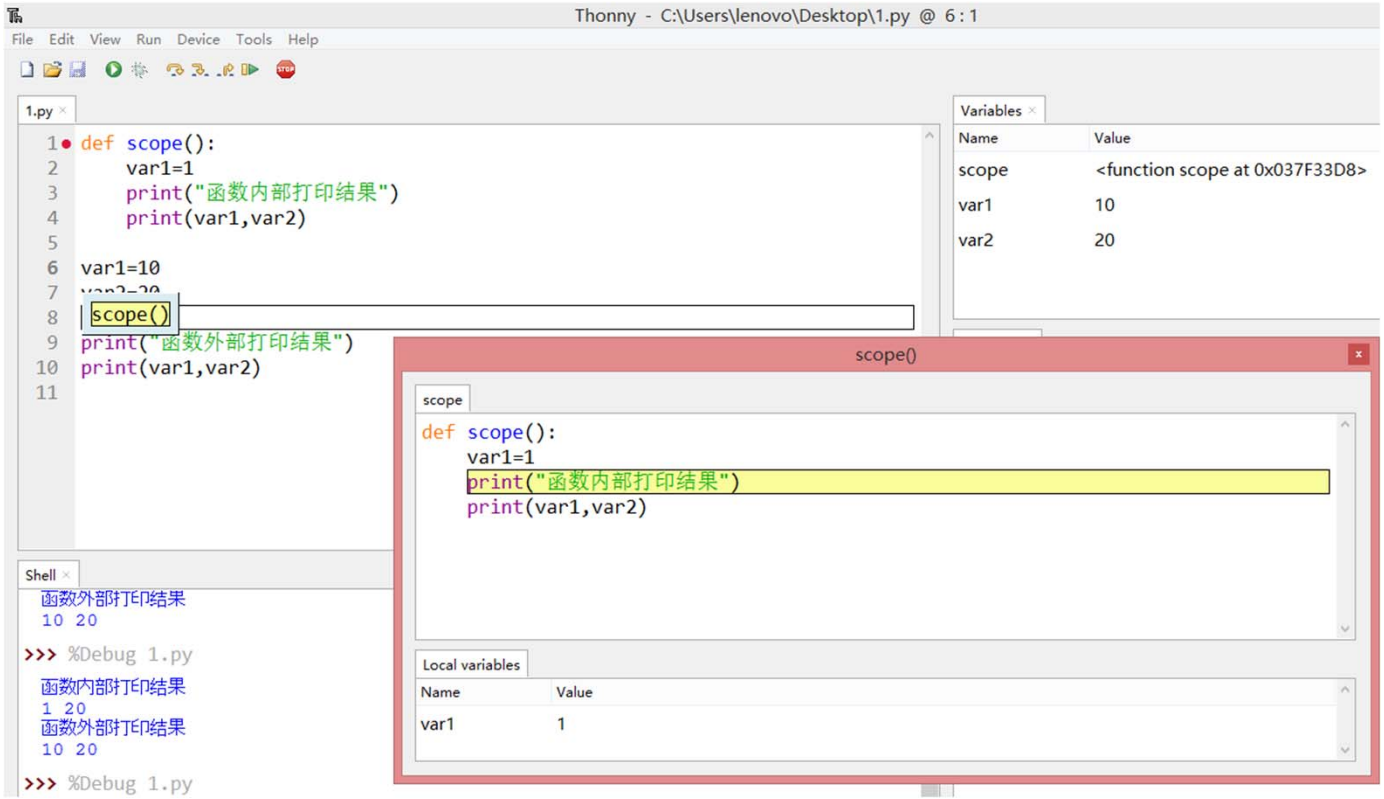
局部变量与全局变量同名

```
def scope():
    var1=1
    print("函数内部打印结果")
    print(var1,var2)
var1=10
var2=20
scope()
print("函数外部打印结果")
print(var1,var2)
```

输出：

函数内部打印结果
1 20
函数外部打印结果
10 20

函数有自己的名字空间


Thonny - C:\Users\lenovo\Desktop\1.py @ 6 : 1
File Edit View Run Device Tools Help
1.py
1 def scope():
2 var1=1
3 print("函数内部打印结果")
4 print(var1,var2)
5
6 var1=10
7
8 scope()
9 print("函数外部打印结果")
10 print(var1,var2)
11
Variables
Name Value
scope <function scope at 0x037F33D8>
var1 10
var2 20
scope()
scope
def scope():
var1=1
print("函数内部打印结果")
print(var1,var2)
Local variables
Name Value
var1 1
Shell
函数外部打印结果
10 20
>>> %Debug 1.py
函数内部打印结果
1 20
函数外部打印结果
10 20
>>> %Debug 1.py

global关键字

如希望在函数中使用全局变量，需要用global关键字声明

```
def scope():
    global var1
    var1=1
    print("函数内部打印结果")
    print(var1,var2)
var1=10
var2=20
scope()
print("函数外部打印结果")
print(var1,var2)
```

局部变量

- 在关键词 **def** 定义函数的范围内，新定义/赋值的变量都是局部变量，在该函数之外引用函数内命名的变量的时候会报错

```
def echo(a):
    a="Hello "+a
    print a

echo("world")
print a
```

```
Hello world

Traceback (most recent call last):
  File "", line 1, in
    a
NameError: name 'a' is not defined
```

局部变量

- 同理，在局部区域引用局部区域以外的变量，也会引起报错。

```
A=0
def A_add_one():
    A=A+1
    return A

A_add_one()
```

```
Traceback (most recent call last):
  File "", line 1, in
    A_add_1()
  File "", line 2, in A_add_1
    A=A+1
UnboundLocalError: local variable 'A' referenced before assignment
```

练习——结果是什么？

```
x=100
def  f():
    y=x
    x=0 #加了这一行以后，函数内上下文的x就被当做了局部变量，会报错
   print(x)
f()
```

- 如何改？x作为全局变量的话，就要改成global x

局部变量

- 局部变量是指那些有固定的变量作用域，只有在此作用域内才能调用此变量。具体而言，比如函数内的局部变量的作用域仅限于函数内。以下建立一个新的函数，命名为**mean()**，用于求平均

```
def mean(x):

    length = 52.0 #局部变量，数据集总共长度52
    sum_x = 0
    for i in x:
        sum_x += i

    return sum_x/length
```

全局变量

- 全局变量是相对局部变量而言的作用范围在全局，即在初始定义赋值后，无论是函数、类、**lambda**函数内都可以引用全局变量。在关键词 **def** 、**class**、**lambda** 之外定义的变量，都作为全局变量。
- 在上面的**mean()**函数内定义的**length**变量移至关键词 **def**之外即变为全局变量：

```
length = 52.0  #全局变量

def mean(x):

    sum_x = 0
    for i in x:
        sum_x += i

    return sum_x/length
```

局部变量转为全局变量

- 有时候会有在函数内定义的局部变量在函数外引用的需求，比如前面的**mean**()函数中最开始"**length**"定义时是局部变量，现只需要在定义变量时使用关键词"**global**"即可将其定义为全局变量

```
def mean(x):

    global length #初始化全局变量length
    length = 52.0 #全局变量length赋值
    sum_x = 0
    for i in x:
        sum_x += i

    return sum_x/length
```

同名变量引用

- 当某局部变量和全局变量都有相同变量名时，函数内引用该变量会直接调用函数内定义的局部变量。

```
length = 1.0  #全局变量设置为1

def mean(x):

    length = 52.0 #局部变量
    sum_x = 0
    for i in x:
        sum_x += i

    return sum_x/length

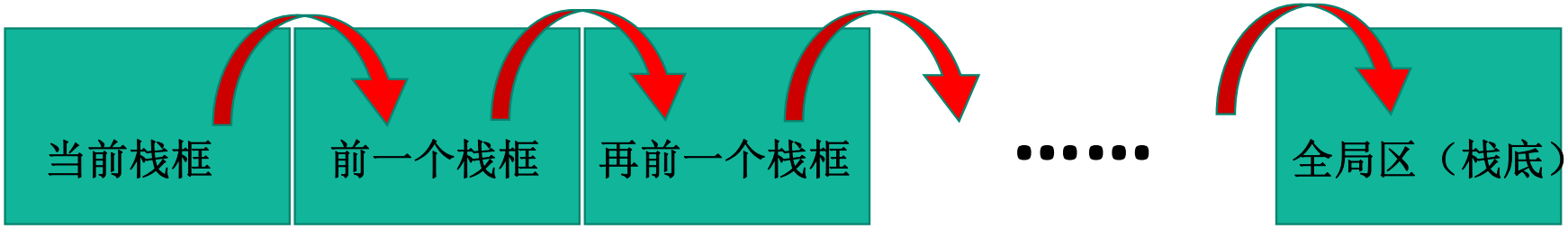
print mean(range(52))
```

```
25.5
```

问题：如果有嵌套函数，并且有多个同名变量该怎么办？

LEGB原理简要介绍

- 当一个函数体内需要引用一个变量的时候，会按照如下顺序查找：
- 首先查找局部变量（Locals）；
- 如果找不到叫做该名称的局部变量，则去函数体的外层去寻找局部变量（Enclosing function locals）。（适用于嵌套函数的情况）
- 如果函数体外部的局部变量中也找不到叫做该名称的局部变量，则从全局变量（Global）中寻找。再找不到，只好去找内置库（Bulit-in）
- 像C语言就没有这种机制，局部区找不到就直接跳到静态变量（static）区了





第十一部分 Python中的模块

模块简介
数据分析领域常用的包
搜索路径管理

模块的导入
利用csv、pandas导入数据
缺失值处理
使用matplotlib绘图

Python的模块函数分三个层次

一.内置函数

不用import语句引入，它里面的函数可直接调用。

二.标准模块函数

用import语句引入后再调用，但不必安装。如math库。

三.第三方模块函数

需先安装，再用import语句引入模块后才能调用里面的函数，如Pandas模块。

第三方库安装

- 打开网页：https://pypi.org/
- 输入模块名，就可查到模块的详细说明



pip命令

- c:\>pip install pandas 安装pandas模块

```
pip <command> [options]

Commands:
install                     Install packages.
download                    Download packages.
uninstall                   Uninstall packages.
freeze                      Output installed packages in requirements format.
list                        List installed packages.
show                        Show information about installed packages.
config                      Manage local and global configuration.
search                      Search PyPI for packages.
wheel                       Build wheels from your requirements.
hash                        Compute hashes of package archives.
completion                  A helper command used for command completion.
help                        Show help for commands.
```

模块简介

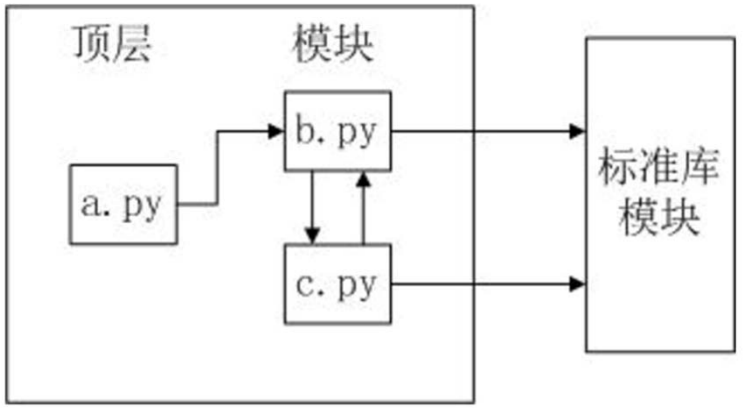
- 内置电池（batteries included）

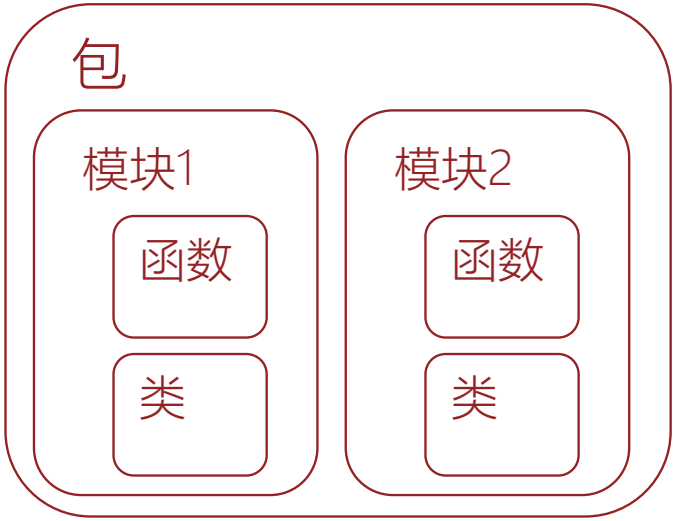
  基础代码库，覆盖了网络通信、文件处理、数据库接口、图形系统、XML处理

- 第三方工具包
  - Pandas：数据处理与分析
  - Numpy：科学计算
  - BeautifulSoup4：处理HTML文档
  - Matplotlib：绘图
  - Scikit-learn：基于SciPy和Numpy的开源机器学习模块
  - Tensorflow：深度学习框架

Python程序的架构

- Python程序的构架指：将一个程序分割为源代码文件的集合以及将这些部分连接在一起的方法
- Python源代码文件：*.py
  - 一个py文件是一个模块（module）
  - 多个模块可以组成一个包（package）

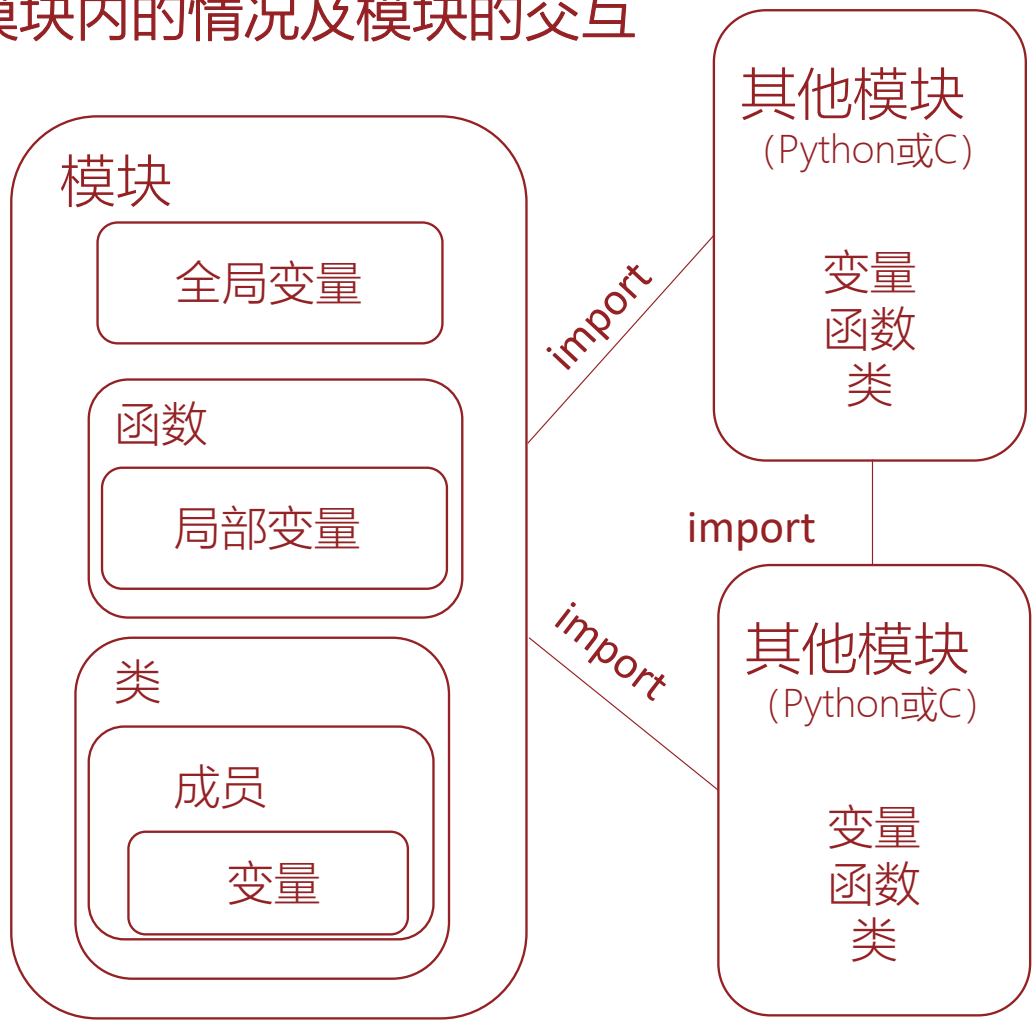




模块的执行环境

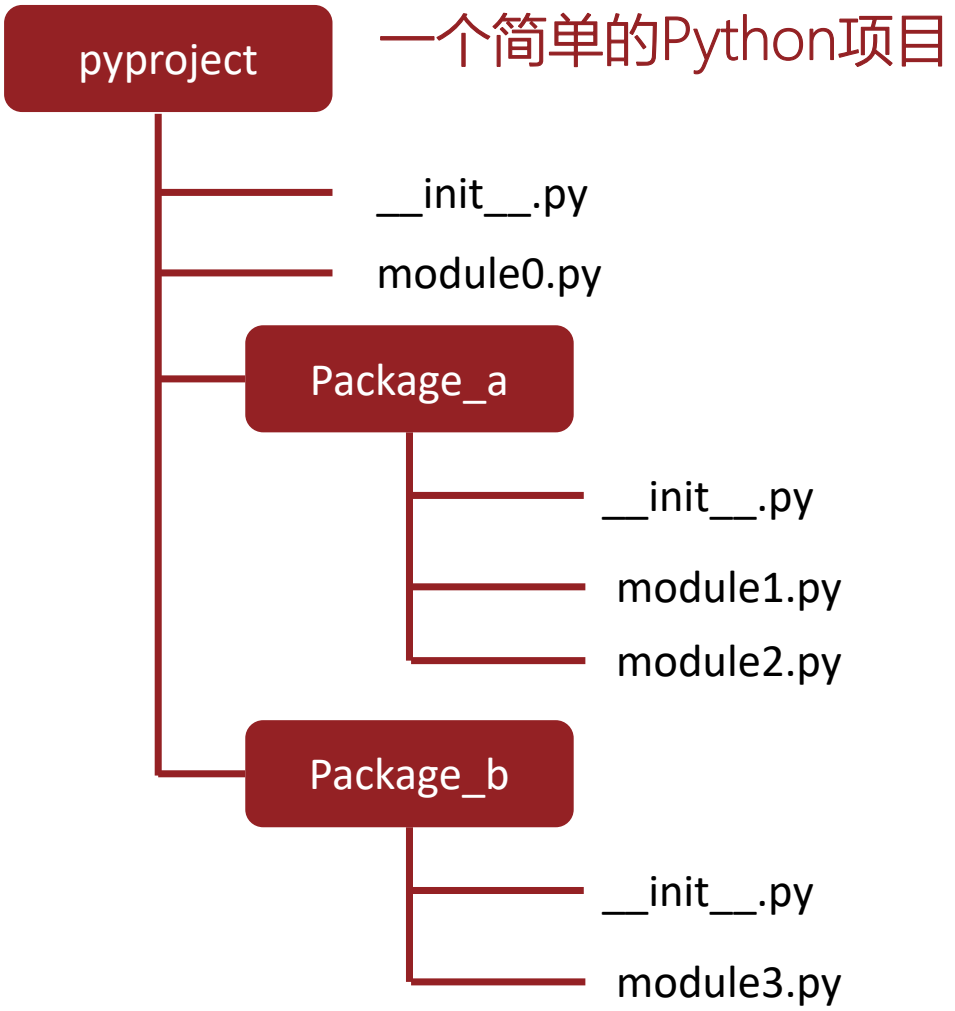
- 模块的执行环境：模块包含变量、函数、类以及其他的模块（如果导入的话），而函数也有自己的本地变量

模块内的情况及模块的交互



模块的作用

- 模块是Python中的最高级别组织单元，它将程序代码和数据封装起来以便重用
- 模块的三个角色：
  1. 代码重用
  2. 系统命名空间的划分（模块可理解为变量名的封装，即模块就是命名空间）
  3. 实现共享服务和数据



数据分析领域常用的包

- 数据库

类别	Python
MySQL	mysql-connector-python(官方)
Oracle	cx_Oracle
MongoDB	pymongo
ODBC	pyodbc

- IO类

类别	Python
excel	xlsxWriter, pandas.(from/to)_excel, openpyxl
csv	csv.writer
json	json
图片	PIL

- 描述性统计类

类别	Python
描述性统计汇总	scipy.stats.describe
均值	scipy.stats.gmean(几何平均数), scipy.stats.hmean(调和平均数), numpy.mean, numpy.nanmean, pandas.Series.mean
中位数	numpy.median, numpy.nanmediam, pandas.Series.median
众数	scipy.stats.mode, pandas.Series.mode
分位数	numpy.percentile, numpy.nanpercentile, pandas.Series.quantile
标准差	scipy.stats.std, scipy.stats.nanstd, numpy.std, pandas.Series.std
方差	numpy.var, pandas.Series.var
变异系数	scipy.stats.variation
协方差	numpy.cov, pandas.Series.cov
(Pearson)相关系数	scipy.stats.pearsonr, numpy.corrcoef, pandas.Series.corr
峰度	scipy.stats.kurtosis, pandas.Series.kurt
偏度	scipy.stats.skew, pandas.Series.skew
直方图	numpy.histogram, numpy.histogram2d, numpy.histogramdd

- 文本基本操作

类别	Python
tokenize	nltk.tokenize(英), jieba.tokenize(中)
stem	nltk.stem
stopwords	stop_words.get_stop_words
中文分词	jieba.cut, smallseg, Yaha, finalseg, genius
TFIDF	gensim.models.TfidfModel

- 回归

类别	Python
普通最小二乘法回归(ols)	statsmodels.ols, sklearn.linear_model.LinearRegression
广义线性回归(gls)	statsmodels.gls

- 假设检验

类别	Python
t检验	statsmodels.stats.ttest_ind, statsmodels.stats.ttost_ind, statsmodels.stats.ttost.paired; scipy.stats.ttest_1samp, scipy.stats.ttest_ind, scipy.stats.ttest_ind_from_stats, scipy.stats.ttest_rel
Pearson相关系数检验	scipy.stats.pearsonr

- 时间序列

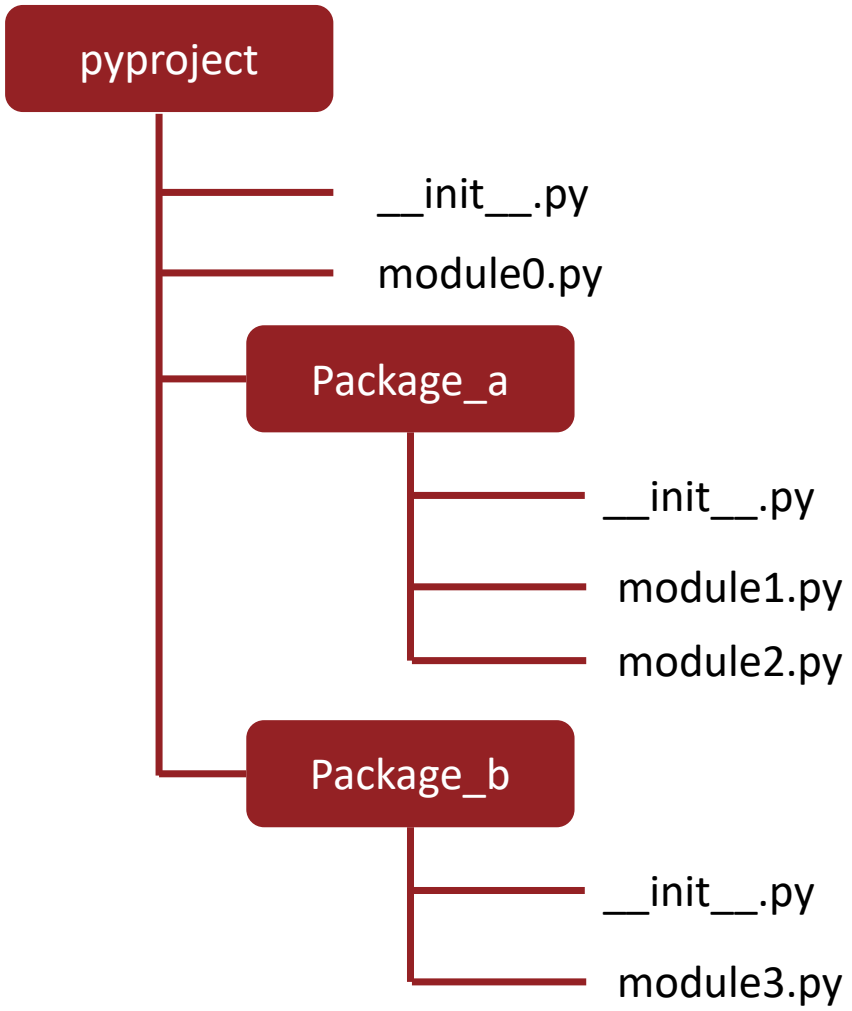
类别	Python
AR	statsmodels.ar_model.AR
ARIMA	statsmodels.arima_model.arima
VAR	statsmodels.var_model.var

- 支持向量机（SVM）

类别	Python
支持向量分类器（SVC）	sklearn.svm.SVC
非支持向量分类器（nonSVC）	sklearn.svm.NuSVC
线性支持向量分类器(Lenear SVC)	sklearn.svm.LinearSVC

模块的导入

- 导入从本质上讲，就是在一个文件中载入另一个文件，并且能够读取那个文件的内容。一个模块内的内容通过这样的方法其属性（object，attribute）能够被外界使用



模块导入的方式

① import x
② from x import *
③ from x import a,b,c

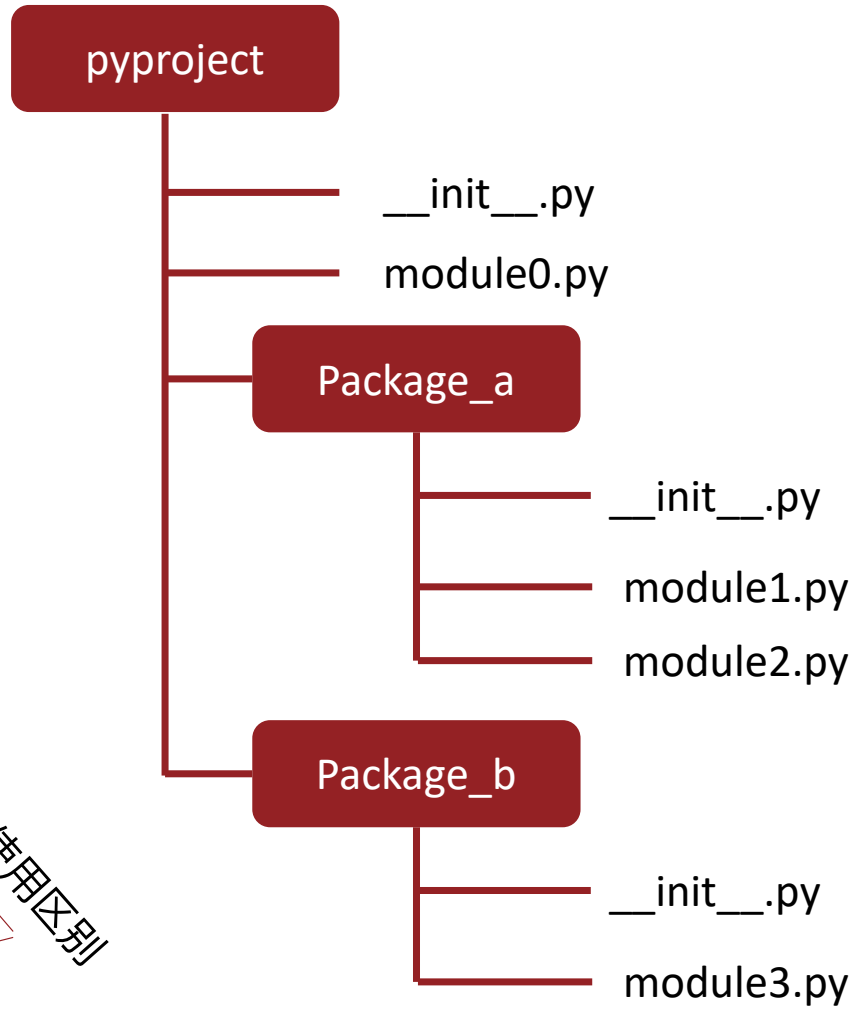
```
import Package_a.module1
import Package_b.module3
from Package_a.module1 import *

import module0
from module0 import *
```

```
import Package_a.module1 as m1
t = m1.Test()
```

```
from Package_a.module1 import *
t = Test()
```

注意使用区别



导入整个模块

- 模块的方式非常简单，在"import"后加上"模块名称"就可以了。

```
import numpy
```

- 通过这一句，计算机就在指定的位置找到了"numpy.py"文件，并准备好该文件拥有的之后会用到的函数和属性。在导入"numpy"后，我们就可以通过点符号"."连接模块名称和函数名，使用该模块中的函数和属性。

```
numpy.sqrt(2)
```

```
1.4142135623730951
```

别名

- 指定的方式是采用"import 模块名称 as 别名"。我们可以将"numpy"简记为"np"，并且在调用时，直接使用"np"就可以：

```
import numpy as np
np.sqrt(2)
```

```
1.4142135623730951
```

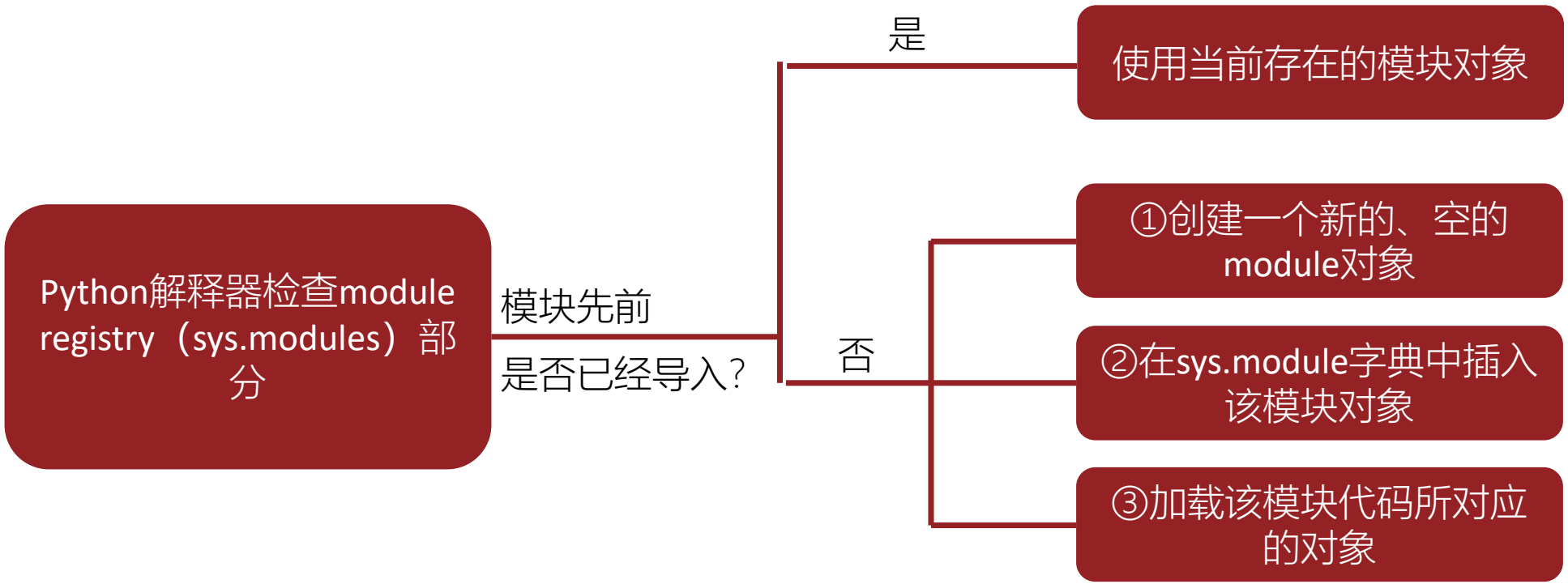
只导入某个对象

- 此过程的实现方式是“from 模块名称 import 函数名称"，但是要注意我们只拿到了某个具体的工具，而没有拿到整个工具箱

```
from numpy import sqrt
sqrt(2)
```

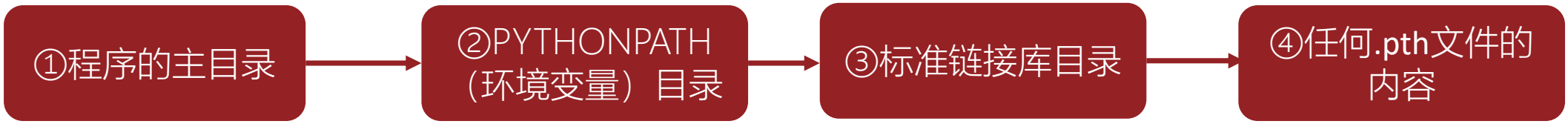
```
1.4142135623730951
```

import模块时，Python都做了哪些事情?



- 然后在新的模块命名空间、执行该模块代码对象（code object）。
  所有由该代码指定的变量均可以通过该模块对象引用

import搜索的路径顺序



- 以上四个组件组合起来就变成了 sys.path，其保存了模块搜索路径在机器上的实际配置，可以通过打印内置的sys.path 列表来查看这些路径
- 搜索路径①和③是系统自定义的，而②和④可以用于拓展路径，从而加入自己的源代码目录

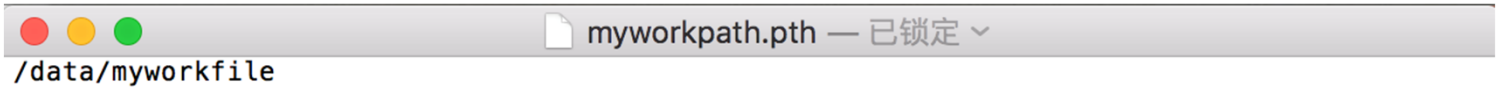
通过.pth文件修改搜索路径

1. 通过 site 模块的 getsitepackages() 函数获得包文件位置

```
import site
site.getsitepackages()
```

```
['//anaconda/envs/py27/lib/python2.7/site-packages',
 '//anaconda/envs/py27/lib/site-python']
```

2. 新建 myworkpath.pth 文件，列出模块搜索路径



3. 将 myworkpath.pth 文件放在包文件目录下

4. 查看路径是否添加成功

```
import sys
sys.path
```

```
['',
 '//anaconda/envs/py27/lib/python27.zip',
 '//anaconda/envs/py27/lib/python2.7',
 '//anaconda/envs/py27/lib/python2.7/plat-darwin',
 '//anaconda/envs/py27/lib/python2.7/plat-mac',
 '//anaconda/envs/py27/lib/python2.7/plat-mac/lib-scriptpackages',
 '//anaconda/envs/py27/lib/python2.7/lib-tk',
 '//anaconda/envs/py27/lib/python2.7/lib-old',
 '//anaconda/envs/py27/lib/python2.7/lib-dynload',
 '//anaconda/envs/py27/lib/python2.7/site-packages',
 '/data/myworkfile',
 '//anaconda/envs/py27/lib/python2.7/site-packages/setuptools-27.2.0-py2.7.egg',
 '//anaconda/envs/py27/lib/python2.7/site-packages/IPython/extensions',
 '/Users/Sid_Ian/.ipython']
```

临时修改模块搜索路径

- 使用 sys.path 在Python程序运行时临时修改模块搜索路径。

```
import sys
sys.path
```

```
['',
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '/data',
 '//anaconda/envs/py27/lib/pyt
 '//anaconda/envs/py27/lib/pyt
 '/Users/Sid_Ian/.ipython']
```

```
sys.path.append('/data')
sys.path
```

```
['',
 '//anaconda/envs/py27/lib/python27.zip',
 '//anaconda/envs/py27/lib/python2.7',
 '//anaconda/envs/py27/lib/python2.7/plat-darwin',
 '//anaconda/envs/py27/lib/python2.7/plat-mac',
 '//anaconda/envs/py27/lib/python2.7/plat-mac/lib-scriptpackages',
 '//anaconda/envs/py27/lib/python2.7/lib-tk',
 '//anaconda/envs/py27/lib/python2.7/lib-old',
 '//anaconda/envs/py27/lib/python2.7/lib-dynload',
 '//anaconda/envs/py27/lib/python2.7/site-packages',
 '/data',
 '//anaconda/envs/py27/lib/python2.7/site-packages/setuptools-27.2.0-py2.7.egg',
 '//anaconda/envs/py27/lib/python2.7/site-packages/IPython/extensions',
 '/Users/Sid_Ian/.ipython',
 '/data']
```

利用csv模块导入数据

- 我们将使用一份包含5000个样本的高中生社交网络信息数据集。
- 数据均采样于2006年，对应于高中三年级的毕业生。每个样本包含40个变量，其中"gradyear"、"gender"、"age"和"friends"四个变量分别代表毕业生的毕业年份、性别、年龄和好友数。



- 其余36个变量代表了36个词语，这36个词语代表了毕业生的5大基本特征：课外活动、时尚、宗教、浪漫和反社会行为。每个词语变量的取值代表对应的词语在高中生的社交网络服务平台发布的消息中出现的频次。

变量名	含义	变量名	含义
gradyear	毕业年份	gender	性别（F：女性；M：男性）
age	年龄	friends	好友数
basketball	篮球	football	足球
soccer	英式足球	softball	垒球
volleyball	排球	swimming	游泳
cheerleading	带领拉拉队	baseball	棒球
tennis	网球	sports	运动
cute	可爱的	sex	性
sexy	性感	hot	火辣
kissed	吻	dance	跳舞
band	乐队	marching	游行
music	音乐	rock	摇滚
god	上帝	church	教堂
jesus	耶稣	bible	圣经
hair	头发	dress	服装
blonde	金发女郎	mall	商业街
shopping	购物	clothes	衣服
hollister	美国时尚休闲大品牌	abercrombie	美国青少年最青睐的品牌
die	死亡	death	死亡
drunk	醉酒	drugs	毒品

- 通过“csv”模块导入数据集文件 teenager_sns.csv 中的数据

  - 导入csv模块

```
import csv
```

  - 使用 `open()` 函数打开文件，保存文件对象

```
f = open('teenager_sns.csv')
```

  - 调用 `csv.reader()` 函数生成reader对象

```
csvreader = csv.reader(f)
```

  - 使用 `list()` 函数将reader对象转换为列表

```
teenager_sns = list(csvreader)
```

- 读完数据后，我们得到一个二维列表"teenager_sns"，列表中的一个元素为文件中的一行，第一个元素为变量名列表，之后的一个元素对应着一个样本。我们以第一个元素和第二个元素为例：

```
print teenager_sns[0]
print teenager_sns[1]
```

```
['gradyear', 'gender', 'age', 'friends', 'basketball', 'football', 'soccer', 'softl
['2006', 'M', '18.982', '7', '0', '0', '0', '0', '0', '0', '0', '0', '0', '0',
```

- 我们发现所有的变量值都为字符型，所以在后面的分析中需要对数据进行类型转换。在导入数据之后，我们首先要做的事情就是初探我们的数据。

表2 某三个毕业生样本

gradyear	gender	age	friends	basketball	football	soccer	softball	volleyball	swimming	cheerleading	baseball	tennis	sports	cute
2006	M	18.982	7	0	0	0	0	0	0	0	0	0	0	0
2006	F	18.801	0	0	1	0	0	0	0	0	0	0	0	1
2006	M	18.335	69	0	1	0	0	0	0	0	0	0	0	0

缺失值处理

- 接下来，分析一下数据集中的 age 变量，看看美国高中毕业生的年龄特征。为了得到完整的 age 列表，使用以下方式：

```
age = []
for row in teenager_sns:
    age.append(row[2])
del age[0]
print age[0:10]
```

```
['18.982', '18.801', '18.335', '18.875', '18.995', '', '18.93', '18.322', '19.055', '18.708']
```

缺失值处理

- 我们先来看看缺失值的占比情况

```
count = 0.0
for row in age:
    if row == '':
        count += 1
total = len(age)
print '缺失值占比为',count/total
```

```
缺失值占比为 0.1602
```

缺失值处理

- 计算得到的缺失值占比为0.1602，也就是说，大约每5个样本里就有一个样本有缺失值，这样的占比还是比较大的，所以我们不能直接删除存在缺失值的样本，直接删除会影响数据的客观性和结果的准确性。

- 一种非常合理的方式是使用均值对缺失值进行填补。

缺失值处理

- 下面，我们对数据集中的毕业生年龄进行分析，我们使用Python内置的 max() 和 min() 函数得到 age_without_missing 中的最大值和最小值：

```
print max(age_without_missing)
print min(age_without_missing)
```

```
106.927
5.194
```

- 这种偏离变量实际范围的值称为异常值。我们将在类的部分对异常值进行检测和处理。

Pandas模块

- Pandas是python的一个数据分析包，
- Pandas最初被作为金融数据分析工具而开发出来，Pandas为时间序列分析提供了很好的支持。
- Pandas是基于NumPy 的一种工具。
- Pandas 纳入了大量函数和一些标准的数据模型，提供了高效地操作大型数据集所需的工具，提供了大量能使我们快速便捷地处理数据的函数和方法，它是使Python成为强大而高效的数据分析环境的重要因素之一。

利用pandas导入数据

- 利用pandas将数据存入一个DataFrame数据结构中

```
import pandas as pd
teenager_sns = pd.read_csv('./teenager_sns.csv')
print teenager_sns
```

```
      gradyear gender     age  friends  basketball  football  soccer  \
0         2006      M  18.982        7           0         0       0
1         2006      F  18.801        0           0         1       0
2         2006      M  18.335       69           0         1       0
3         2006      F  18.875        0           0         0       0
4         2006    NaN  18.995       10           0         0       0
5         2006      F     NaN      142           0         0       0
6         2006      F  18.930       72           0         0       0
7         2006      M  18.322       17           0         0       0
8         2006      F  19.055       52           0         0       0
9         2006      F  18.708       39           0         0       0
10        2006      F  18.543        8           0         0       0
11        2006      F  19.463       21           0         1       0
12        2006      F  18.097       87           0         0       0
13        2006    NaN     NaN        0           0         0       0
14        2006      F  18.398        0           0         0       0
15        2006    NaN     NaN        0           0         0       0
```

- 得到了一个pandas内置的DataFrame类的实例

```
print type(teenager_sns)
```

```
<class 'pandas.core.frame.DataFrame'>
```

- 提取age列，并得到其描述统计信息

```
teenager_sns['age'].describe()
```

```
count    4199.000000
mean       19.108433
std         6.116729
min         5.194000
25%        18.376000
50%        18.675000
75%        18.987000
max       106.927000
Name: age, dtype: float64
```

- 使用pandas内置的mean()和fillna()方法进行缺失值填补

```
age_avg = teenager_sns['age'].mean()
teenager_sns['age'].fillna(age_avg)
```

```
0        18.982000
1        18.801000
2        18.335000
3        18.875000
4        18.995000
5        19.108433
6        18.930000
7        18.322000
8        19.055000
9        18.708000
10       18.543000
11       19.463000
12       18.097000
13       19.108433
```

DataFrame数据类型

DataFrame是Pandas库的一种数据类型。DataFrame是一个行和列都具有标签的表格，它与Excel电子表格并无不同。

DataFrame使用非常方便，当你在处理二维表格数据时，都应该使用它们。DataFrame可由元组、列表、Python字典或另一个DataFrame构造出来

用列表产生DataFrame变量data

- from plotly.offline import plot
- from plotly import figure_factory as FF
- import pandas as pd

- data=pd.DataFrame([["2050921018","詹延峰","计算数学",65,85,76], ["2050921036","李小鹏","金融学类",86,95,85], ["2050921039","裴凡法","经济学类",86,95,65], ["2040912116","茅舒瑶","社会保障",90,95,100], ["2050912017","陈见影","化学工程",62,75,92], ["2050912064","梅钦钦","材料科学",87,95,80], ["2050109153","王影平","大气科学",86,89,72], ["2050151003","韩平医","化学工程",82,99,60]], columns=("学号","姓名","专业","笔试","平时","实验"))

- table = FF.create_table(data)　#用plotly产生输出表格
- plot(table, show_link=False)

程序输出

学号	姓名	专业	笔试	平时	实验
2050921018	詹延峰	计算数学	65	85	76
2050921036	李小鹏	金融学类	86	95	85
2050921039	裴凡法	经济学类	86	95	65
2040912116	茅舒瑶	社会保障	90	95	100
2050912017	陈见影	化学工程	62	75	92
2050912064	梅钦钦	材料科学	87	95	80
2050109153	王影平	大气科学	86	89	72
2050151003	韩平医	化学工程	82	99	60

用DataFrame计算总评分

- from plotly.offline import plot
- from plotly import figure_factory as FF
- import pandas as pd

- data=pd.DataFrame([["2050921018","詹延峰","计算数学",65,85,76], ["2050921036","李小鹏","金融学类",86,95,85], ["2050921039","裴凡法","经济学类",86,95,65], ["2040912116","茅舒瑶","社会保障",90,95,100], ["2050912017","陈见影","化学工程",62,75,92], ["2050912064","梅钦钦","材料科学",87,95,80], ["2050109153","王影平","大气科学",86,89,72], ["2050151003","韩平医","化学工程",82,99,60]], columns=("学号","姓名","专业","笔试","平时","实验"))
- data["总评成绩"]=data["笔试"]*0.5+data["平时"]*0.25+data["实验"]*0.25
- table = FF.create_table(data)
- plot(table, show_link=False)

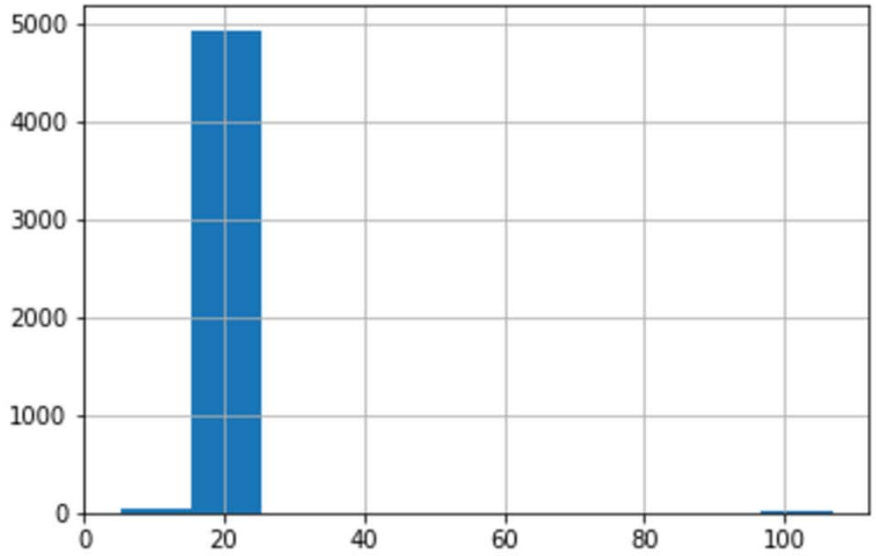
程序输出

学号	姓名	专业	笔试	平时	实验	总评成绩
2050921018	詹延峰	计算数学	65	85	76	72.75
2050921036	李小鹏	金融学类	86	95	85	88.0
2050921039	裴凡法	经济学类	86	95	65	83.0
2040912116	茅舒瑶	社会保障	90	95	100	93.75
2050912017	陈见影	化学工程	62	75	92	72.75
2050912064	梅钦钦	材料科学	87	95	80	87.25
2050109153	王影平	大气科学	86	89	72	83.25
2050151003	韩平医	化学工程	82	99	60	80.75

- 使用pandas内置的hist() 方法画出缺失值填补后的年龄分布图

```
teenager_sns['age'].hist()
```

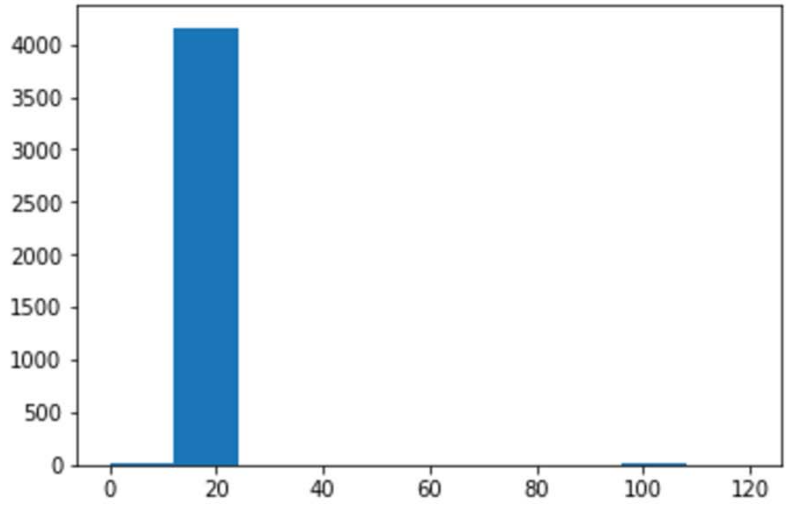
```
<matplotlib.axes._subplots.AxesSubplot at 0x111a59f50>
```



利用 matplotlib 画出年龄的分布图

```
# 画出年龄的分布图
import pandas as pd # 结构化数据处理
import matplotlib.pyplot as plt # 绘图
import numpy as np # 转化成数组
%matplotlib inline

teenager_sns = pd.read_csv('./teenager_sns.csv') # 读取数据为DataFrame
age = np.array(teenager_sns['age']) # 将Series转换为array
plt.hist(age,range=[0,120])
```



- 结合 pandas 和 numpy 进行绘图
- 绘制分布图有利于观察出异常情况

总结

- 什么是模块?
- 数据分析领域常用的包
- 搜索路径管理，模块的导入
- 利用csv模块导入数据、缺失值处理
- 利用pandas导入数据及缺失值处理、使用matplotlib绘制分布图



第十二部分 Python的GUI编程

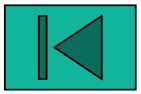
- GUI程序开发简介
- Tkinter与主要组件
- Turtle库简介

一、概述

- 使用图形用户界面（GUI），可以使程序更友好。
- Python作为一种“胶水性”语言，提供了众多GUI开发库的绑定，适合快速开发GUI。
- 目前大部分开发库还不支持Python 3，且库的更新速度不是很快。如果使用的是Python 3，可选择的开发库就不是很多了。
- 目前常用的开发库有：Python内置的Tkinter、Graphics、Turtle等，以及非常强大的PyQT等。如果使用的是Python 2，还有如wxPython、PyGTK、PMW等开发库可以选择。

介绍以下图形库

- Tkinter—Python 标准GUI
- Turtle —Python内置的图形库。



二、GUI程序开发简介

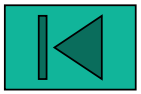
- 日常使用的大量客户端程序都属于GUI程序，即在一个界面上有很多功能块，包括：标签、按钮、输入框、菜单等。
- 开发GUI程序，首先需要有一块空白画面，然后在其上划分出不同的区域，放上不同的模块，最后完成每一个模块的功能。

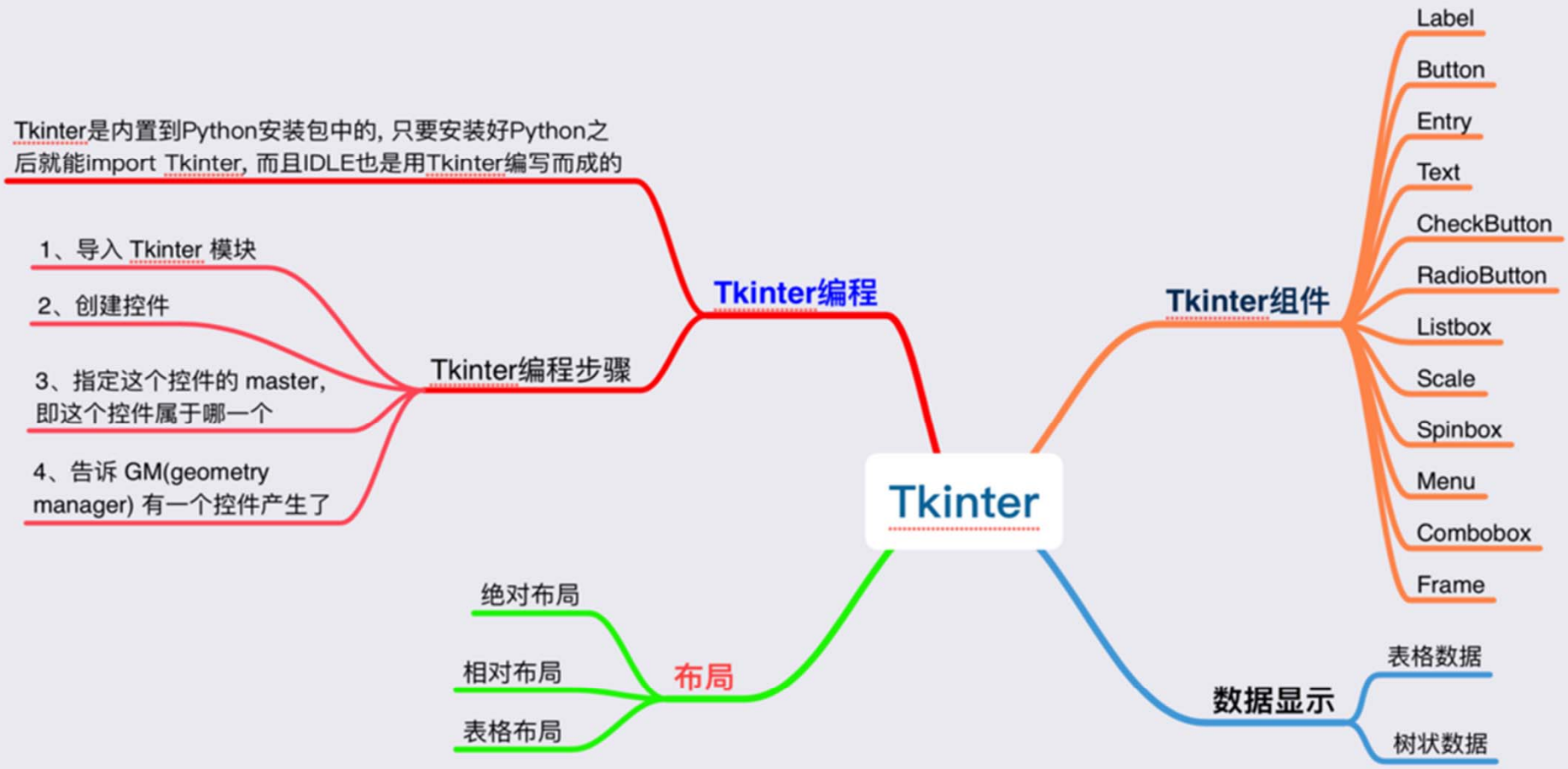
GUI程序开发基本步骤和过程：

- 首先要有底层的根窗口对象，在其基础上创建一个个小窗口对象。每一个窗口都是一个容器，可将所需的组件置于其中。每种GUI开发库都拥有大量的组件，一个GUI程序就是由各种不同功能的组件组成的，而根窗口对象则包含了所有组件。
- 组件本身也可以作为一个容器，它可以包含其他组件，如下拉框。这种包含其他组件的称为父组件，反之，包含在其他组件中的组件称为子组件。

- 构建出了GUI程序的每一个组件，只完成了程序的界面，但此时只能看不能用，需要给每一个组件**添加对应的功能**。
- 使用GUI程序时，会进行各种操作，如鼠标移动，按下或松开鼠标键，按下键盘按键等，这些操作称为**事件**。每个组件对应着一些行为，如在文本框中输入文本，单击按钮等，这些也称为事件。
- GUI程序启动的时候就一直监控这些事件，当某个事件发生的时候，就进行对应的处理并返回相应的结果。因此，GUI程序是由这一整套事件驱动的，这个过程称为**事件驱动处理**。

- 一个事件发生后，GUI程序捕获该事件、作出对应的处理并返回结果的过程称为回调。如计算器程序，单击了“=”按钮之后，便产生了一个事件，需要计算最终的结果，程序便开始对算式进行计算，返回最终结果并显示出来。这个计算并显示结果的过程即为回调。
- 当为程序需要的每一个事件都添加完相应的回调处理之后，整个GUI就完成了。




Tkinter
Tkinter编程
Tkinter是内置到Python安装包中的，只要安装好Python之后就能import Tkinter，而且IDLE也是用Tkinter编写而成的
Tkinter编程步骤
1、导入 Tkinter 模块
2、创建控件
3、指定这个控件的 master，即这个控件属于哪一个
4、告诉 GM(geometry manager) 有一个控件产生了
布局
绝对布局
相对布局
表格布局
Tkinter组件
Label
Button
Entry
Text
CheckButton
RadioButton
Listbox
Scale
Spinbox
Menu
Combobox
Frame
数据显示
表格数据
树状数据

三、Tkinter与主要组件

- **Tk是Python默认的工具集（即图形库），Tkinter是Tk的Python接口**，通过Tkinter可以方便地调用Tk进行图形界面开发。
- Tk与其他开发库相比，不是最强大的，模块工具也不是非常的丰富。但它非常简单，所提供的功能开发一般的应用也完全够用了，且能在大部分平台上运行。
- Python自带的IDEL也是用Tkinter开发的。
- Tkinter的不足之处是**缺少合适的可视化界面设计工具，需要通过代码来完成窗口设计和元素布局。**

- Tkinter中提供了较为丰富的控件，完全能满足基本的GUI程序的需求。
- 由于Tkinter模块已经在Python中内置，使用之前，只需将其导入。
- 两种导入方式为：

import tkinter as tk—导入tkinter，但没引入任何组件，在使用时需要使用tk前缀，如需要引入按钮，则表示为：tk.Button。

from tkinter import *—将tkinter中的所有组件一次性引入。

1、在程序中使用Tkinter

- 利用Tkinter模块来引用Tk,构建和运行GUI程序，通常需要5步：
  ① 导入Tkinter模块；
  ② 创建一个顶层窗口；
  ③ 在顶层窗口的基础上构建所需要的GUI模块和功能；
  ④ 将每一个模块与底层程序代码关联起来；
  ⑤ 执行主循环。

Tkinter主要组件简介

组件	功能
Button	按钮。类似标签，但提供额外功能，如鼠标按下、释放及键盘操作事件
Canvas	画布。提供绘图功能（直线、椭圆、多边形、矩形），可以包含图形或位图
Checkbutton	选择按钮。一组方框，可以选择其中的任意个
Radiobutton	单选按钮。一组方框，其中只有一个可被选择
Entry	文本框。单行文字域，用来收集键盘输入
Frame	框架。包含其他组件的纯容器
Label	标签。用来显示文字或图片
Listbox	列表框。一个选项列表，用户可以从中选择
Menu	菜单。单击后弹出一个选项列表，用户可以从中选择
Menubutton	菜单按钮。用来包含菜单的组件（有下拉式、层叠式）
Message	消息框。类似于标签，但可以显示多行文本
Scale	进度条。线性“滑块”组件，可设定起始值和结束值，显示当前位置的精确值
Scrollbar	滚动条。对其支持的组件（文本域、画布、列表框、文本框）提供滚动功能
Text	文本域。多行文字区域，可用来收集（或显示）用户输入的文字
Toplevel	顶级。类似框架，但提供一个独立的窗口容器。

组件的共同属性：

- dimensions ： 尺寸
- colors：颜色
- fonts：字体
- anchors：锚
- relief styles：浮雕式
- bitmaps：显示位图
- cursors：光标的外形
- 说明：每种组件还有其各自特有的属性

注意：

- Tk使用了一种包管理器来管理所有的组件，当定义完组件之后，需要调用pack()**方法来控制组件的显示方式**，若不调用pack()方法，组件将不会显示。
- **在交互环境下**，编写Tkinter测试代码时，运行过Tk()（创建顶层窗口的函数）之后即进入主循环，可以看到顶层窗口。而**若是运行py文件**，一定要调用mainloop()方法进入主循环，方可看到顶层窗口。

创建GUI应用程序窗口代码模板：

```
from tkinter import *
tk = Tk()
#此处添加控件代码
...
tk.mainloop()
```

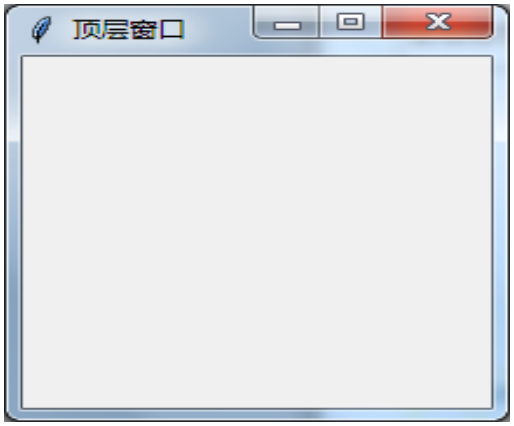
2、顶层窗口

- 顶层窗口也称为**根窗口**。
- 顶层窗口实际上是一个普通窗口，包括**一个标题栏和窗口管理器所提供的窗口装饰部分**，如最大化按钮等。
- 在一个Tkinter开发的应用程序中，只需要创建一个顶层窗口即可，且此窗口的创建必须是在其他窗口创建之前。

例：创建顶层窗口

```
#创建顶层窗口
from tkinter import *
root=Tk()
root.title("顶层窗口") #给窗口自定义名称，否则默认显示tk
root.mainloop()        #进入主循环，否则运行时将一闪而过看不到界面
```

执行结果：



3、标签

- 标签组件可以用来显示图片和文本，通过在文本中添加换行符来控制换行，也可以通过控制组件的大小实现自动换行。
- 例：使用标签（Label）编写一个程序，在程序主体中显示“Hello World!”。

程序及执行结果：

```
#使用Label实现Hello World程序
from tkinter import *
root=Tk()
root.title("顶层窗口")
label=Label(root, text="Hello World!")   #定义标签
label.pack() #调用pack方法
root.mainloop()
```

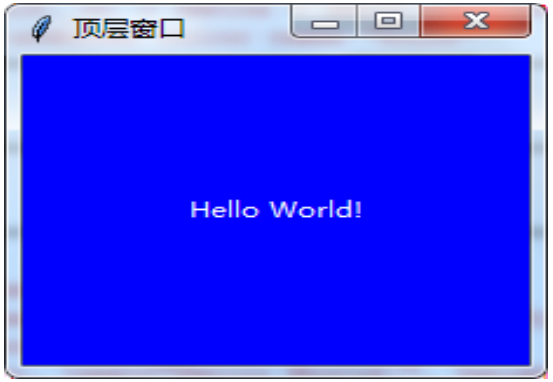


Label组件常用参数

参数	描述
height	组件的高度（所占行数）
width	组件的宽度（所占字符个数）
fg	前景字体颜色
bg	背景颜色
justify	多行文本的对齐方式，可选参数为： **LEFT、 CENTER、RIGHT**
padx	文本左右两侧的空格数（默认为1）
pady	文本上下两侧的空格数（默认为1）

增加了参数后的程序及执行结果：

```
#使用Label实现Hello World程序（增加了参数）
from tkinter import *
root=Tk()
root.title("顶层窗口")
label=Label(root, text="Hello World!", height=10, width=30, \
            fg="white", bg="blue")   #定义标签
label.pack() #调用pack方法
root.mainloop()
```



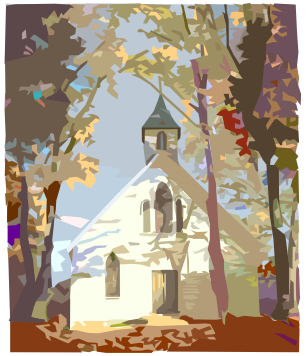

```
from tkinter import *
root=Tk()
root.title("顶层窗口")
label=Label(root,text="Hello world!",height=10,width=30,fg="white",bg="blue")
label.pack()
root.mainloop()
```

控件属性设置有三种方式：

*1.*创建对象时，指定宽度与高度

*2.*使用属性*width*和*height*来指定宽度与高度

*3.*使用*configure*或*config*方法来指定宽度与高度



```
from Tkinter import *
root = Tk()
one = Label(root,text ='helloworld',width = 30,height = 2)
one.pack()

two = Label(root,text =helloworld')
two['width'] = 30
two['height'] = 3
two.pack()

two = Label(root,text ='helloworld')
two.configure(width =30,height = 3)
two.pack()
```

```
from Tkinter import *
root = Tk()
one = Label(root,text ='helloworld',width = 30,height = 2)
one.pack()

two = Label(root,text =helloworld')
two['width'] = 30
two['height'] = 3
two.pack()

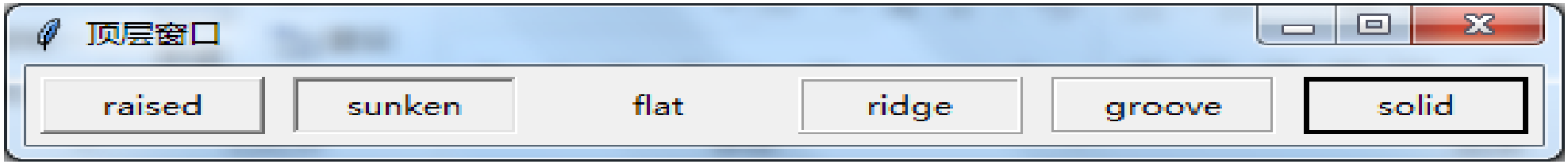
two = Label(root,text ='helloworld')
two.configure(width =30,height = 3)
two.pack()
```

4、框架

- 框架（Frame）相对于其他组件而言，它只是个**容器**，因为它没有方法，但它可以捕获键盘和鼠标的事件来进行回调。
- **框架一般用作包含一组控件的主体**，且可以定制外观。

例：创建不同样式的框架

```
#创建不同样式的框架
from tkinter import *
root=Tk()
root.title("顶层窗口")
for relief in [RAISED, SUNKEN, FLAT, RIDGE, GROOVE, SOLID]:
    f=Frame(root, borderwidth=2, relief=relief)   #定义框架
    #定义标签，且使用side参数设定排列方式
    Label(f, text=relief, width=10).pack(side=LEFT)
    #显示框架，并设定向左排列，x和y轴的宽度均为5个像素
    f.pack(side=LEFT, padx=5, pady=5)
root.mainloop()
```



```
from tkinter import *
root=Tk()
root.title("顶层窗口")
for relief in [RAISED,SUNKEN,FLAT,RIDGE,GROOVE,SOLID]:
    f=Frame(root,borderwidth=2,relief=relief)
    Label(f,text=relief,width=10).pack(side=LEFT)
    f.pack(side=LEFT,padx=5,pady=5)
root.mainloop()
```

5、按钮

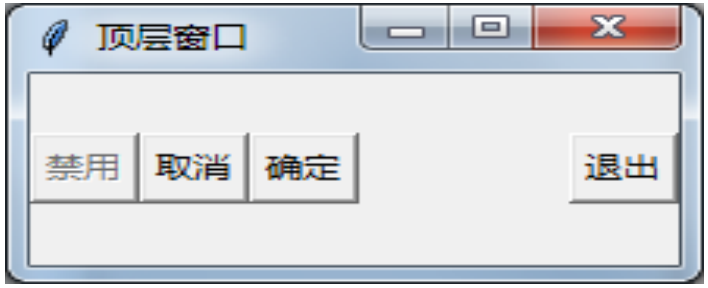
- 按钮组件（Button），方便与用户进行交互。
- 严格地说，按钮（Button）也可被看作标签，只是它可以捕获键盘和鼠标事件。
- 按钮可以**禁用**，禁用之后的按钮不能进行单击等任何操作。
- 如果将按钮放进**TAB群**中，就可以使用TAB键来进行跳转和定位。

例：创建按钮示例

```
#创建按钮
from tkinter import *
root=Tk()
root.title("顶层窗口")

#使用state参数来设定按钮的状态
Button(root, text="禁用", state=DISABLED).pack(side=LEFT)
Button(root, text="取消").pack(side=LEFT)
Button(root, text="确定").pack(side=LEFT)
Button(root, text="退出", command=root.quit).pack(side=RIGHT) #绑定了系统内置的退出回调

root.mainloop()
```



Button组件常用参数

参数	描述
height	组件的高度（所占行数）
width	组件的宽度（所占字符个数）
fg	前景字体颜色
bg	背景颜色
activebackground	按钮按下时的背景颜色
activeforeground	按钮按下时的前景颜色
justify	多行文本的对齐方式，可选参数为： **LEFT**、 **CENTER**、**RIGHT**
padx	文本左右两侧的空格数（默认为1）
pady	文本上下两侧的空格数（默认为1）
state	设置组件状态，默认为**NORMAL**，可设置为：**DISABLED**—禁用组件（必须大写）

增加了参数后的程序和执行结果：

```
#创建按钮（修改了参数）
from tkinter import *
root=Tk()
root.title("顶层窗口")

#使用state参数来设定按钮的状态
Button(root, text="禁用", state=DISABLED, height=2, width=10).pack(side=LEFT)
Button(root, text="取消", height=2, width=10, fg="blue").pack(side=LEFT)
Button(root, text="确定", height=2, width=10, fg="red").pack(side=LEFT)
Button(root, text="退出", height=2, width=10, fg="black", activebackground="blue",\
       activeforeground="yellow", command=root.quit).pack(side=RIGHT) #绑定了系统内置的退出回调

root.mainloop()
```



按下“退出”按钮：



6、输入框

- 一个GUI程序，接收用户的输入几乎是必不可少的。
- 输入框（Entry）组件就是用来接收用户输入的最基本的组件。
- 可以为输入框设置默认值，也可以禁止用户输入。
- 当用户输入的内容一行显示不下的时候，输入框会自动生成滚动条。

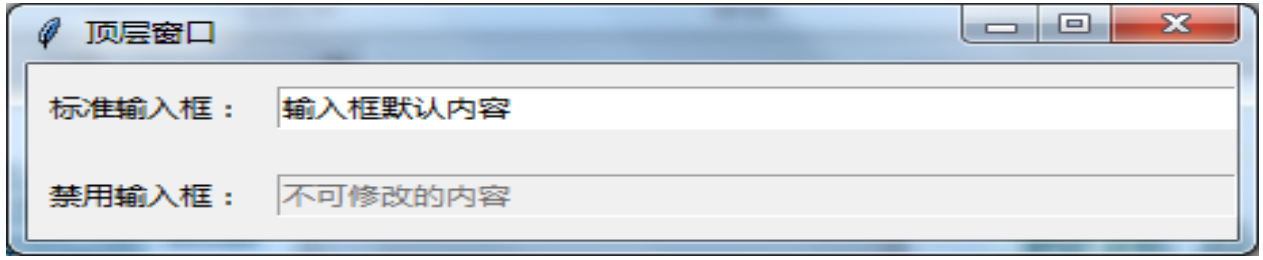
例：创建输入框示例

```
#创建输入框
from tkinter import *
root=Tk()
root.title("顶层窗口")

f1=Frame(root)   #定义框架
Label(f1, text="标准输入框：  ").pack(side=LEFT, padx=5, pady=10)
e1=StringVar()   #定义输入框内容
Entry(f1, width=50, textvariable=e1).pack(side=LEFT)  #基本输入框
e1.set('输入框默认内容')  #设置一般输入框默认内容
f1.pack()

f2=Frame(root)   #定义框架
e2=StringVar()
Label(f2, text="禁用输入框：  ").pack(side=LEFT, padx=5, pady=10)
Entry(f2, width=50, textvariable=e2, state=DISABLED).pack(side=LEFT)#禁用的输入框
e2.set("不可修改的内容")   #设置禁用的输入框内容
f2.pack()

root.mainloop()
```



Entry参数组件常用参数

参数	描述
height	组件的高度（所占行数）
width	组件的宽度（所占字符个数）
fg	前景字体颜色
bg	背景颜色
show	将Entry框中的文本替换为指定字符，用于输入密码等，如设置show="*"
state	设置组件状态，默认为NORMAL，可设置为：DISABLED—禁用组件，READONLY—只读

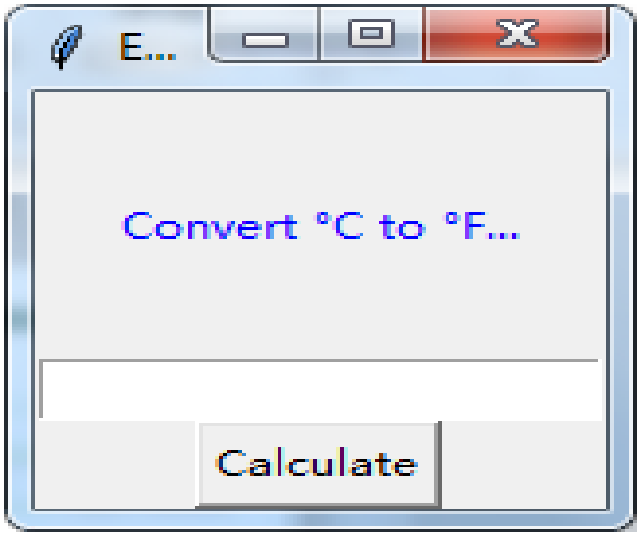
例：将摄氏度转换为华氏度

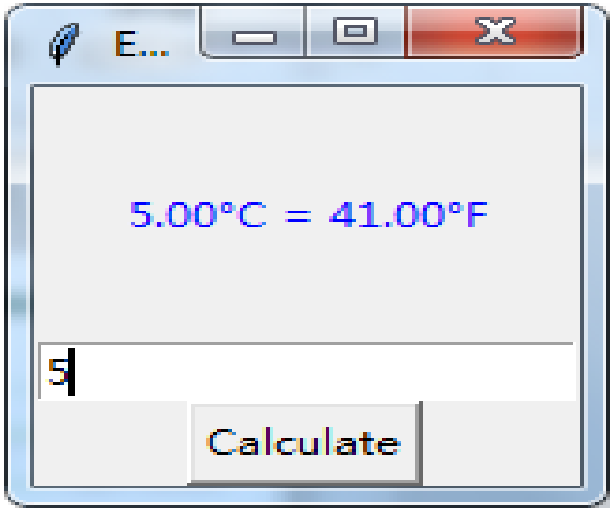
```
#将摄氏度转换为华氏度
import tkinter as tk

def cTofClicked():
    cd = float(entryCd.get())
    labelcTof.config(text = "%.2f°C = %.2f°F" %(cd, cd*1.8+32))

top = tk.Tk()
top.title("Entry Test")
labelcTof = tk.Label(top, text = "Convert °C to °F...", height = 5,\
                     width = 20, fg = "blue")
labelcTof.pack()
entryCd = tk.Entry(top, text = "0")
entryCd.pack()
btnCal = tk.Button(top, text = "Calculate", command = cTofClicked)
btnCal.pack()
top.mainloop()
```

程序执行:





7、单选按钮

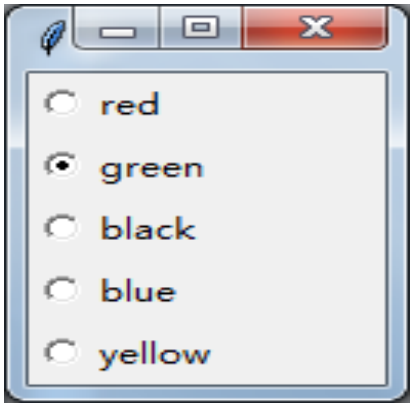
- 单选按钮（Radiobutton）是一组排他性的选择框，只能从该组中选择一个选项，当选择了其中一项之后便会取消其他选项的选择。
- 要想使用单选按钮，必须将这一组单选按钮与一个相同的变量关联起来，由用户为这个变量选择不同的值。

例：创建单选按钮示例

```
#创建单选按钮
from tkinter import *
root=Tk()
root.title("顶层窗口")

foo=IntVar()  #定义变量
for text, value in [('red', 1), ('green', 2), ('black', 3),\
                    ('blue', 4), ('yellow', 5)]:
    r=Radiobutton(root, text=text, value=value, variable=foo)
    r.pack(anchor=W) #向西对齐（可选参数：NSEW）
foo.set(2)  #设定默认选项

root.mainloop()
```



```
from tkinter import *
root=Tk()
root.title("顶层窗口")
foo=StringVar()
for text,value in [('red',"a"),('green',"b"),('black',"c"),('blue',"d"),('yellow',"e")]:
        r=Radiobutton(root,text=text,value=value,variable=foo)
        r.pack(anchor=W)
foo.set("d")

root.mainloop()
```

Radiobutton组件常用参数

参数	描述
variable	单选按钮索引变量，通过变量的值确定哪个单选按钮被选中。一组单选按钮使用同一个索引变量
value	单选按钮选中时变量的值
command	单选按钮选中时执行的命令（函数）

8、复选按钮

- **复选按钮之间没有互斥作用**，可以一次选择多个。
- 同样地，每一个按钮都需要与**一个变量相关联**，且每一个复选按钮关联的变量都是不同的。若像单选按钮一样，关联的是同一个按钮，则当选中其中一个的时候，会将所有按钮都选上。
- 可以给每一个**复选按钮绑定一个回调**，当该选项被选中时，执行该回调。

例：基本复选按钮示例

```
from tkinter import *
root=Tk()
root.title("顶层窗口")
l=[('red',1),('green',2),('black',3),('blue',4),('yellow',5)]

for text,value in l:
   foo=IntVar()
   c=Checkbutton(root,text=text,variable=foo)
   c.pack(anchor=W)
root.mainloop()
```

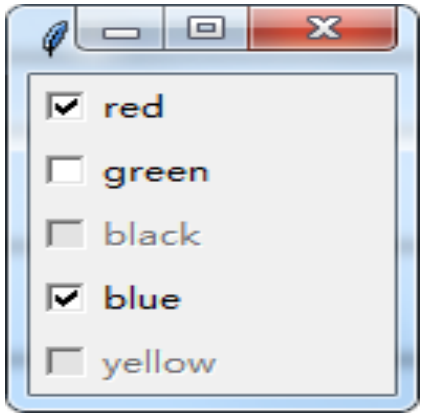
```
#创建基本复选按钮
from tkinter import *
root=Tk()
root.title("顶层窗口")

l=[('red', 1), ('green', 2), ('black', 3, ),('blue', 4),\
    ('yellow', 5)] #设定按钮的值
for text, value in l:
    foo=IntVar()
    c=Checkbutton(root, text=text, variable=foo)
    c.pack(anchor=W)

root.mainloop()
```



例：创建带“禁用”状态的复选按钮




```
#创建带“禁用”状态的复选按钮
from tkinter import *
root=Tk()
root.title("顶层窗口")

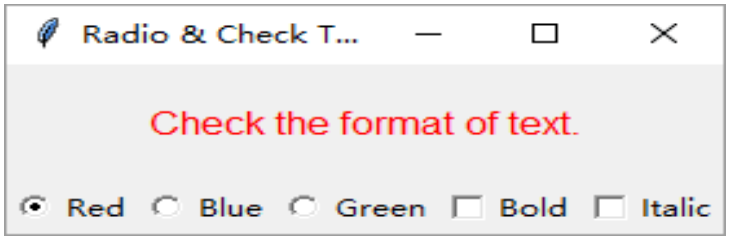
l=[('red', 1, NORMAL), ('green', 2, NORMAL), ('black', 3, DISABLED),\
   ('blue', 4, NORMAL), ('yellow', 5, DISABLED)] #设定按钮的值，默认是正常状态NORMAL
for text, value, status in l:
    foo=IntVar()
    c=Checkbutton(root, text=text, variable=foo, state=status) #使用state设定按钮状态
    c.pack(anchor=W)

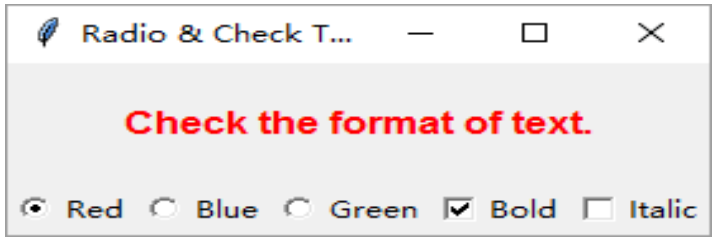
root.mainloop()
```

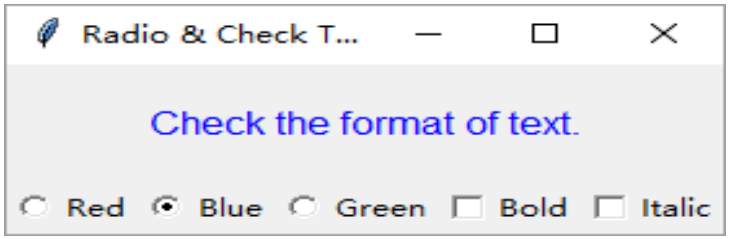
Checkbutton组件常用参数

参数	描述
variable	复选按钮索引变量，通过变量的值确定哪些复选按钮被选中。每个复选按钮使用不同的变量，使复选按钮之间相互独立
onvalue	复选按钮选中（有效）时变量的值
offvalue	复选按钮未选中（无效）时变量的值
command	复选按钮选中时执行的命令（函数）

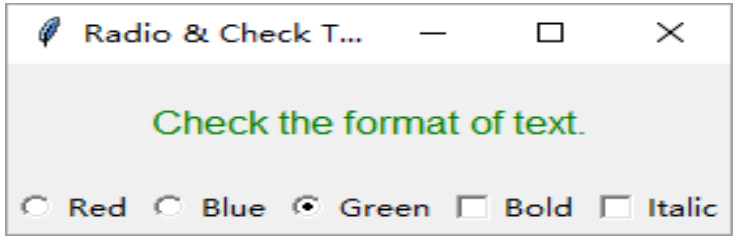
例：单选按钮及复选按钮综合应用

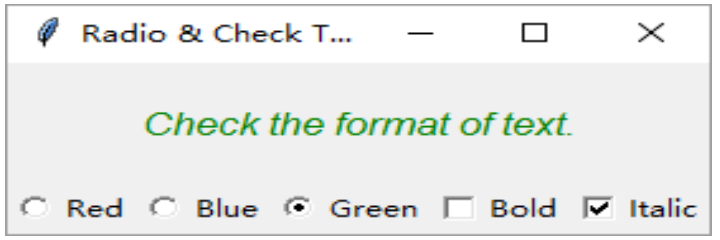












程序:

```
#单选及复选按钮综合应用
import tkinter as tk

def colorChecked():
    labelHello.config(fg = color.get())

def typeChecked():
    textType = typeBold.get() + typeItalic.get()
    if textType == 1:
        labelHello.config(font = ("Arial", 12, "bold"))
    elif textType == 2:
        labelHello.config(font = ("Arial", 12, "italic"))
    elif textType == 3:
        labelHello.config(font = ("Arial", 12, "bold italic"))
    else :
        labelHello.config(font = ("Arial", 12))

top = tk.Tk()
top.title("Radio & Check Test")

labelHello = tk.Label(top, text = "Check the format of text.", \
                      height = 3, font=("Arial", 12))
labelHello.pack()
```

```
color = tk.StringVar()
tk.Radiobutton(top, text = "Red", variable = color, \
               value = "red", command = colorChecked).pack(side = tk.LEFT)
tk.Radiobutton(top, text = "Blue", variable = color, \
               value = "blue", command = colorChecked).pack(side = tk.LEFT)
tk.Radiobutton(top, text = "Green", variable = color, \
               value = "green", command = colorChecked).pack(side = tk.LEFT)
color.set("yellow")

typeBold = tk.IntVar()
typeItalic = tk.IntVar()
tk.Checkbutton(top, text = "Bold", variable = typeBold, \
               onvalue = 1, offvalue = 0, command = typeChecked).pack(side = tk.LEFT)
tk.Checkbutton(top, text = "Italic", variable = typeItalic, \
               onvalue = 2, offvalue = 0, command = typeChecked).pack(side = tk.LEFT)

top.mainloop()
```

```
from tkinter import *
def colorChecked():
    labelHello.config(fg=color.get())
def typeChecked():
    textType=typeBold.get()+typeItalic.get()
    if textType == 1:
        labelHello.config(font=("Arial",12,"bold"))
    elif textType==2:
        labelHello.config(font=("Arial",12,"bold italic"))
    else:
        labelHello.config(font=("Arial",12))
 root=Tk()
root.title("顶层窗口")
labelHello = Label(root,text="Check the font ",height=3,font=("Arial",12))
labelHello.pack()
color=StringVar()
Radiobutton(root,text="Red",variable=color,value="red",command=colorChecked).pack(side=LEFT)
Radiobutton(root,text="Blue",variable=color,value="blue",command=colorChecked).pack(side=LEFT)
Radiobutton(root,text="Green",variable=color,value="green",command=colorChecked).pack(side=LEFT)
color.set("yellow")
typeBold=IntVar()
typeItalic=IntVar()
Checkbutton(root,text="Bold",variable=typeBold,onvalue=1,offvalue=0,command=typeChecked).pack(side=LEFT)
Checkbutton(root,text="Italic",variable=typeItalic,onvalue=1,offvalue=0,command=typeChecked).pack(side=LEFT)
root.mainloop()
```



9、消息

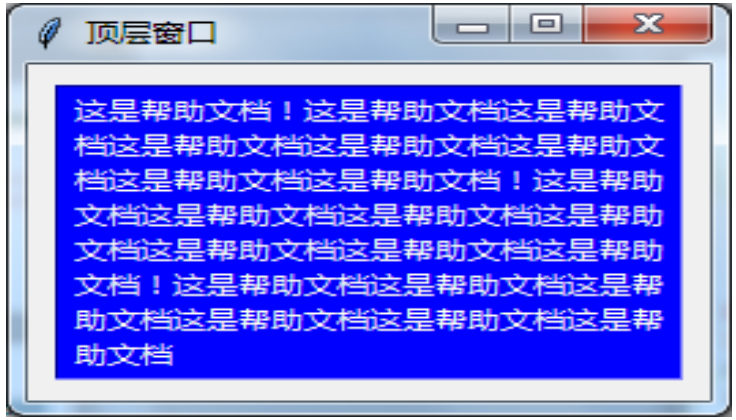
- 很多时候需要给用户发送消息，一般内容较多，如帮助信息等。
- 消息（Message）控件提供了显示多行文本的方法，且可以设置字体和背景色。Message组件提供了一个标准的方法，可以非常方便地实现这项功能。

例：创建消息示例

```
#创建消息
from tkinter import *
root=Tk()
root.title("顶层窗口")

Message(root, text="这是帮助文档！这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档"
        "这是帮助文档！这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档"
        "这是帮助文档！这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档这是帮助文档",
        bg='blue', fg='ivory', relief=GROOVE).pack(padx=10, pady=10)

root.mainloop()
```



10、滚动条

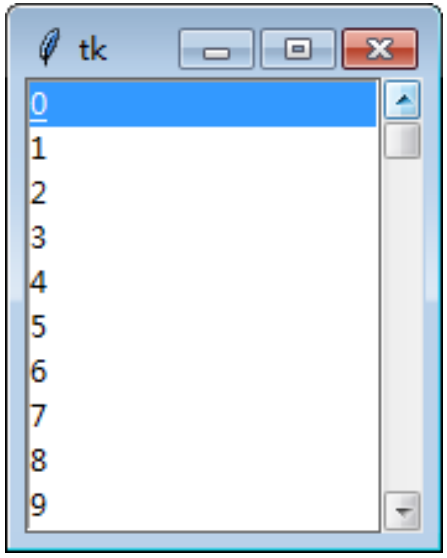
- 滚动条（Scrollbar）组件可以添加至任何一个组件，一些组件在界面显示不下时会自动添加滚动条，但可以使用滚动条组件来对其进行控制。
- 在用python进程GUI编程的时候，经常需要用到listbox这个组件，而如何控制Listbox组件的显示问题，则要用到Scrollbar组件。两个GUI组件之间动作的绑定——一种组件的动作会触发另外一种组件的相应动作
- Scrollbar也就是滚动条，基本上是和listboxs/canvases/text fields这些控件一起结合使用的

Scrollbar控件与其他控件的结合

- 将一个垂直方向的Scrollbar和listboxs/canvases/text fields这些控件结合起来，只需要按照下面的步骤即可：

  1. 将这些控件的yscrollcommand选项设置为scrollbar的set方法。
  2. 将scrollbar的command选项设置为这些控件的yview方法。

```
from Tkinter import *
master = Tk()
scrollbar = Scrollbar(master)
scrollbar.pack(side=RIGHT, fill=Y)
listbox = Listbox(master, yscrollcommand=scrollbar.set)
for i in range(1000):
    listbox.insert(END, str(i))
    listbox.pack(side=LEFT, fill=BOTH)
scrollbar.config(command=listbox.yview)
mainloop()
```

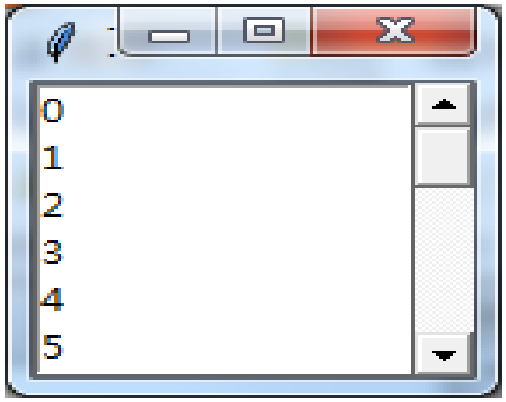


例：滚动条示例

```
#创建滚动条
from tkinter import *
root=Tk()
root.title("顶层窗口")

l=Listbox(root, height=6, width=15) #设定一个列表框组件
scroll=Scrollbar(root, command=l.yview) #定义滚动条，并绑定一个回调
l.configure(yscrollcommand=scroll.set)
l.pack(side=LEFT)
scroll.pack(side=RIGHT, fill=Y)
for item in range(20):
    l.insert(END, item)

root.mainloop()
```



11、列表框

- 列表框（ListBox）组件是一个选项列表。




```
#创建列表框
from tkinter import *
root=Tk()
root.title("顶层窗口")

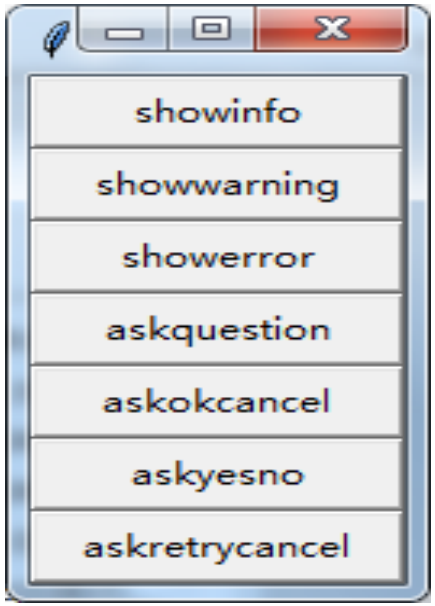
l=Listbox(root, width=15)
l.pack()
for item in ['apple', 'orange', 'peach', 'banana', 'melon']:
    l.insert(END, item)

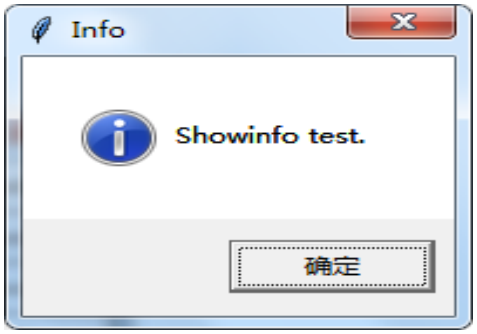
root.mainloop()
```

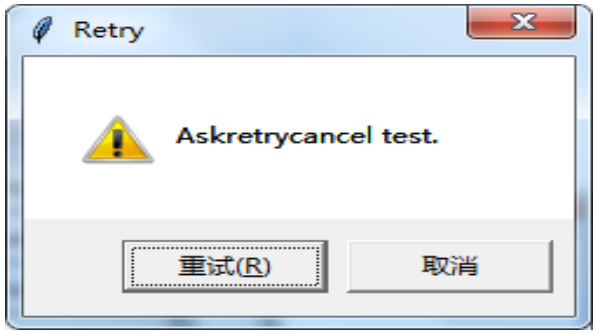
12、消息框

- 消息窗口（messagebox）用于弹出提示框向用户进行告警，或让用户选择下一步如何操作。
- 消息框包括很多类型，常用的有info、warning、error、yesno、okcancel等，包含不同的图标、按钮以及弹出提示音。

例：各种消息框应用


showinfo
showwarning
showerror
askquestion
askokcancel
askyesno
askretrycancel


Info
Showinfo test.
确定


Retry
Askretrycancel test.
重试(R)
取消

源程序:

```
#各种消息框应用
import tkinter as tk
from tkinter import messagebox as msgbox

def btn1_clicked():
    msgbox.showinfo("Info", "Showinfo test.")
def btn2_clicked():
    msgbox.showwarning("Warning", "Showwarning test.")
def btn3_clicked():
    msgbox.showerror("Error", "Showerror test.")
def btn4_clicked():
    msgbox.askquestion("Question", "Askquestion test.")
def btn5_clicked():
    msgbox.askokcancel("OkCancel", "Askokcancel test.")
def btn6_clicked():
    msgbox.askyesno("YesNo", "Askyesno test.")
def btn7_clicked():
    msgbox.askretrycancel("Retry", "Askretrycancel test.")
```

```
top = tk.Tk()
top.title("MsgBox Test")

btn1 = tk.Button(top, text = "showinfo", command = btn1_clicked)
btn1.pack(fill = tk.X)
btn2 = tk.Button(top, text = "showwarning", command = btn2_clicked)
btn2.pack(fill = tk.X)
btn3 = tk.Button(top, text = "showerror", command = btn3_clicked)
btn3.pack(fill = tk.X)
btn4 = tk.Button(top, text = "askquestion", command = btn4_clicked)
btn4.pack(fill = tk.X)
btn5 = tk.Button(top, text = "askokcancel", command = btn5_clicked)
btn5.pack(fill = tk.X)
btn6 = tk.Button(top, text = "askyesno", command = btn6_clicked)
btn6.pack(fill = tk.X)
btn7 = tk.Button(top, text = "askretrycancel", command = btn7_clicked)
btn7.pack(fill = tk.X)

top.mainloop()
```

13、绘图组件

- 绘图组件（Canvas—画布）可以在GUI中实现2D图形的绘制，相当于画图板。组件内置了多种绘图函数，可以通过简单的2D坐标绘制直线、矩形、圆形、多边形等。

例1：绘图应用

```
#绘图应用

import tkinter as tk

def drawCircle(self, x, y, r, **kwargs):
    print(kwargs) #两个星号的参数传进来的是字典形式，一个星号的参数传进来的是元组
    return self.create_oval(x-r, y-r, x+r, y+r, **kwargs)

top = tk.Tk()
top.title("Canvas Test")

cvs = tk.Canvas(top, width = 600, height = 400)
cvs.pack()

cvs.create_line(50, 50, 50, 300)

cvs.create_line(100, 50, 200, 300, fill = "red", dash = (4, 4), arrow = tk.LAST)

cvs.create_rectangle(200, 50, 400, 200, fill = "blue")

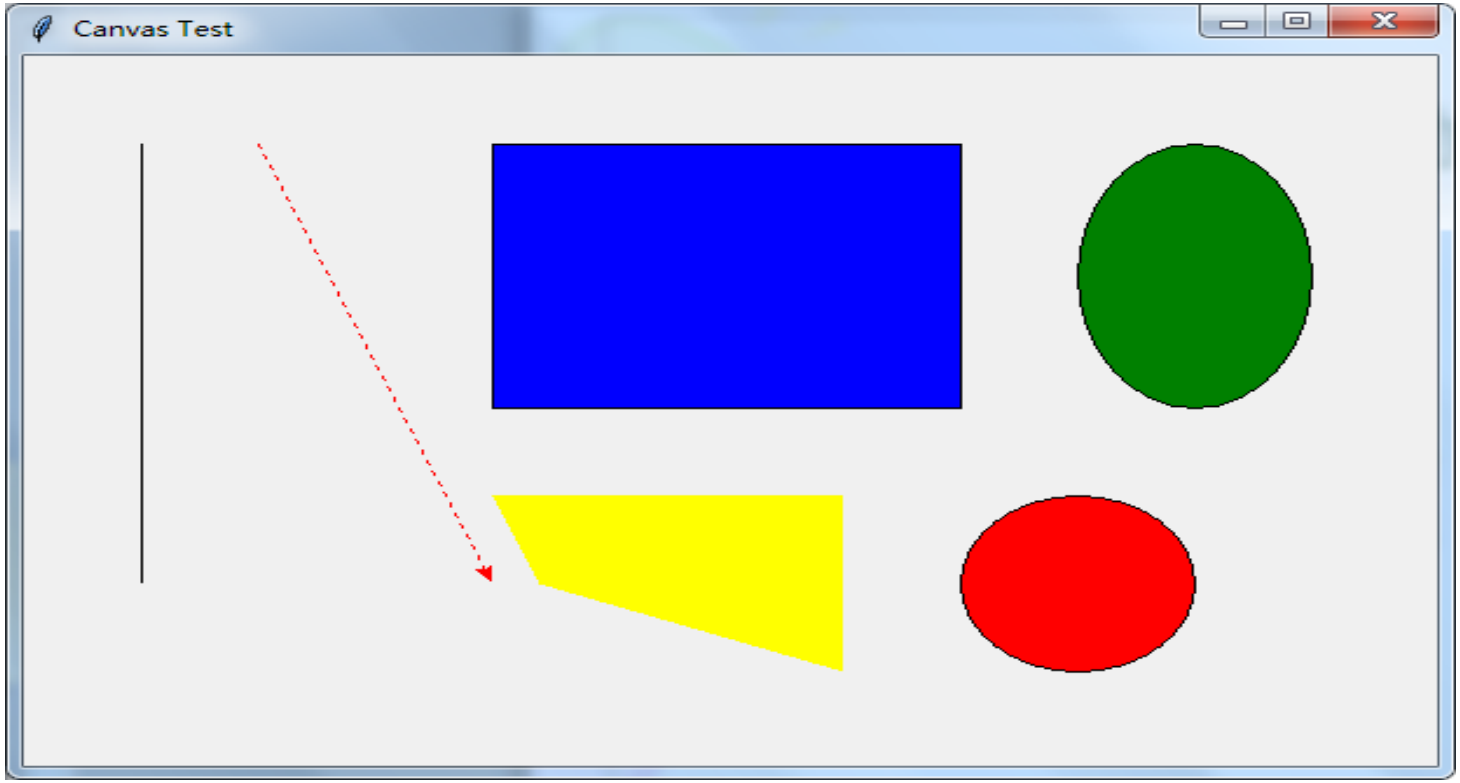
cvs.create_oval(450, 50, 550, 200, fill = "green" )

drawCircle(cvs, 450, 300, 50, fill = "red")

cvs.create_polygon(200, 250, 350, 250, 350, 350, 220, 300, fill="yellow")

top.mainloop()
```

程序执行：



说明：

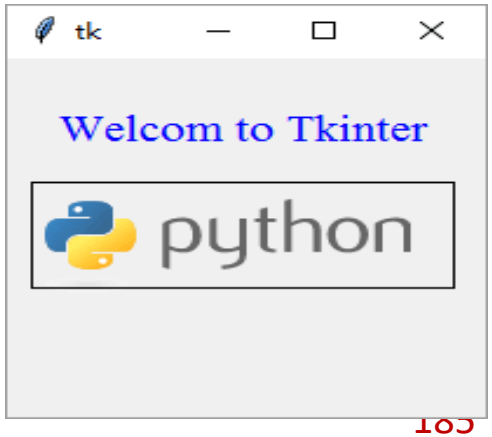
- 直线（line），即线段，通过两个端点定义。坐标顺序为x1、y1、x2、y2。
- 矩形（rectangle）通过对角线上的两个点来定义。
- 需要注意的是Canvas中没有画圆函数，过绘制椭圆间接实现了绘制圆形的函数drawCircle()。椭圆（oval）是通过外切矩形的对角线两点来定义的。

例：画布显示文字、图片和图形

```
#显示文字、图片
from tkinter import *

tk=Tk()
canvas=Canvas(tk, width=200, height=200)
canvas.pack()
canvas.create_text(100, 40, text="Welcom to Tkinter",\
                   fill="blue", font=("Times", 16))
myImage=PhotoImage(file="python_logo.gif")
canvas.create_image(10, 70, anchor=NW, image=myImage)
canvas.create_rectangle(10, 70, 190, 130)

tk.mainloop()
```



例：控制图形移动

```
#控制图形移动
from tkinter import *

def main():
    tk = Tk()
    canvas = Canvas(tk, width = 400, height = 400)
    canvas.pack()

    def moverectangle(event):
        if event.keysym == "Up":
            canvas.move(1,0,-5) #1-移动的图形，0-移动的横坐标，-5-移动的纵坐标
        elif event.keysym == "Down":
            canvas.move(1,0,5)
        elif event.keysym == "Left":
            canvas.move(1,-5,0)
        elif event.keysym == "Right":
            canvas.move(1,5,0)

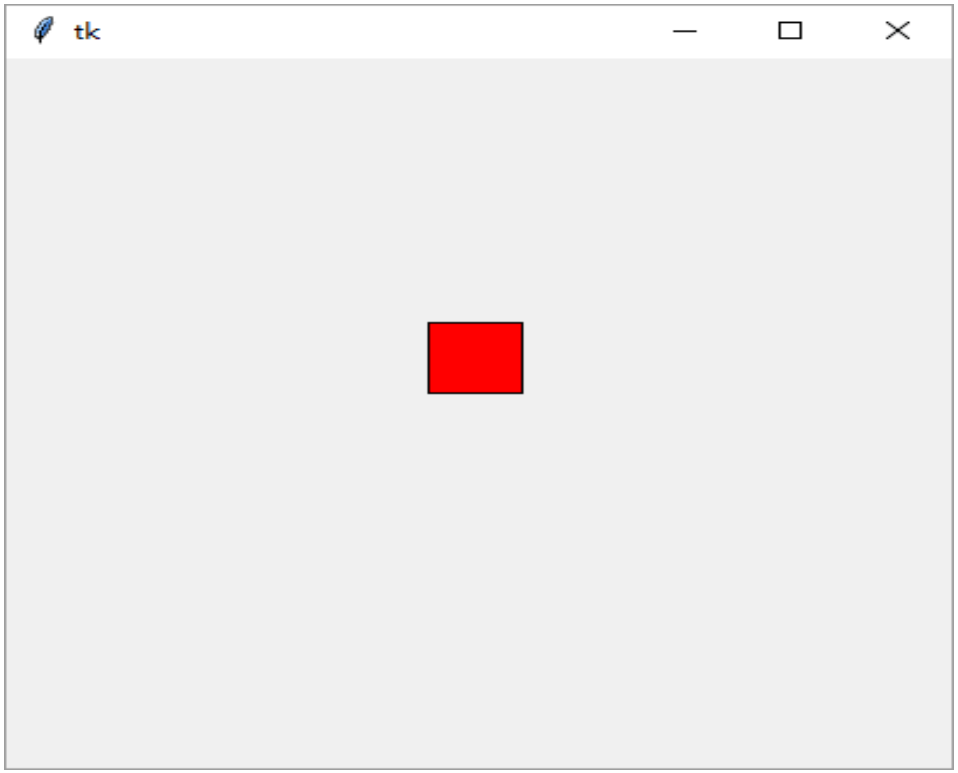
    canvas.create_rectangle(180,180,220,220,fill="red")
    canvas.bind_all("<KeyPress-Up>",moverectangle)
    canvas.bind_all("<KeyPress-Down>",moverectangle)
    canvas.bind_all("<KeyPress-Left>",moverectangle)
    canvas.bind_all("<KeyPress-Right>",moverectangle)

if __name__ == '__main__':
    main()
```

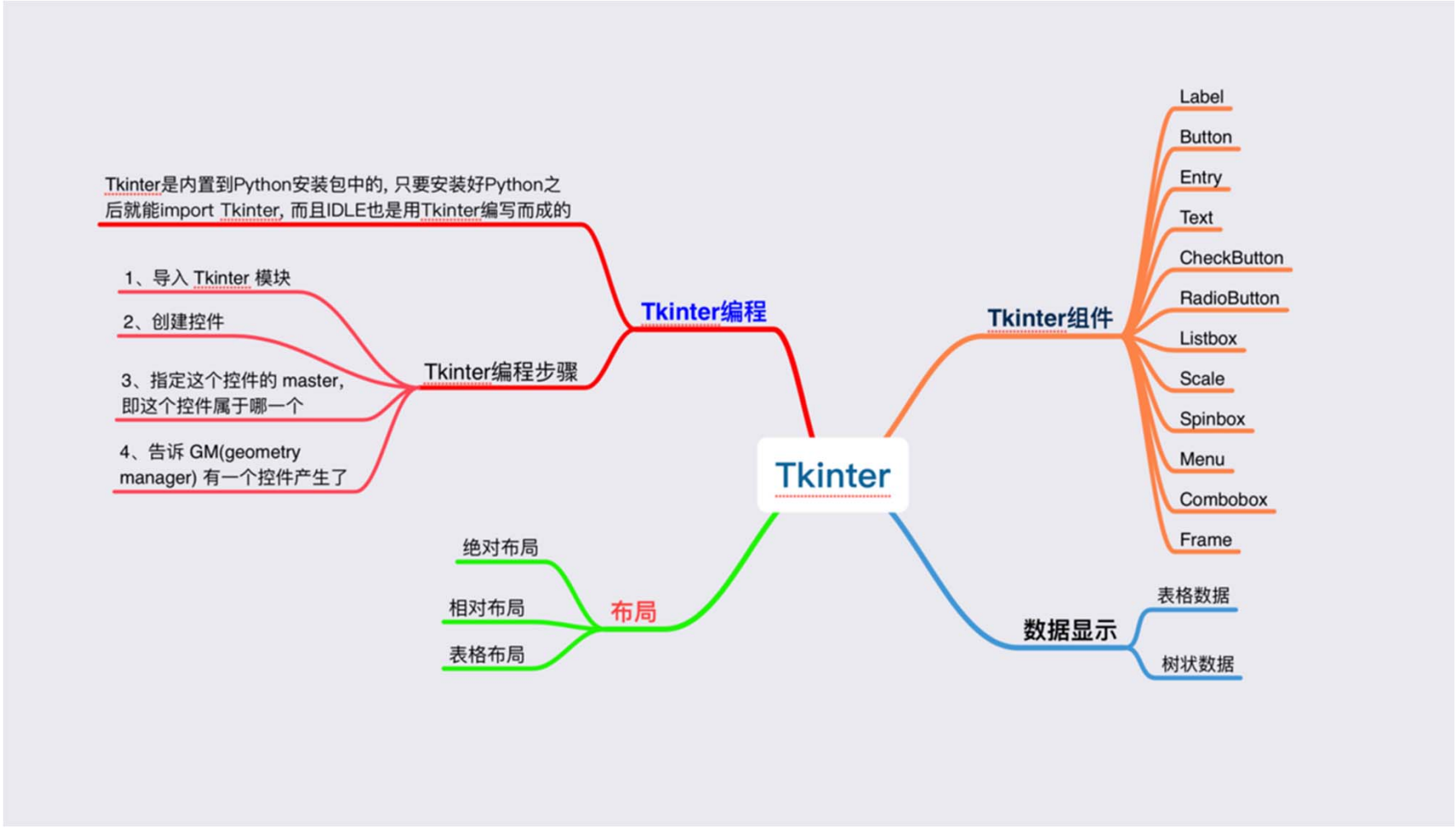
```
from tkinter import *
def main():
    tk=Tk()
    canvas=Canvas(tk,width=400,height=400)
    canvas.pack()
    def moverectangle(event):
        if event.keysym=="Up":
            canvas.move(1,0,-5)
        elif event.keysym=="Down":
            canvas.move(1,0,5)
        elif event.keysym=="Left":
            canvas.move(1,-5,0)
        elif event.keysym=="Right":
            canvas.move(1,5,0)

    canvas.create_rectangle(180,180,220,220,fill="red")
    canvas.bind_all("<KeyPress-Up>",moverectangle)
    canvas.bind_all("<KeyPress-Down>",moverectangle)
    canvas.bind_all("<KeyPress-Left>",moverectangle)
    canvas.bind_all("<KeyPress-Right>",moverectangle)
if __name__=="__main__":
    main()
```

用键盘的上、下、左、右键可移动红色方块



Tkinter之组件布局和事件绑定



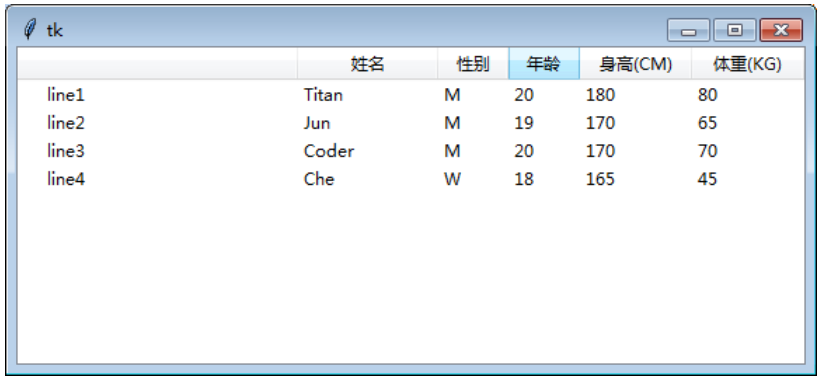
数据显示

在tkinter中的数据展示方式有两种表格数据和树状数据，但是都用到同一个组件Treeview

1. 表格数据

- 表格数据，顾名思义就是用表格形式展示数据
- 要使用Treeview首先要引用tkinter中的ttk模块

```
from tkinter import ttk
# 此处省略window的相关代码
# 创建表格
tree = ttk.Treeview(window)
tree.pack()
# 定义列title(接受一个元组)
tree["columns"] = ('name', 'sex', 'age', 'height', 'weight')
# 设置列宽度
tree.column('name', width=100)
tree.column('sex', width=50)
tree.column('age', width=50)
tree.column('height', width=80)
tree.column('weight', width=80)
# 设置表头(列名)
tree.heading('name', text='姓名')
tree.heading('sex', text='性别')
tree.heading('age', text='年龄')
tree.heading('height', text='身高(CM)')
tree.heading('weight', text='体重(KG)')
# 添加数据
tree.insert('', 0, text='line1', values=('Titan', 'M', 20, 180, 80))
tree.insert('', 1, text='line2', values=('Jun', 'M', 19, 170, 65))
tree.insert('', 2, text='line3', values=('Coder', 'M', 20, 170, 70))
tree.insert('', 3, text='line4', values=('Che', 'W', 18, 165, 45))
```



insert: 参数介绍
-参数1: 上一层级的目录
-- 参数2: 当前数据在当前层级的中的索引值
-- 参数3: 当前数据的标识, 所有层及数据的该标识不能相同
-- 参数4: 显示的数据 - 注: 所有数据的参数3(标识)不能相同

2. 树状数据

树状数据这里指的是,类似文件夹的层级目录一样

```
from tkinter import ttk
# 创建表格
tree = ttk.Treeview(window)
tree.pack()
# 添加一级树枝
treeA1 = tree.insert('', 0, '浙', text='浙江', values=('A1'))
treeA2 = tree.insert('', 1, '鲁', text='山东', values=('A2'))
treeA3 = tree.insert('', 2, '苏', text='江苏', values=('A3'))
# 添加二级树枝
treeA1_1 = tree.insert(treeA1, 0, 'H', text='杭州', values=('A1_1'))
treeA1_2 = tree.insert(treeA1, 1, 'Z', text='舟山', values=('A1_2'))
treeA1_3 = tree.insert(treeA1, 2, 'J', text='嘉兴', values=('A1_3'))
treeA2_1 = tree.insert(treeA2, 0, 'N', text='济南', values=('A2_1'))
treeA2_2 = tree.insert(treeA2, 1, 'L', text='临沂', values=('A2_2'))
treeA2_3 = tree.insert(treeA2, 2, 'Q', text='青岛', values=('A2_3'))
treeA2_4 = tree.insert(treeA2, 3, 'Y', text='烟台', values=('A2_4'))
# 三级树枝
treeA1_1_1 = tree.insert(treeA1_1, 0, 'G', text='江干', values=('A1_1_1'))
treeA1_1_1 = tree.insert(treeA1_1, 1, 'X', text='萧山', values=('A1_1_2'))
```

Tkinter 事件绑定

◆一个 Tkinter 应用程序大部分时间是花在事件循环当中的，也就是通过 mainloop() 进入的事件循环。

◆事件可以有各种来源：用户触发的鼠标和键盘操作和窗口管理器触发的重绘事件

◆Tkinter 给我们提供了一个强大的机制去管理这些事件，去定义这些事件的操作。

◆对每一个组件来说，可以通过 bind() 的方法来将自己定义的函数或方法绑定到具体的事件上。

widget.bind(event, handler)

当被触发的事件满足该组件绑定的事件的时候，Tkinter 就会带着事件对象（Event、事件本身的描述）去调用自定义的 handler() 方法。

实例

捕获鼠标点击的位置的演示

```
from tkinter import *

root =Tk()

#当触发<Button-1>，Tkinter会带着事件本身去调用callback，会把它传入到callback中，因此要个形参来接收对应的事件描述
def callback(event):
    print("点击位置：", event.x, event.y) #这个x和y表示的是相对于应用程序左上角的x和y。root的x和y相对的是屏幕

frame = Frame(root, width=200, height=200)
frame.bind(“<Button-1>”, callback)
# -左边是事件本身，右边是事件详细描述。Button表示的是鼠标的点击事件，1表示的是左键，2代表滚轮，3表示右键，4、5对于Linux系统才有用，表示滚轮向上滚和向下滚。
frame.pack()

mainloop()
```

关于键盘的演示

```
from tkinter import *

root =Tk()

def callback(event):
    print(event.char)  #获取当前键盘按下的字符

frame = Frame(root, width=200, height=200)
frame.bind("<Key>", callback)

frame.focus_set()
frame.pack()

mainloop()
```

利用<Motion>事件获取鼠标实时位置的方式

```
from tkinter import *

root =Tk()

def callback(event):
    print("当前位置： ", event.x, event.y)

frame = Frame(root, width=200, height=200)
frame.bind("<Motion>", callback)
 #Motion事件表示当鼠标进入组件时，就会响应这个事件
frame.focus_set()
frame.pack()

mainloop()
```

事件序列

- Tkinter 使用一种称为**事件序列的机制来允许用户定义事件**
- 用户需要使用 **bind() 方法**将具体的事件序列与自定义的方法相绑定。
- 事件序列是以字符串的形式表示的，可以表示一个或多个相关联的事件（如果是多个事件，那么对应的方法只有在满足所有事件的前提下才会被调用）
- 事件序列使用以下语法描述：

  <modifier-type-detail>：事件序列包含在尖括号（<...>）中的

事件序列

<modifier-type-detail>：事件序列是包含在尖括号（<...>）中的

- type部分的内容是必须的，它通常用于描述普通的事件类型，例如鼠标点击或键盘按键点击
- modifier部分的内容是可选的，它通常用于描述组合键，例如Ctrl + c（<Control-Key-C>），Shift + 鼠标左键点击（<Shift-Button-1>）
- detail部分的内容是可选的，它通常用于描述具体的按键，例如Button-1表示的是鼠标左键

如：
事件序列<Button-1>：用户点击鼠标左键
事件序列<KeyPress-H>：用户点击H按键
事件序列<Control-Shift-KeyPress-H>：用户同时点击Ctrl + Shift +H

type

Active	当组件的状态从“未激活”变为“激活”的时候触发该事件
Button	当用户点击鼠标按键的时候触发该事件 detail部分指定是具体用哪个键：<Button-1>鼠标左键，<Button-2>鼠标中键（滚轮点击），<Button-3>鼠标右键，<Button-4>滚轮上滚（Linux），<Button-5>滚轮下滚（Linux）
ButtonRelease	当用户释放鼠标按键的时候触发该事件 在大多数情况下，比Button要更好使用，因为如果当用户不小心按下鼠标键，用户可以将鼠标移出组件再释放鼠标，从而避免不小心触发事件
Configure	当组件的尺寸改变的时候触发该事件（窗口管理器触发的重绘事件，当你调整组件的尺寸或者移动应用程序，组件会和窗口一样被重绘）
Deactivate	当组件的状态从“激活”变为“未激活”的时候触发该事件 Destroy当组件被销毁时触发该事件
Enter	当鼠标指针进入组件的时候触发该事件 注意：不是用户按下回车键（回车键是Return<Key-Return>）

Expose	当窗口或组件的某部分不再被覆盖的时候触发该事件
FocusIn	当组件获得焦点的时候触发该事件 用户可以用Tab键将焦点转移到该组件上（需要该组件的takefocus选项为True） 你也可以调用focus_set()方法使该组件获得焦点
FocusOut	当组件失去焦点的时候触发该事件
KeyPress	当用户按下键盘按键的时候触发该事件 detail可以指定具体的按键，例如<KeyPress-H>表示当大写字母H被按下的时候触发该事件 KeyPress可以缩写为Key
KeyRelease	当用户释放键盘按键的时候触发该事件
Leave	当鼠标指针离开组件的时候触发该事件
Map	当组件被映射的时候触发该事件 意思是在应用程序中显示该组件的时候，例如调用get()方法
Motion	当鼠标在组件内移动的时候触发该事件
MouseWheel	当鼠标滚轮滚动的时候触发该事件 目前该事件仅支持Windows和Mac系统
Unmap	当组件被取消映射的时候触发该事件 意思是在应用程序中不再显示该组件的时候，例如调用grid_remove()方法
Visibility	当应用程序至少有一部分在屏幕中是可见的时候触发该事件

modifier

在事件序列中，modifier部分的内容可以是以下这些：

Alt	当按下Alt按键的时候
Any	表示任何类型的按键被按下的时候 例如<Any-keyPress>表示当用户按下任何按键时触发事件
Control	当按下Ctrl按键的时候
Double	当后续两个事件被连续触发的时候 例如<Double-Button-1>表示当用户双击鼠标左键时触发事件
Lock	当打开大写字母锁定键（CapsLock）的时候
Shift	当按下Shift按键的时候
Triple	跟Double类似，当后续三个事件被连续触发的时候

Event对象

当Tkinter去回调定义的函数的时候，就会带着Event对象（作为参数）去调用，Event对象属性：

widget	产生该事件的组件
x, y	当前的鼠标的位置坐标（相对于窗口左上角，单位为像素）
x_root, y_root	当前的鼠标的位置坐标（相对于屏幕左上角，单位为像素）
char	按键对应的字符（键盘事件专属，不会显示Control，Shift等键）
keysym	按键名，见下方Key names（键盘事件专属）
keycode	按键码，见下方Key names（键盘事件专属）
num	按钮数字（鼠标事件专属）
width, height	组件的新尺寸（Configure事件专属）
type	该事件类型

Key names

当事件为<Key>，<KeyPress>，<KeyRelease>的时候，detail可以通过设定具体的按键名(keysym)来筛选。

例如<Key-H>表示按下键盘上的大写字母H时候触发事件，<Key-Tab>表示按下键盘上的Tab按键的时候触发事件。

下边列举了键盘所有特殊按键的keysym和keycode

按键名（keysym）	按键码（keycode）	代表的按键
Alt_L	64	左边的Alt按键
Alt_R	113	右边的Alt按键
BackSpace	22	BackSpace（退格）按键
Cancel	110	break按键
Caps_Lock	66	CapsLock（大写字母锁定）按键
Control_L	37	左边的Control
Control_R	109	右边的Control
Delete	107	Delete按键
Down	104	↓按键

可以通过如下程序试验：

```
from tkinter import *

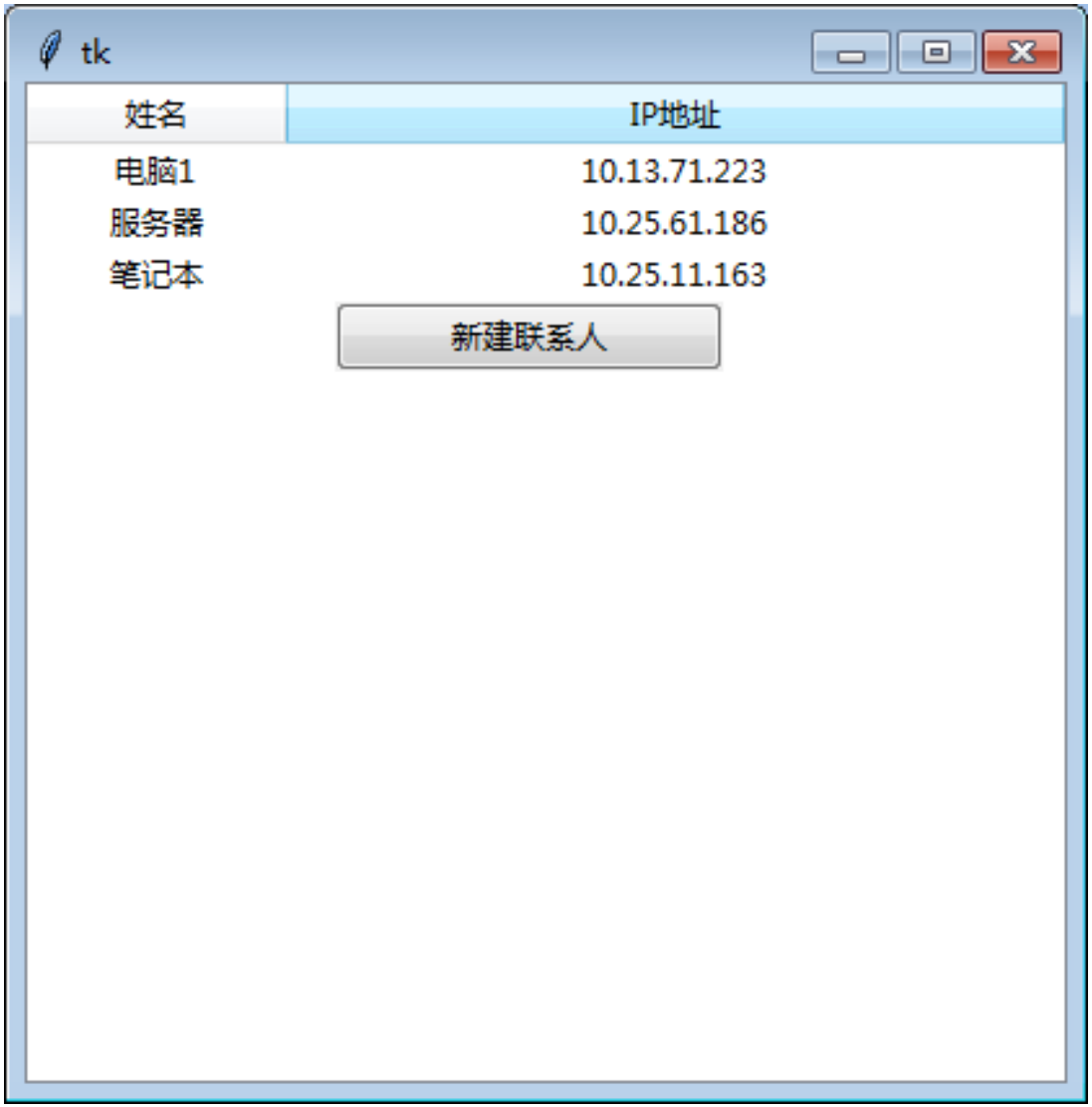
root =Tk()

def callback(event):
    print(event.keysym)
    print(event.keycode)

frame = Frame(root, width=200, height=200)
frame.bind("<Key>", callback)
frame.focus_set()
frame.pack()

mainloop()
```

Python tkinter Treeview创建GUI表格（可编辑、新建单元格）



1.使用Treeview创建表格界面

```
from tkinter import ttk
from tkinter import *
root = Tk()  # 初始框的声明
columns = ("姓名", "IP地址")
treeview = ttk.Treeview(root, height=18, show="headings", columns=columns)  # 表格

treeview.column("姓名", width=100, anchor='center') # 表示列,不显示
treeview.column("IP地址", width=300, anchor='center')

treeview.heading("姓名", text="姓名") # 显示表头
treeview.heading("IP地址", text="IP地址")

treeview.pack(side=LEFT, fill=BOTH)

name = ['电脑1','服务器','笔记本']
ipcode = ['10.13.71.223','10.25.61.186','10.25.11.163']
for i in range(min(len(name),len(ipcode))): # 写入数据
    treeview.insert('', i, values=(name[i], ipcode[i]))
root.mainloop()  # 进入消息循环
```




2.编写功能函数，并与鼠标事件绑定

通过功能函数，实现按表头排序、编辑单元格、新建单元格。

◆ **编辑单元格函数set_cell_value原理：**

在原单元格上方新建一个文本框Text和按钮Button，用来编辑内容，按下按钮将单元格内容存入单元格，并销毁文本框Text和按钮Button。

```
def set_cell_value(event): # 双击进入编辑状态
    for item in treeview.selection():
        #item = I001
        item_text = treeview.item(item, "values")
        #print(item_text[0:2])  # 输出所选行的值
    column= treeview.identify_column(event.x)# 列
    row = treeview.identify_row(event.y)  # 行
    cn = int(str(column).replace('#',''))
    rn = int(str(row).replace('I',''))
    entryedit = Text(root,width=10+(cn-1)*16,height = 1)
    entryedit.place(x=16+(cn-1)*130, y=6+rn*20)
    def saveedit():
        treeview.set(item, column=column, value=entryedit.get(0.0, "end"))
        entryedit.destroy()
        okb.destroy()
    okb = ttk.Button(root, text='OK', width=4, command=saveedit)
    okb.place(x=90+(cn-1)*242,y=2+rn*20)
```

◆ **新建单元格函数newrow原理：**

调用Treeview.insert方法，并在表格下方设置‘新建’按钮Button，用来关联函数，按钮位置随表格行数实时更新。

```
def newrow():
    name.append('待命名')
    ipcode.append('IP')
    treeview.insert('', len(name)-1, values=(name[len(name)-1],
ipcode[len(name)-1]))
    treeview.update()
    newb.place(x=120, y=(len(name)-1)*20+45)
    newb.update()
```

完整代码：

```
from tkinter import ttk
from tkinter import *
root = Tk()  # 初始框的声明
columns = ("姓名", "IP地址")
treeview = ttk.Treeview(root, height=18, show="headings", columns=columns)  # 表格

treeview.column("姓名", width=100, anchor='center') # 表示列,不显示
treeview.column("IP地址", width=300, anchor='center')

treeview.heading("姓名", text="姓名") # 显示表头
treeview.heading("IP地址", text="IP地址")

treeview.pack(side=LEFT, fill=BOTH)

name = ['电脑1','服务器','笔记本']
ipcode = ['10.13.71.223','10.25.61.186','10.25.11.163']
for i in range(min(len(name),len(ipcode))): # 写入数据
    treeview.insert('', i, values=(name[i], ipcode[i]))
```

```
def treeview_sort_column(tv, col, reverse):
    l = [(tv.set(k, col), k) for k in tv.get_children('')]
    l.sort(reverse=reverse)  # 排序方式
for index, (val, k) in enumerate(l):  # 根据排序后索引移动
        tv.move(k, '', index)
    tv.heading(col, command=lambda: treeview_sort_column(tv, col, not reverse))
```

```
def set_cell_value(event): # 双击进入编辑状态
    for item in treeview.selection():
        #item = I001
        item_text = treeview.item(item, "values")
        print(item_text[0:2])  # 输出所选行的值
    column= treeview.identify_column(event.x)# 列
    row = treeview.identify_row(event.y)  # 行
    cn = int(str(column).replace('#',''))
    rn = int(str(row).replace('I',''))
    entryedit = Text(root,width=10+(cn-1)*16,height = 1)
    entryedit.place(x=16+(cn-1)*130, y=6+rn*20)
    def saveedit():
        treeview.set(item, column=column, value=entryedit.get(0.0, "end"))
        entryedit.destroy()
        okb.destroy()
    okb = ttk.Button(root, text='OK', width=4, command=saveedit)
    okb.place(x=90+(cn-1)*242,y=2+rn*20)
```

```
def newrow():
    name.append('待命名')
    ipcode.append('IP')
    treeview.insert('', len(name)-1, values=(name[len(name)-1], ipcode[len(name)-1]))
    treeview.update()
    newb.place(x=120, y=(len(name)-1)*20+45)
    newb.update()

treeview.bind('<Double-1>', set_cell_value) # 双击左键进入编辑
newb = ttk.Button(root, text='新建联系人', width=20, command=newrow)
newb.place(x=120,y=(len(name)-1)*20+45)


for col in columns:  # 绑定函数，使表头可排序
    treeview.heading(col, text=col, command=lambda _col=col: treeview_sort_column(treeview, _col, False))
root.mainloop()  # 进入消息循环
```

例：编程实现简易聊天窗口GUI



聊天窗口布局：

- 左上：聊天历史信息显示
- 左中：当前信息编辑区域
- 左下：按钮区域
- 右侧：信息显示区域

Frame控件:

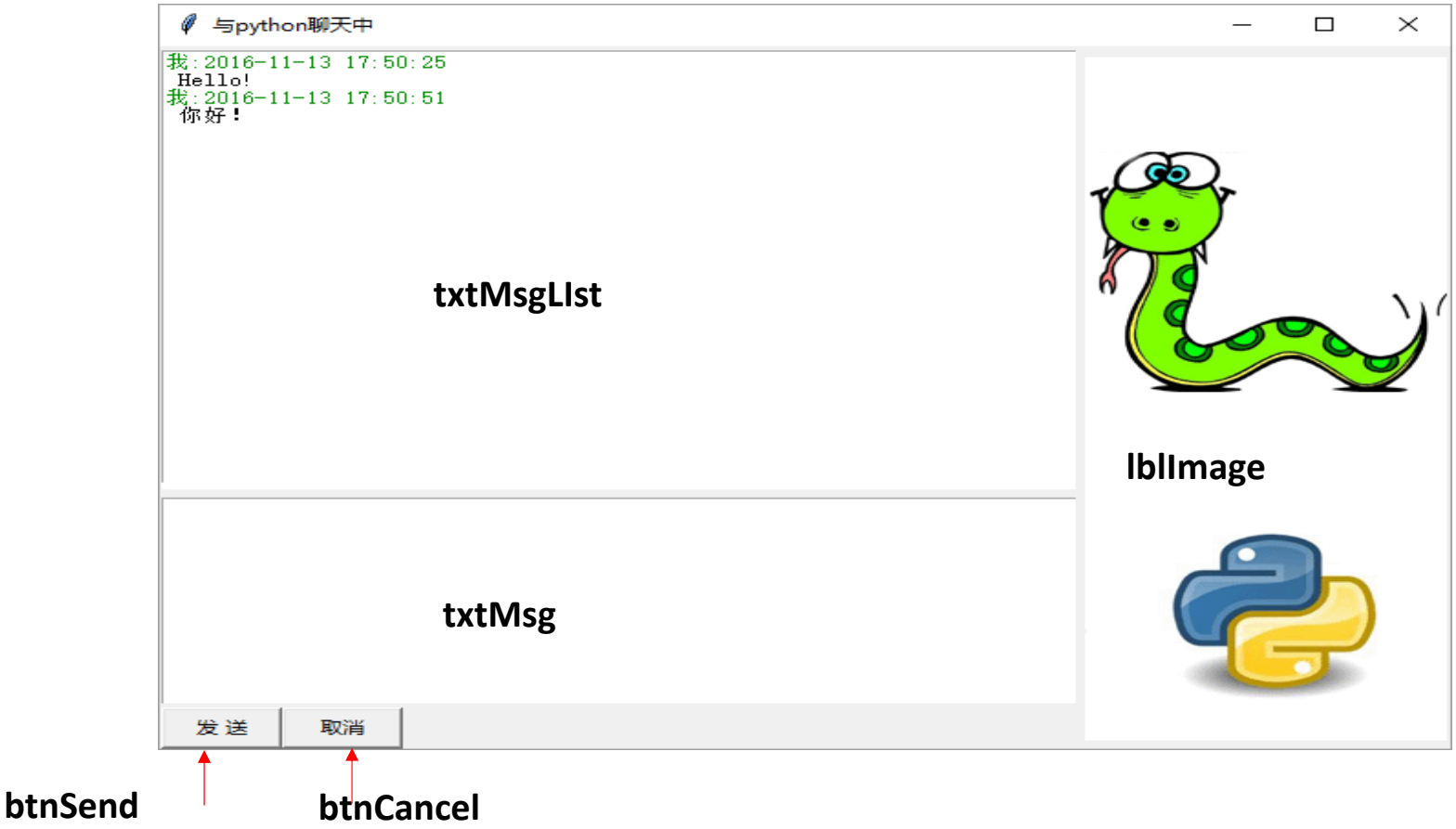
- 容器区域布局:
  - frmLT、frmLC、frmLB、frmRT

```
#创建frame容器
frmLT = Frame(width=500, height=320, bg='white')
frmLC = Frame(width=500, height=150, bg='white')
frmLB = Frame(width=500, height=30)
frmRT = Frame(width=200, height=500)
```

统一控件对象命名规则：

- “控件类型”+“功能”
  - frmLT， frame+LeftTop
  - txtMsg， text控件+消息
  - btnSend， button控件+发送

窗口控件：


与python聊天中
我：2016-11-13 17:50:25
Hello!
我：2016-11-13 17:50:51
你好！
txtMsgLIst
lblImage
txtMsg
发 送
取消
btnSend
btnCancel

创建各窗口控件：

```
#创建控件
txtMsgList = Text(frmLT)
txtMsgList.tag_config('greencolor', foreground='#008C00')#创建tag
txtMsg = Text(frmLC);
txtMsg.bind("<KeyPress-Up>", sendMsgEvent)
btnSend = Button(frmLB, text='发 送', width = 8, command=sendMsg)
btnCancel = Button(frmLB, text='取消', width = 8, command=cancelMsg)
imgInfo = PhotoImage(file = "python.gif")
lblImage = Label(frmRT, image = imgInfo)
lblImage.image = imgInfo
```

sendMsg()

- 回调函数
- 功能：在消息框列表输出历史消息，同时将当前时间、当前信息作为最后一条历史消息。然后删除消息发送框的内容。

```
def sendMsg():#发送消息
    strMsg = '我:' + time.strftime("%Y-%m-%d %H:%M:%S",
                                   time.localtime()) + '\n '
    txtMsgList.insert(END, strMsg, 'greencolor')
    txtMsgList.insert(END, txtMsg.get('0.0', END))
    txtMsg.delete('0.0', END)
```

- sendMsgEvent():

```
def sendMsgEvent(event):  #发送消息事件
    if event.keysym == "Up":
        sendMsg()
```

- cancelMsg ():

```
def cancelMsg():#取消消息
    txtMsg.delete('0.0', END)
```

grid()方法：界面上控件的布局



grid()方法：界面上控件的布局

```
#窗口布局
frmLT.grid(row=0, column=0, columnspan=2, padx=1, pady=3)
frmLC.grid(row=1, column=0, columnspan=2, padx=1, pady=3)
frmLB.grid(row=2, column=0, columnspan=2)
frmRT.grid(row=0, column=2, rowspan=3, padx=2, pady=3)
#固定大小
frmLT.grid_propagate(0)
frmLC.grid_propagate(0)
frmLB.grid_propagate(0)
frmRT.grid_propagate(0)

btnSend.grid(row=2, column=0)
btnCancel.grid(row=2, column=1)
lblImage.grid()
txtMsgList.grid()
txtMsg.grid()

#主事件循环
t.mainloop()
```

源程序:

```
#ChatWin.py
from tkinter import *
import time

def main():

  def sendMsg():#发送消息
    strMsg = '我:' + time.strftime("%Y-%m-%d %H:%M:%S",
                                   time.localtime()) + '\n '
    txtMsgList.insert(END, strMsg, 'greencolor')
    txtMsgList.insert(END, txtMsg.get('0.0', END))
    txtMsg.delete('0.0', END)

  def cancelMsg():#取消消息
    txtMsg.delete('0.0', END)

  def sendMsgEvent(event): #发送消息事件
    if event.keysym == "Up":
      sendMsg()

  #创建窗口
  t = Tk()
  t.title('与python聊天中')

  #创建frame容器
  frmLT = Frame(width=500, height=320, bg='white')
  frmLC = Frame(width=500, height=150, bg='white')
  frmLB = Frame(width=500, height=30)
  frmRT = Frame(width=200, height=500)
```

```
    #创建控件
    txtMsgList = Text(frmLT)
    txtMsgList.tag_config('greencolor', foreground='#008C00')#创建tag
    txtMsg = Text(frmLC);
    txtMsg.bind("<KeyPress-Up>", sendMsgEvent)
    btnSend = Button(frmLB, text='发 送', width = 8, command=sendMsg)
    btnCancel = Button(frmLB, text='取消', width = 8, command=cancelMsg)
    imgInfo = PhotoImage(file = "python.gif")
    lblImage = Label(frmRT, image = imgInfo)
    lblImage.image = imgInfo

    #窗口布局
    frmLT.grid(row=0, column=0, columnspan=2, padx=1, pady=3)
    frmLC.grid(row=1, column=0, columnspan=2, padx=1, pady=3)
    frmLB.grid(row=2, column=0, columnspan=2)
    frmRT.grid(row=0, column=2, rowspan=3, padx=2, pady=3)
    #固定大小
    frmLT.grid_propagate(0)
    frmLC.grid_propagate(0)
    frmLB.grid_propagate(0)
    frmRT.grid_propagate(0)

    btnSend.grid(row=2, column=0)
    btnCancel.grid(row=2, column=1)
    lblImage.grid()
    txtMsgList.grid()
    txtMsg.grid()

    #主事件循环
    t.mainloop()

if __name__ == '__main__':
    main()
```

四、Turtle库简介

- Turtle库是Python语言中一个很流行的绘制图像的函数库。
- 使用turtle库，可想象为一个小乌龟，在一个横轴为x、纵轴为y的坐标系原点，(0,0—中心)位置开始，方向向右。
- 它根据一组函数指令的控制，在这个平面坐标系中移动，从而在它爬行的路径上绘制了图形。
- 详细信息见：https://docs.python.org/3/library/turtle.html

1、控制画笔绘制状态方法

方法名称	方法含义
pendown()	放下画笔，移到指定点后继续绘制
penup()	提起画笔，用于另起一个地方绘制时用，与**pendown()**配对使用
pensize(width)	设置画笔线条的粗细为指定大小

2、turtle运动方法

方法名称	方法含义
forward()	沿着当前方向前进指定距离
backward()	沿着当前相反方向后退指定距离
right(angle)	向右旋转angle角度
left(angle)	向左旋转angle角度
goto(x,y)	移动到绝对坐标（x,y）处
setx( )	将当前x轴移动到指定位置
sety( )	将当前y轴移动到指定位置
setheading(angle)	设置当前朝向为angle角度（0表示向东，90表示向北，180表示向西，270表示向南）
home()	设置当前画笔位置为原点，朝向东。
circle()	绘制一个指定半径，角度、以及步骤的圆圈
dot(r, color)	绘制一个指定直径和颜色的圆点
undo()	撤销画笔最后一步动作
speed()	设置画笔的绘制速度，参数为0-10之间

例：应用circle方法绘制图形

```
#应用circle方法绘制图形
import turtle

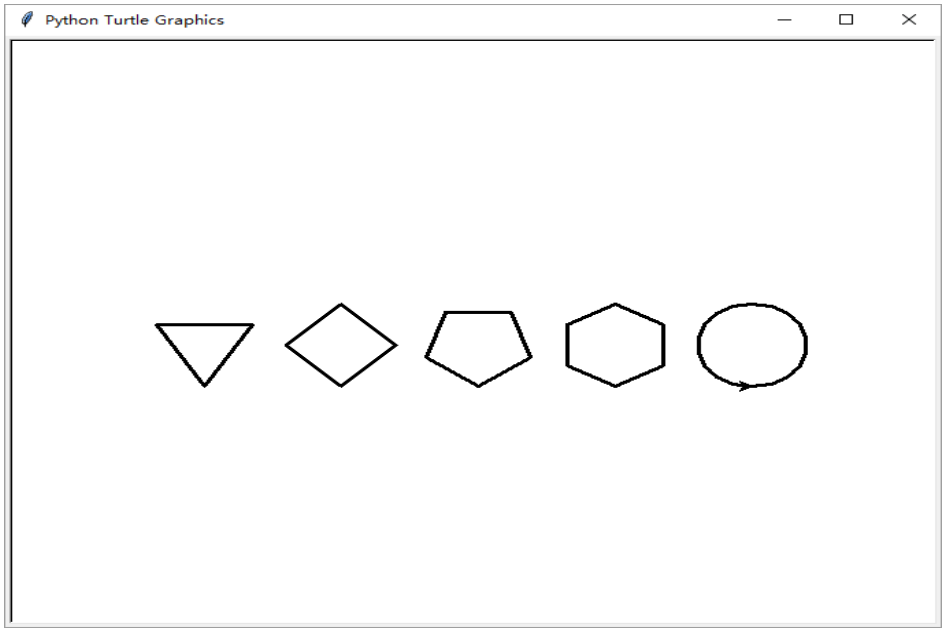
turtle.pensize(3)
turtle.penup()
turtle.goto(-200,-50) #定位
turtle.pendown()
turtle.circle(40, steps=3) #由3条线段构成圆，即画三角形

turtle.penup()
turtle.goto(-100,-50)
turtle.pendown()
turtle.circle(40, steps=4)

turtle.penup()
turtle.goto(0,-50)
turtle.pendown()
turtle.circle(40, steps=5)

turtle.penup()
turtle.goto(100,-50)
turtle.pendown()
turtle.circle(40, steps=6)

turtle.penup()
turtle.goto(200,-50)
turtle.pendown()
turtle.circle(40) #缺省第二个参数，即画圆
turtle.down
```



3、Turtle颜色和字体绘制方法

方法名称	方法含义
color()	设置画笔的颜色
begin_fill()	填充图形前，调用该方法
end_fill()	填充图形结束
filling()	返回填充的状态，True为填充，False为未填充
clear()	清空当前窗口，但不改变当前画笔的位置
reset()	清空当前窗口，并重置位置等状态为默认值
screensize()	设置画布的长和宽
hideturtle()	隐藏画笔的turtle形状
showturtle()	显示画笔的turtle形状
isvisible()	如果turtle可见，则返回True
write(str，font=None)	输出font字体的字符串

例：修改例10-19，给所绘图形填充颜色

```
import turtle

turtle.pensize(3)
turtle.penup()
turtle.goto(-200,-50)
turtle.pendown()
turtle.begin_fill()          #修改
turtle.color("red")          #修改
turtle.circle(40, steps=3)
turtle.end_fill()            #修改

turtle.penup()
turtle.goto(-100,-50)
turtle.pendown()
turtle.begin_fill()
turtle.color("blue")
turtle.circle(40, steps=4)
turtle.end_fill()

turtle.penup()
turtle.goto(0,-50)
turtle.pendown()
turtle.begin_fill()
turtle.color("green")
turtle.circle(40, steps=5)
turtle.end_fill()

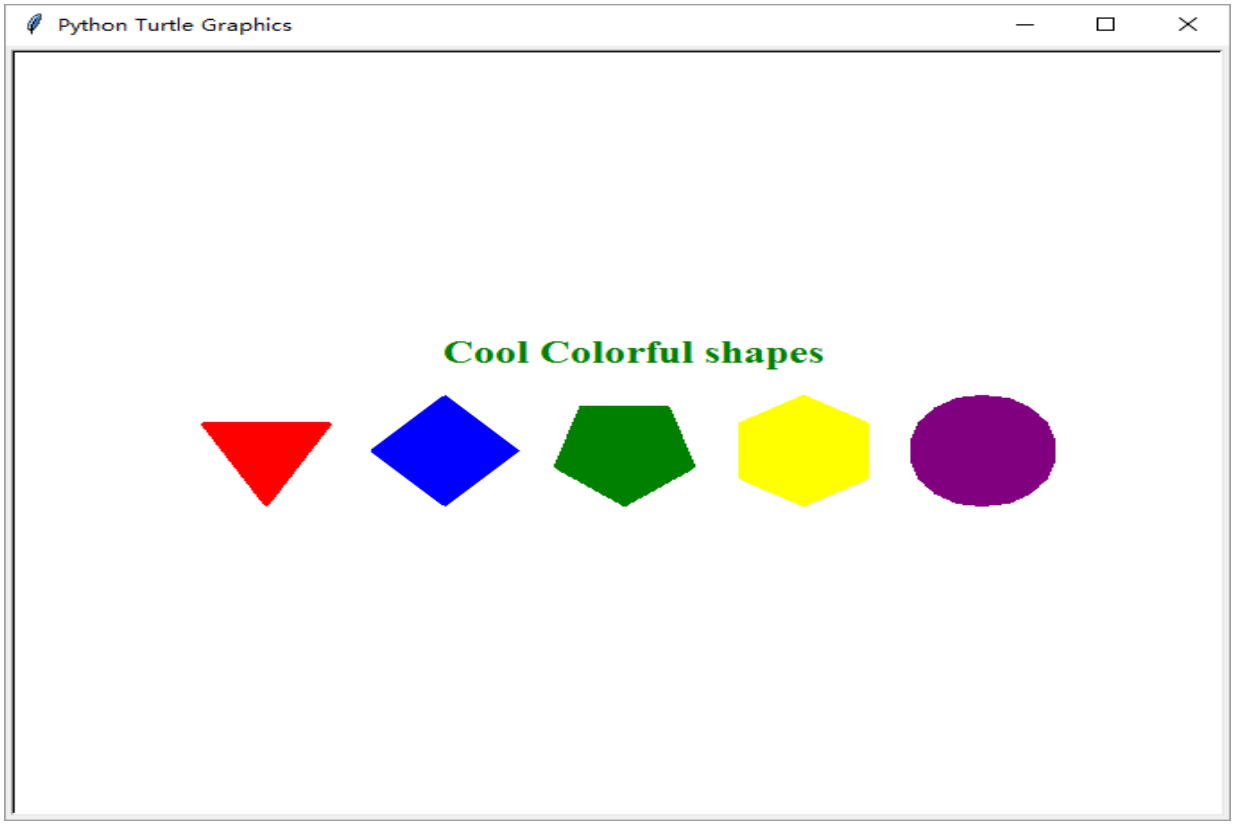
turtle.penup()
turtle.goto(100,-50)
turtle.pendown()
turtle.begin_fill()
turtle.color("yellow")
turtle.circle(40, steps=6)
turtle.end_fill()

turtle.penup()
turtle.goto(200,-50)
turtle.pendown()
turtle.begin_fill()
turtle.color("purple")
turtle.circle(40)
turtle.end_fill()

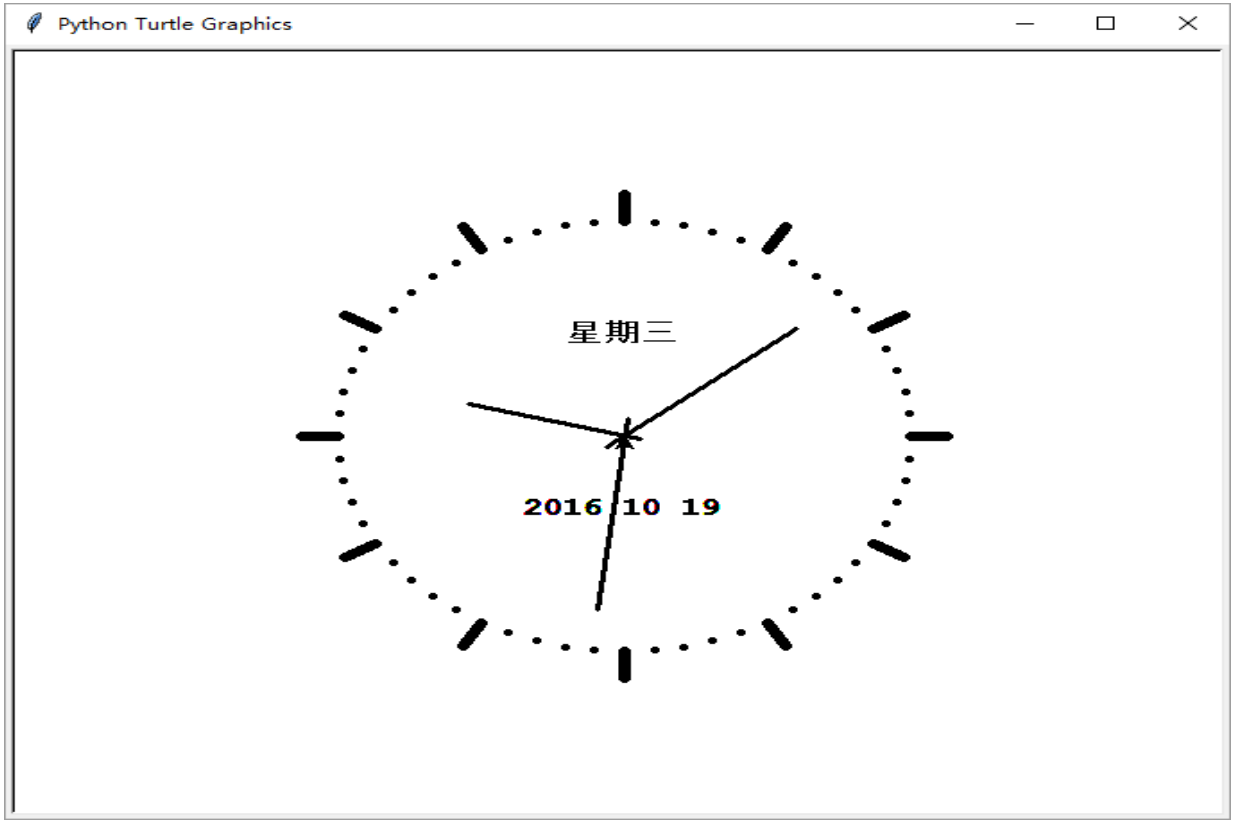
turtle.color("green")
turtle.penup()
turtle.goto(-100,50)
turtle.pendown()
turtle.write(("Cool Colorful shapes"), font=("Times", 18, "bold"))
turtle.hideturtle()

turtle.down
```

执行结果:

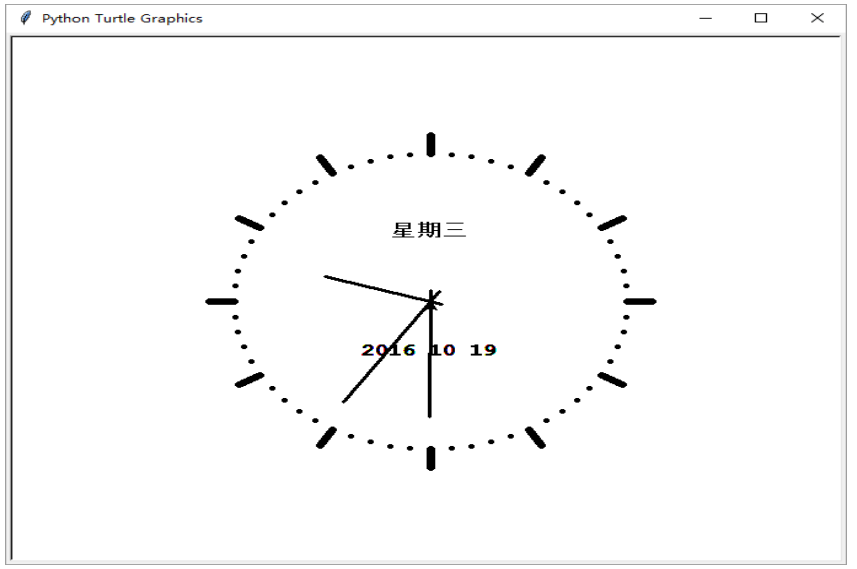


例10-21：编写程序模拟时钟，要求时钟根据计算机系统时间实时动态更新。



需要建立五个turtle对象：

- 1个turtle: 绘制外表盘
- 3个turle: 模拟表针行为
- 1个turtle: 输出表盘上的文字



模拟时钟程序过程：

- 第一步：建立Turtle对象并初始化。
  - 表盘绘制Turtle对象
  - 文本输出Turtle对象
  - 3个指针Turtle对象
- 第二步：静态表盘绘制
- 第三步：根据时钟更新表针位置和时间信息
- 说明：需要导入turtle库和datetime库

表身绘制函数SetupClock(radius)：

```
def SetupClock(radius):
    #建立表的外框
    reset()
    pensize(7)
    for i in range(60):
        Skip(radius)
        if i % 5 == 0:
            forward(20)  #每5个刻度为短线段
            Skip(-radius-20)
        else:
            dot(5)          #其余刻度为圆点
            Skip(-radius)
        right(6)
```

跨越函数Skip（step）：

```
def Skip(step):
    penup()
    forward(step)
    pendown()
```

定义表针函数mkHand()：

```
def mkHand(name, length):
    #注册Turtle形状，建立表针Turtle
    reset()
    Skip(-length*0.1)
    begin_poly()
    forward(length*1.1)
    end_poly()
    handForm = get_poly()
    register_shape(name, handForm)
```

- 注册Turtle形状命令：
- register_shap(name, shape=None)
- name：shape的名字，可以是一个gif图像
- shape：turtle形状，可以为空

初始化函数Init（）：

```
def Init():
    global secHand, minHand, hurHand, printer
    mode("logo")# 重置Turtle指向北
    #建立三个表针Turtle并初始化
    mkHand("secHand", 125)
    mkHand("minHand",  130)
    mkHand("hurHand", 90)
    secHand = Turtle()
    secHand.shape("secHand")
    minHand = Turtle()
    minHand.shape("minHand")
    hurHand = Turtle()
    hurHand.shape("hurHand")
    for hand in secHand, minHand, hurHand:
        hand.shapesize(1, 1, 3)
        hand.speed(0)
    #建立输出文字Turtle
    printer = Turtle()
    printer.hideturtle()
    printer.penup()
```

更新时钟函数Tick()：

```
def Tick():
    #绘制表针的动态显示
    t = datetime.today()
    second = t.second + t.microsecond*0.000001
    minute = t.minute + second/60.0
    hour = t.hour + minute/60.0
    secHand.setheading(6*second) #秒针每秒转动6度
    minHand.setheading(6*minute) #分针每分转动6度
    hurHand.setheading(30*hour)  #时针每小时转动30度

    tracer(False)      #动画关闭（为保证显示速度）
    printer.forward(65)
    printer.write(Week(t), align="center",
                  font=("Courier", 14, "bold"))
    printer.back(130)
    printer.write(Date(t), align="center",
                  font=("Courier", 14, "bold"))
    printer.home()
    tracer(True)       #恢复动画效果

    ontimer(Tick, 100)#100ms后继续调用tick
```

主函数main()：

```
def main():
    tracer(False)
    Init()
    SetupClock(160)
    tracer(True)
    Tick()
    mainloop()
```

源程序:

```
#时钟模拟
from turtle import *
from datetime import *

def Skip(step):
    penup()
    forward(step)
    pendown()

def mkHand(name, length):
    #注册Turtle形状，建立表针Turtle
    reset()
    Skip(-length*0.1)
    begin_poly()
    forward(length*1.1)
    end_poly()
    handForm = get_poly()
    register_shape(name, handForm)
```

```
def Init():
    global secHand, minHand, hurHand, printer
    mode("logo")# 重置Turtle指向北
    #建立三个表针Turtle并初始化
    mkHand("secHand", 125)
    mkHand("minHand",  130)
    mkHand("hurHand", 90)
    secHand = Turtle()
    secHand.shape("secHand")
    minHand = Turtle()
    minHand.shape("minHand")
    hurHand = Turtle()
    hurHand.shape("hurHand")
    for hand in secHand, minHand, hurHand:
        hand.shapesize(1, 1, 3)
        hand.speed(0)
    #建立输出文字Turtle
    printer = Turtle()
    printer.hideturtle()
    printer.penup()
```

```
def SetupClock(radius):
    #建立表的外框
    reset()
    pensize(7)
    for i in range(60):
        Skip(radius)
        if i % 5 == 0:
            forward(20)  #每5个刻度为短线段
            Skip(-radius-20)
        else:
            dot(5)       #其余刻度为圆点
            Skip(-radius)
        right(6)

def Week(t):
    week = ["星期一", "星期二", "星期三",
            "星期四", "星期五", "星期六", "星期日"]
    return week[t.weekday()]

def Date(t):
    y = t.year
    m = t.month
    d = t.day
    return "%s %d %d" % (y, m, d)
```

```
def Tick():
    #绘制表针的动态显示
    t = datetime.today()
    second = t.second + t.microsecond*0.000001
    minute = t.minute + second/60.0
    hour = t.hour + minute/60.0
    secHand.setheading(6*second) #秒针每秒转动6度
    minHand.setheading(6*minute) #分针每分转动6度
    hurHand.setheading(30*hour)  #时针每小时转动30度

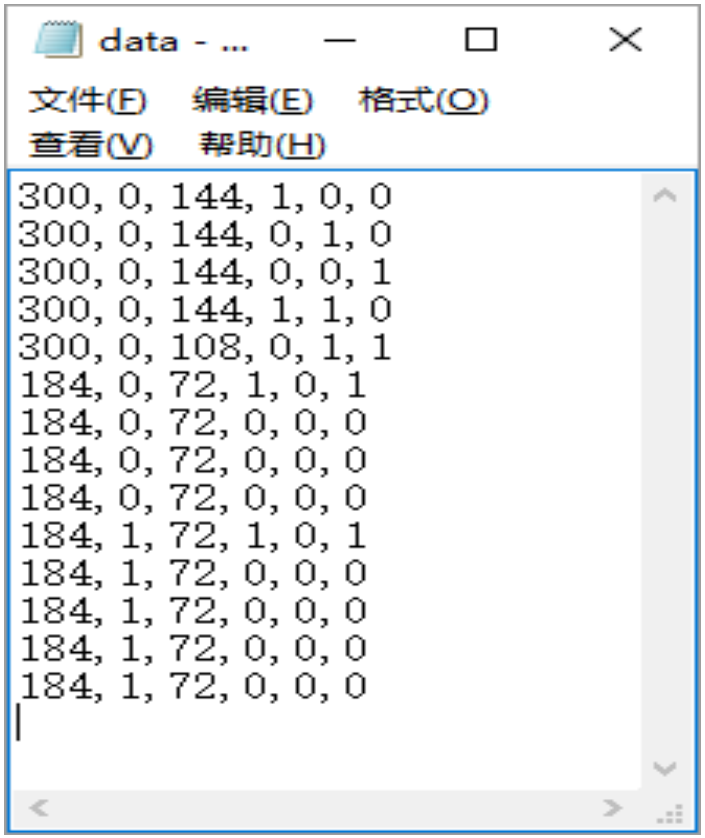
    tracer(False)      #动画关闭（为保证显示速度）
    printer.forward(65)
    printer.write(Week(t), align="center",
                  font=("Courier", 14, "bold"))
    printer.back(130)
    printer.write(Date(t), align="center",
                  font=("Courier", 14, "bold"))
    printer.home()
    tracer(True)       #恢复动画效果

    ontimer(Tick, 100)#100ms后继续调用tick

def main():
    tracer(False)
    Init()
    SetupClock(160)
    tracer(True)
    Tick()
    mainloop()

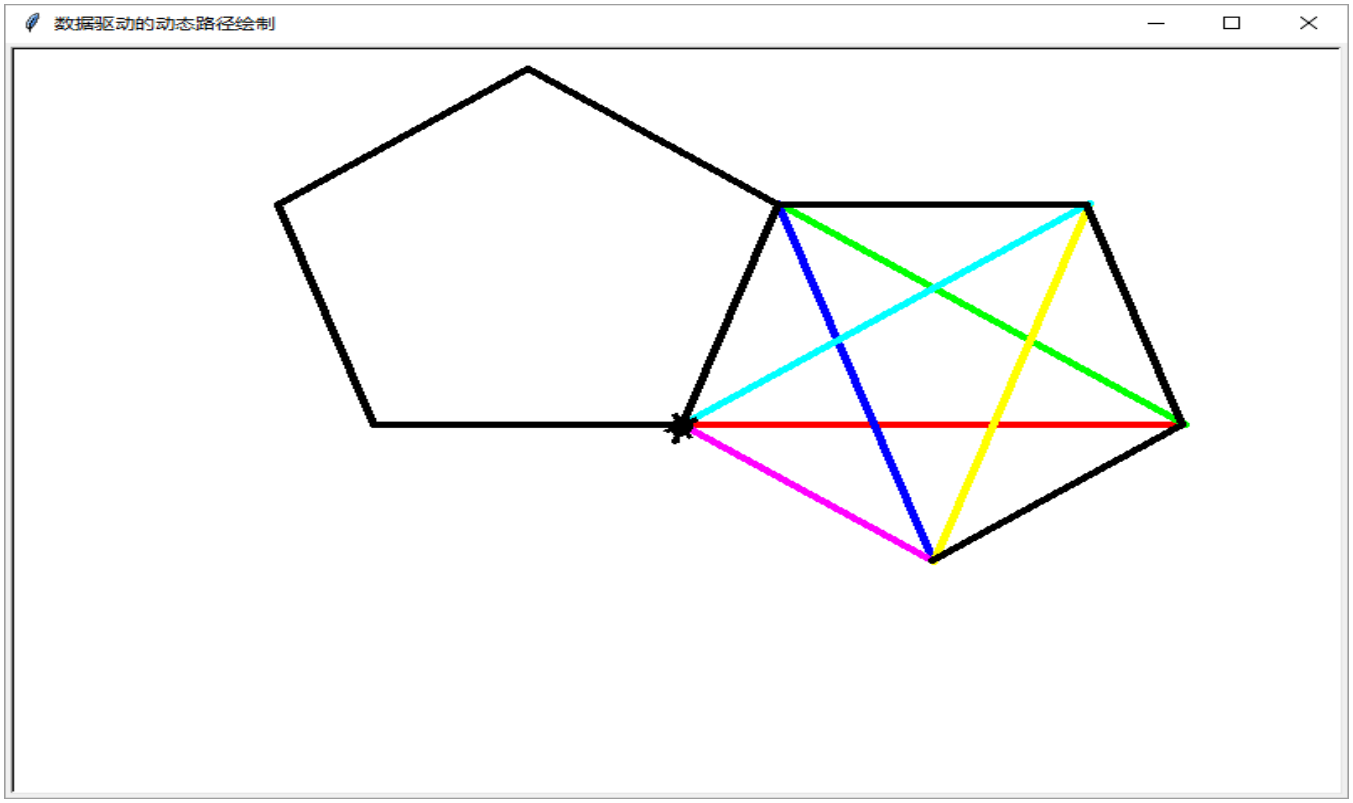
if __name__ == "__main__":
    main()
```

例10-22：编写程序。根据文件data.txt中的数据，使用turtle库来动态绘制图形路径



- 数据说明：
- 第1列：路径前进像素数
- 第2列：转动方向。0为左，1为右
- 第3列：转动的角度
- 第4—6列：绘制颜色的rgb值

程序执行结果：



程序实现的具体过程为：

（1）使用import命令为程序引入turtle库

（2）设置窗口信息和Turtle画笔

```
#根据数据文件在窗口中动态路径绘制
import turtle

def main():
    #设置窗口信息
    turtle.title('数据驱动的动态路径绘制')
    turtle.setup(800, 600, 0, 0)
    #设置画笔
    pen = turtle.Turtle()
    pen.color("red")
    pen.width(5)
    pen.shape("turtle")
    pen.speed(5)
```

（3）读取数据文件到列表result中

```
result=[]
file = open("data.txt","r")
for line in file:
    result.append(list(map(float, line.split(','))))
print(result)
```

```
[[300.0, 0.0, 144.0, 1.0, 0.0, 0.0], [300.0, 0.0, 144.0, 0.0, 1.0, 0.0], [300.0,
 0.0, 144.0, 0.0, 0.0, 1.0], [300.0, 0.0, 144.0, 1.0, 1.0, 0.0], [300.0, 0.0, 10
8.0, 0.0, 1.0, 1.0], [184.0, 0.0, 72.0, 1.0, 0.0, 1.0], [184.0, 0.0, 72.0, 0.0,
0.0, 0.0], [184.0, 0.0, 72.0, 0.0, 0.0, 0.0], [184.0, 0.0, 72.0, 0.0, 0.0, 0.0],
 [184.0, 1.0, 72.0, 1.0, 0.0, 1.0], [184.0, 1.0, 72.0, 0.0, 0.0, 0.0], [184.0, 1
.0, 72.0, 0.0, 0.0, 0.0], [184.0, 1.0, 72.0, 0.0, 0.0, 0.0], [184.0, 1.0, 72.0,
0.0, 0.0, 0.0]]
```

(4) 根据每一条数据记录进行绘制

```
for i in range(len(result)):
    pen.color((result[i][3],result[i][4],result[i][5]))
    pen.forward(result[i][0])
    if result[i][1]:
        pen.rt(result[i][2])
    else:
        pen.lt(result[i][2])
```

(5) 画笔回到原点

完整程序:

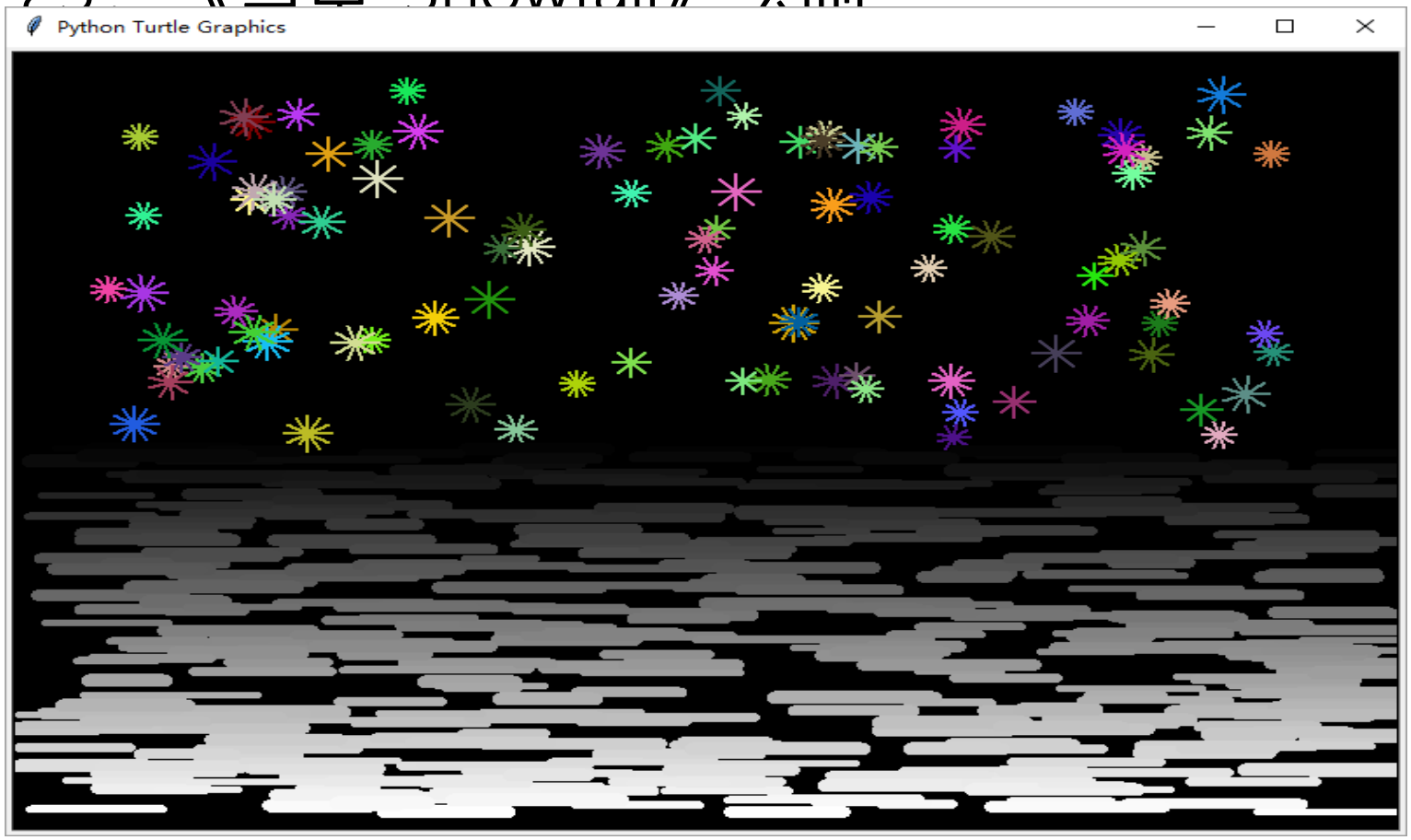
```
#根据数据文件在窗口中动态路径绘制
import turtle

def main():
    #设置窗口信息
    turtle.title('数据驱动的动态路径绘制')
    turtle.setup(800, 600, 0, 0)
    #设置画笔
    pen = turtle.Turtle()
    pen.color("red")
    pen.width(5)
    pen.shape("turtle")
    pen.speed(5)
    #读取文件
    result=[]
    file = open("data.txt","r")
    for line in file:
        result.append(list(map(float, line.split(','))))
    print(result)
```

```
    #动态绘制
    for i in range(len(result)):
        pen.color((result[i][3],result[i][4],result[i][5]))
        pen.forward(result[i][0])
        if result[i][1]:
            pen.rt(result[i][2])
        else:
            pen.lt(result[i][2])
    pen.goto(0,0)

if __name__ == '__main__':
    main()
```

例10-23：《雪景-Snowfall》绘制



随机因素：

- 雪花位置
- 雪花颜色
- 雪花大小
- 花瓣数目
- 地面灰色线条长短
- 地面灰色线条位置
- 因此，需要导入随机数模块

主函数:

```
def main():
    setup(800, 600, 0, 0)  #设定窗口大小
    tracer(False)
    bgcolor("black")       #设定背景为黑色
    snow()                 #调用绘制雪花函数
    ground()               #调用绘制雪地函数
    tracer(True)
    mainloop()
```

snow()函数：

```
def snow():
    hideturtle()
    pensize(2)
    speed(100)
    for i in range(100): #绘制100朵雪花
        r = random()   #调用random函数随机生成雪花颜色的rgb值
        g = random()
        b = random()
        pencolor(r, g, b)
        penup()
        setx(randint(-350,350)) #在指定区域的随机坐标处绘制雪花
        sety(randint(1,270))
        pendown()
        dens = randint(8,12) #随机生成雪花花瓣数
        snowsize = randint(10,14) #随机生成雪花大小
        for j in range(dens): #绘制雪花
            forward(snowsize)
            backward(snowsize)
            right(360/dens)
```

ground()函数：

```
def ground():
    hideturtle()
    speed(100)
    for i in range(400): #绘制地面400个线条
        pensize(randint(5,10)) #随机生成画笔大小（即粗细）
        x = randint(-400,350)   #在指定区域随机生成线条位置坐标
        y = randint(-280,-1)
        r = -y/280   #颜色的rgb值根据y坐标变化（因线条灰度呈现由远及近的效果）
        g = -y/280   #将rgb值控制在0-1之间
        b = -y/280
        pencolor((r, g, b))
        penup()
        goto(x,y)
        pendown()
        forward(randint(40,100))
```

源程序：

```
#Snowfall.py
from turtle import *
from random import *

def ground():
    hideturtle()
    speed(100)
    for i in range(400): #绘制地面400个线条
        pensize(randint(5,10))  #随机生成画笔大小（即粗细）
        x = randint(-400,350)   #在指定区域随机生成线条位置坐标
        y = randint(-280,-1)
        r = -y/280   #颜色的rgb值根据y坐标变化（因线条灰度呈现由远及近的效果）
        g = -y/280   #将rgb值控制在0-1之间
        b = -y/280
        pencolor((r, g, b))
        penup()
        goto(x,y)
        pendown()
        forward(randint(40,100))
```

```
def snow():
    hideturtle()
    pensize(2)
    speed(100)
    for i in range(100): #绘制100朵雪花
        r = random()   #调用random函数随机生成雪花颜色的rgb值
        g = random()
        b = random()
        pencolor(r, g, b)
        penup()
        setx(randint(-350,350)) #在指定区域的随机坐标处绘制雪花
        sety(randint(1,270))
        pendown()
        dens = randint(8,12) #随机生成雪花花瓣数
        snowsize = randint(10,14) #随机生成雪花大小
        for j in range(dens): #绘制雪花
            forward(snowsize)
            backward(snowsize)
            right(360/dens)


def main():
    setup(800, 600, 0, 0) #设定窗口大小
    tracer(False)
    bgcolor("black")   #设定背景为黑色
    snow()             #调用绘制雪花函数
    ground()           #调用绘制雪地函数
    tracer(True)
    mainloop()

if __name__ == "__main__":
    main()
```



第十三部分 正则表达式

正则表达式简介
正则表达式匹配过程
正则表达式语法
re模块
正则表达式应用
正则表达式解析HTML

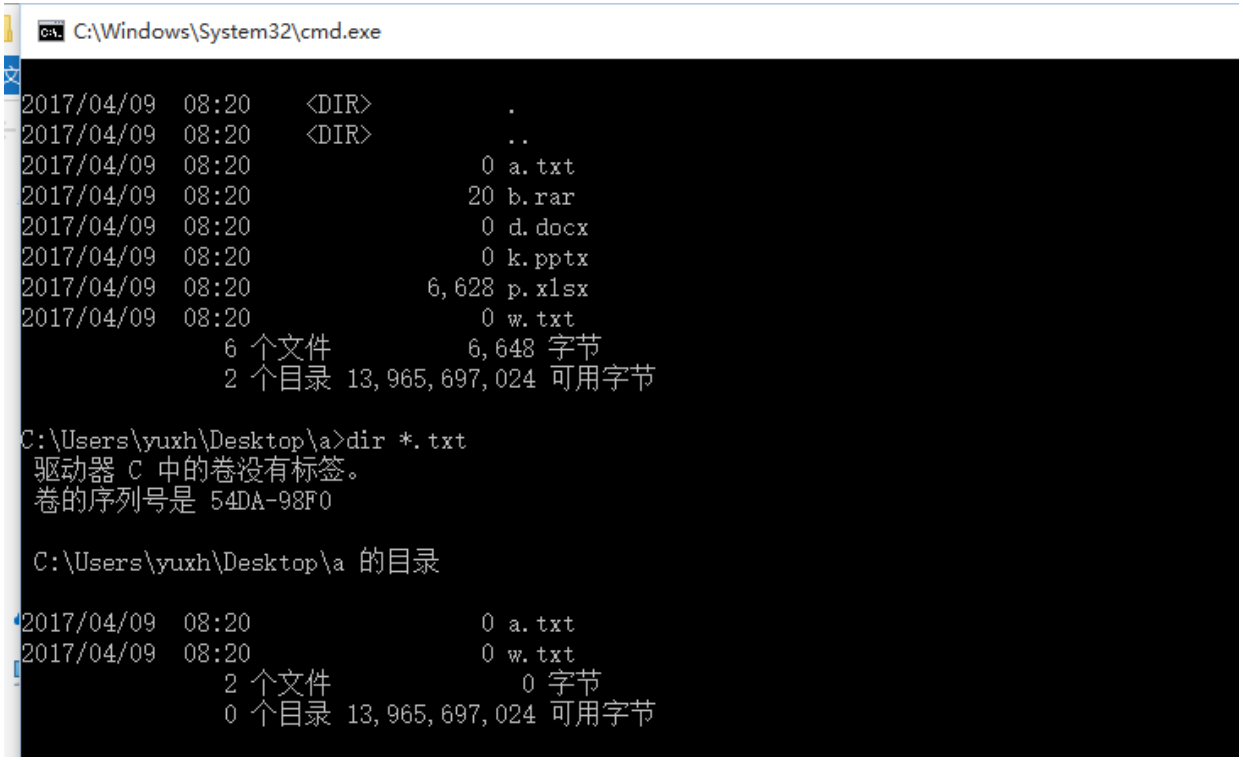
cmd的通配符

- DOS操作系统通配符：文件名描述中可以使用两个通配符 * 和 ?

  ? 可以与任何实际字符匹配

  * 可与任意一串字符匹配

dir *.txt命令输出了所有以txt结尾的文件名



cmd的通配符

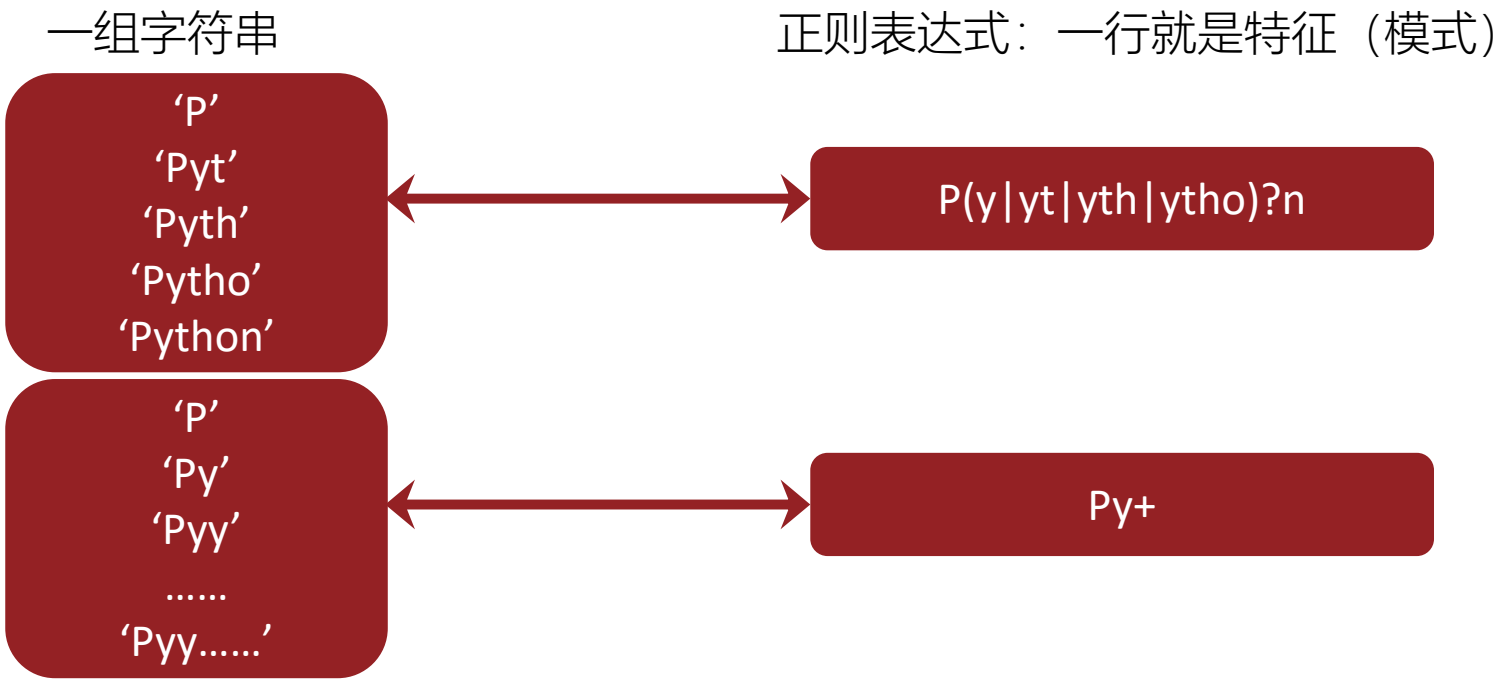
- 在普通字符串的基础上增加通配符，形成了一种功能更强的模式语言
- 一个模式描述一集字符串

  a?b

  a* b
- 缺点：加入了通配符的模式语言还不够灵活（描述能力不够强）
- 逻辑学家 Kleene 提出正则表达式（Regular Expression, regex, regexp, RE, re）

正则表达式简介

- 正则表达式（regular expression），又记作re或regex
- 是用来简洁表达一组字符串的表达式



正则表达式历史

- 1956年，一位叫 Stephen Kleene 的数学家在 McCulloch 和 Pitts 早期工作的基础上，发表了一篇标题为“神经网事件的表示法”的论文，引入了正则表达式的概念
- 随后，将这一工作应用于使用 Ken Thompson 的计算搜索算法的一些早期研究，Ken Thompson 是 Unix 的主要发明人
- 正则表达式的第一个应用程序就是Unix中的 qed 编辑器
- 从那时起直到现在：则表达式都是基于文本的编辑器和搜索工具中的一个重要部分

各个编程语言的正则表达式工具

- 正则表达式是一种通用的字符串表达框架
- Java：util.regrex包
- Linux / Unix：grep awk sed等
- Perl：正则表达式模块
- C++：GNU Regex Library，Boost.Regex，PCRE，PCRE++
- Python：re库

字符串匹配的应用

- 许多计算机应用的最基本操作是字符串匹配：
  - 如编辑器或字处理系统工作时，在文本中查找单词或句子（中文字 或词语），在程序里找拼写错误的标识符等
  - email 程序的垃圾邮件过滤器，google 等网络检索系统
  - 各种防病毒软件，主要靠在文件里检索病毒模式串

字符串匹配的应用

- 在分子生物学领域：DNA 内有四种碱基：腺嘌呤(adenine)，胞嘧啶(cytosine)，鸟嘌呤(guanine)，胸腺嘧啶(thymine)。它们的不同组合翻译形成不同的氨基酸、蛋白质和其他更高级的生命结构
  - DNA 片段可看作是A、T、C、G、U构成的模式，如 ACGGCTAATTC
  - DNA的酶切过程



字符串匹配算法设计

- 串匹配的基础是逐个比较字符
  - 如果从目标串的某个位置开始，模式串的每个字符都与目标串里的 对应字符相同，这就是一个匹配
  - 如果出现一对不同字符，就是不匹配
- 算法设计的关键：
  - 1，怎样比较字符；
  - 2，发现不匹配后下一步怎么做
- 对上述两点采用不同的策略，就形成了不同的算法

朴素算法

- 朴素匹配算法：
  - 1，从左到右匹配；
  - 2，发现不匹配时，考虑目标串里的下一位置是否存在匹配
- 朴素匹配算法简单，易理解，但效率低
- 最坏情况是每趟比较都在最后出现不等，最多比较 n - m + 1 趟，总比较次数为 m*(n - m + 1)，所以算法时间复杂性为 O(m*n)

最坏情况的一个实例

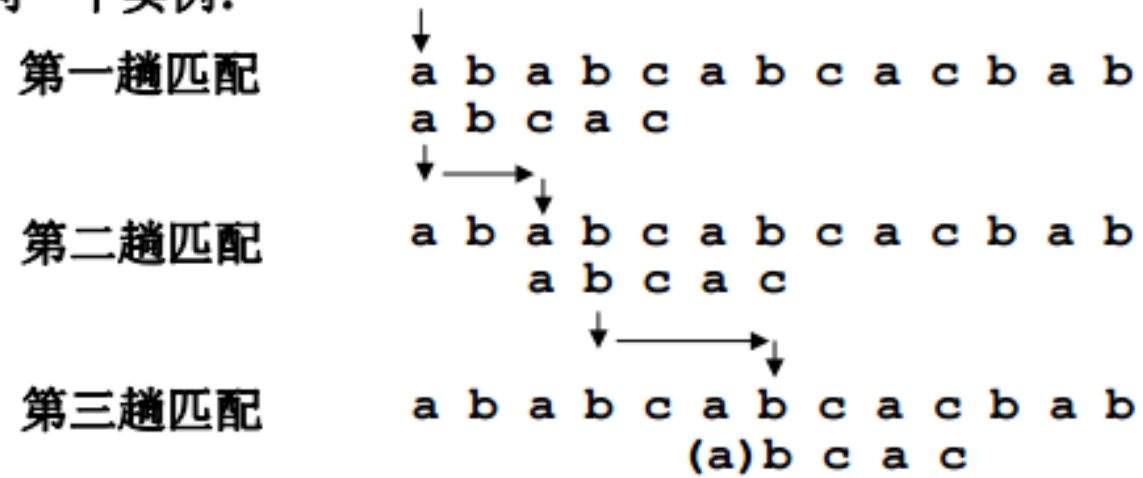
目标串：000000000000000000000000000000001

模式串：00000001

KMP算法介绍

- KMP 算法的基本想法：在匹配失败时，利用已做匹配中得到的信息， 把模式串尽可能前移。匹配中只做不得不做的字符比较，不回溯

处理同一个实例：

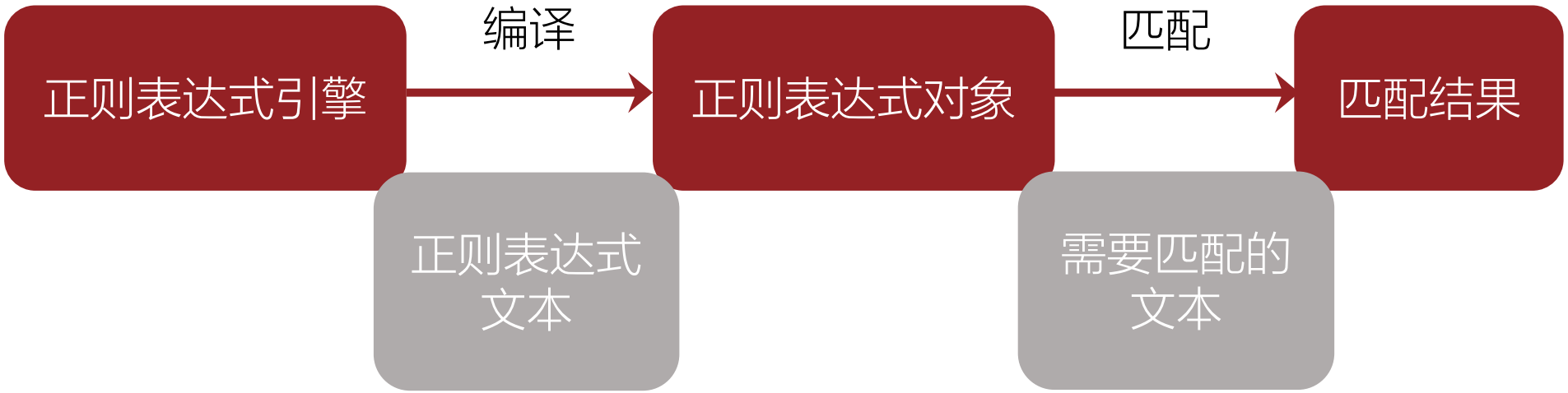


利用第一次匹配失败所获得的信息，第二次直接就前进了两格。

- 具体算法过于复杂，不做详细介绍

正则表达式匹配过程

- 依次拿出表达式和文本中的字符比较，如果每一个字符都能匹配，则匹配成功
- 一旦有匹配不成功的字符则匹配失败



正则表达式语法

- 正则表达式语言的基本成分是字符集里的普通字符，另外还有几种特殊的组合结构，最基本的是：
  - 正顺序组合 αβ
  - 选择组合 α | β
  - 星号 α*
  - 括号 ()：将括号中的内容当成一个单位，一起操作

要注意的是，这里的*的含义（重复任意次）和cmd里面的*（任意串）含义不同。

正则表达式语法

- 举例：
  - abc
  - a(b*)(c*)
  - a((b|c)*)

- 这里不需要cmd中的通配符？和 *
  - 通配符？可以用 | 描述（由于字符集是有穷集）
  - 通配符 * 可以通过 | 和星号描述

正则表达式常用操作符

操作符	说明	实例
.	表示任何单个字符	
[ ]	字符集，对单个字符给出取值范围	[abc]表示a、b、c，[a-z]表示a到z单个字符
[^]	非字符集，对单个字符给出排除范围	[^abc]表示非a或b或c的单个字符
*	前一个字符0次或无限次扩展	abc* 表示 ab、abc、abcc、abccc等
+	前一个字符1次或无限次扩展	abc+ 表示 abc、abcc、abccc等
?	前一个字符0次或1次扩展	abc? 表示 ab、abc
|	左右表达式任意一个	abc|def 表示 abc、def

正则表达式常用操作符

操作符	说明	实例
{m}	扩展前一个字符m次	ab{2}c表示abbc
{m,n}	扩展前一个字符m至n次（含n）	ab{1,2}c表示abc、abbc
^	匹配字符串开头	^abc表示abc且在一个字符串的开头
$	匹配字符串结尾	abc$表示abc且在一个字符串的结尾
( )	分组标记，内部只能使用 | 操作符	(abc)表示abc，(abc|def)表示abc、def
\d	数字，等价于[0-9]	
\w	单词字符，等价于[A-Za-z0-9_]	

匹配集合与补集

- 字符组表达式 [...] 匹配括号中列出的任一个字符
  - [abc] 可以匹配字符 a 或 b 或 c
  - 区间形式 [0-9] 是顺序列出的缩写，匹配所有十进制数字
  - 字符 [0-9a-zA-Z] 匹配所有字母（英文字母）和数字
- [^...] 中的 ^ 表示求补，这种模式匹配所有未在括号里列出的字符
  - [^0-9] 匹配所有非十进制数字的字符
  - [^ \t\v\n\f\r] 匹配所有非空白字符（非空格/制表符/换行符）
  - 如果需要在字符组里包括 ^，就不能放在第一个位置，以免混淆含义。可以写\^恢复其本来含义；如 果需要在字符组包括 -]，也必须写 \- 或 \]

常规字符匹配

- 非特殊字符称为常规字符，是描述正则表达式的基础。正则表达式串里的常规字符在匹配中只与自己匹配

r1 = re.compile('abc’)

- 只有在pattern中运用了之前所说的特殊字符之后，才能使得正则表达式获得更加强大的匹配功能。

常用字符组

1. 为了方便，re 用换意串形式定义了几个常用字符组，包括：
    1. \d：与十进制数字匹配，等价于 [0-9]
    2. \D：与非十进制数字的所有字符匹配，等价于 [^0-9]
    3. \s：与所有空白字符匹配，等价于 [\t\v\n\f\r]
    4. \S：与所有非空白字符匹配，等价于 [^ \t\v\n\f\r]
    5. \w：与所有字母数字字符匹配，等价于 [0-9a-zA-Z]
    6. \W：与所有非字母数字字符匹配，等价于 [^0-9a-zA-Z]
2. 圆点字符 . 可以匹配任意一个字符
    1. a..b 匹配所有以 a 开头 b 结束的四字符串

重复

- 常希望写重复匹配的模式（部分），任意次或若干次重复：
- 基本重复运算符是 *，α* 与 α 的 0 次或任意多次出现匹配
  - 例如[1-9][0-9]*或者[1-9]\d*代表任意大的正整数
- 与 * 略微不同的重复运算符 + 表示 1 次或多次重复
  - 例：允许以0开头时，描述正整数的一种简单写法'\d+'
  - 'α +'，等价于 'αα* '

重复

- 常希望写重复匹配的模式（部分），任意次或若干次重复

- 用？运算符表示？表示 0 次或 1 次重复
  - 例，描述整数（表示整数的字符串）的一种简单模式 '-?\d+'，其前面的'-?'部分表示可以以负号开头。

- 确定次数的重复用 {n} 表示，α{n} 与 α 匹配的串的 n 次重复匹配
  - 描述北京常规的固话号码：'(010-)?[2-9][0-9]{7}'表示可选010前缀，2-9开头，后面的任意数字加起来一共可以有七位。

重复

- 常希望写重复匹配的模式（部分），任意可以在指定范围内重复

- 重复范围用 {m,n} 表示，α{m,n} 与 α 匹配的串的 m 到 n 次重复匹配
  - a{3,7} 与 3 到 7 个 a 构成的串匹配
  - go{2,10}gle 与 google, gooogle, ..., goooooooooogle 匹配
  - 重复范围中的 m 和 n 均可以省略，α{,n} 表示 α{0,n}，而 α{m,} 表示 α{m,infinity}。
  - 之前介绍的几种重复都可以用这个形式表示： α{n} 等价于 α{n,n}，α? 等价于 α{0,1} α* 等价于 α{0,infinity}，α+ 等价于 α{1,infinity}

重复

- 要特别注意的是，所有涉及重复的地方都会涉及一个概念叫做'贪婪匹配'

- 例如：字符串为abcddddddddd，正则表达式为abcd{3:5}
- 那么，正则表达式可以匹配abcddd、abcdddd、abcddddd
- 但是一般程序设计语言中，大多会默认正则表达式进行贪婪匹配，即可以匹配的多个互相有包含关系的时候，只匹配最大的那一个

- 在后面讲述Python中正则表达式的用法时，我们会有具体的例子

括号

- 如果没有括号，则运算为顺序运算。所有运算符仅仅作用一个单位。
- 在正则表达式中使用括号()与一般括号的含义一样：即将内部的东西形成一个整体，再整体进行外部的运算

- 例如：ab*代表一个a与许多个b连在一起
- 而使用(ab)*代表任意多个(ab)连在一起

- 在正则表达式较大、较复杂的时候，要特别注意运算的规则，以及运用括号达到自己真正需要表达的意思。

首尾匹配

- 行首匹配：以 ^ 符号开头的模式，只能与一行的前缀子串匹配
  - 例如'^for'只能匹配以'for'为前缀的一行。如果将其与一行'books for children'匹配，将得到 None
- 行尾匹配：以 $ 符号结尾的模式，只与一行的后缀匹配
  - 例如'fish$'只匹配以'fish'为后缀的一行。如果将其与一行'cats like to eat fishes'匹配，将得到 None
- 很多语言中^$都分别代表行首/行尾的标记。例如文本编辑器VIM

Python正则表达式语法总结

- 刚才，我们已经介绍了所有的特殊字符。进行一个总结：
- 转义字符\
- 选择字符|
- 集合字符[、]、^、-
- 通配符.
- 重复字符*、?、+、{}
- 括号（）
- 首尾字符^、$
- 以及其他一些被\转义的普通字符可以发挥特殊的作用，如\D\d\S\s等等

首尾匹配

- 注意：上面所说的^与$代表的一行的开头与结束，而不是整个字符串的开头与结束。如果一个字符串中出现了多个换行符号，则模式或许可能与多个行首、行尾形成匹配。
- 串首匹配：\A 开头的模式只与被匹配串的前缀匹配
  - 例如'\Afor'即只能与以'for'开头的字符串形成匹配。
- 串尾匹配：\Z 结束的模式只与被匹配串的后缀匹配

正则表达式语法实例

正则表达式	对应字符串
P(y|yt|yth|ytho)?n	’Pn’、’Pytn’、’Pythn’、’Python’
Python+	’Python’、’Pythonn’、’Pythonnn’……
Py[th]on	’Pyton’、’Pyhon’
Py[^th]?on	’Pyon’、’Pyaon’……~~’Pyton’~~、~~’Pyhon’~~……
Py{:3}n	’Pn’、’Pyn’、’Pyyn’、’Pyyyn’……

经典正则表达式语法实例

正则表达式	说明
^[A-Za-z]+$	由26个字母组成的字符串
^[A-Za—z0-9]+$	由26个字母和数字组成的字符串
^-?\d+$	整数形式的字符串
^[0-9]*[1-9][0-9]*$	正整数形式的字符串
[1-9]\d{5}	中国境内邮政编码，6位
[\u4e00-\u9fa5]	匹配中文字符
\d{3}-\d{8}|\d{4}-\d{7}	国内电话号码

匹配IP地址的正则表达式

- IP地址字符串形式的正则表达式（IP地址分为4段，每段0-255）

  \d+.\d+.\d+.\d+ 或 \d{1,3}.\d{1,3}.\d{1,3}.\d{1,3}

  精确写法

0-99	[1-9]?\d
100-199	1\d{2}
200-249	2[0-4]\d
250-255	25[0-5]

  匹配Ip地址

(([1-9]?\d|1\d{2}|2[0-4]\d|25[0-5]).){3}([1-9]?\d|1\d{2}|2[0-4]\d|25[0-5])

Python的正则表达式

- 正则表达式是独立于语言存在的，不同语言有不同的实现方式
- | 与 * 是正则表达式的通用语法，Python中使用正则表达式时，包括但不限于这些简单的语法
- Python：内置模块**re**

正则表达式的表示类型

- 原生字符串（raw string）是 Python 里一种写字符串的形式
- 比如：**R’abcdefg’** 或 **r’C:\courses\python\progs’**

- **R’C:\courses\python\progs’** 的等价写法是：

  ’C:\\courses\\python\\progs’’



元字符

- re 规定了一组特殊字符，称为元字符，它们在匹配字符串时起着特殊的作用。这种字符一共 14 个：

  . ^ $ * + ? \ | { } [ ] ( )

- 原始字符串中，所有字符表达原来的内容
- 普通字符串中，只有少量如 \ 的转义字符代表特殊内容，别的字符表达原来的内容
- 正则表达式中，上述所有特殊字符都代表特殊内容
- 表示能力：正则表达式 > 普通字符串 > 原始字符串



pattern 类

- Python的正则表达式包re中，最基本的类就是 pattern
- 正则表达式的对象可以从普通的字符串生成，语句为：

r1 = re.compile(普通字符串)

re模块

- 利用特殊字符与普通字符进行搭配，就可以设计出各种各种的正则表达式，以满足我们的需要。
- 如前面所介绍的，当我们以正则表达式的语法写出了一个普通的字符串，并执行**r1 = re.compile**(普通字符串) 语句之后，**r1**就成为了一个正则表达式对象，可以按照正则表达式的规则来匹配各种字符串。
- 下面我们将介绍**re**模块中，正则表达式的各种使用方式。我们可以调用各种函数，来使用这些正则表达式。

- 事实上，使用上述几个函数的方式有两种——将其作为一般的函数，或者将其作为正则表达式对象的方法。
- 例如使用**search**函数的时候，我们有两种等价的方式：
  - **re.search(pattern,字符串)**#直接把普通字符串pattern当正则表达式语使用
  - **r1=re.compile(pattern)**#先用**pattern**生成正则表达式对象r1

    r1.search(字符串)
- 这两种写法是等价的。如果只需要使用一次正则表达式，可以用第一种写法，以节省时间。如果要多次使用，不妨先将r1生成一个正则表达式以便多次复用。

re模块

- re模块中的函数有：

  - **search**
  - **findall**
  - **finditer**
  - **match**
  - **fullmatch**
  - **split**
  - **sub**
  - **subn**
  - **pattern**

re模块

- search

- 用法是**re.search(pattern,字符串)**
- 对字符串进行模式搜索，如果找到第一个匹配的位置，返回一个**MatchObject**对象；这个对象中储存了匹配位置、匹配内容等信息。后面我们会详细讲。
- 若该字符串没有模式匹配，则返回**None**。
- 例如：手机评论数据中出现很多的关于屏幕、音频、拍照之类的功能评价，那么在实际数据分析中，我们可能需要看评论中是否会出现'屏幕'。**re.search**()函数便是根据需要匹配的模式进行搜索的函数

手机评论数据集

- 数据集是从某手机评论网站上爬下来的，共25068条记录。每条记录都已经分好词、去除标点符号，数据可以直接使用。以下显示前五条分好词记录：

```
1. 1 屏幕 还 算 清晰 阳光 下 还 算 不错 2 屏幕 的 按键 设计 新颖 3 有 分量 很 重 男人 就 应该 是 这样 的 我 不 喜欢 轻飘飘 的 手机
2. 还 算 清晰 灵敏 而且 灵敏度 可以 调节
3. 320 万 像素 摄像头 支持 LED 闪光 和 自动 聚焦 拥有 35 毫米 耳机 接口 兼容 多种 影音 格式文件 播放
4. 水货 性价比 超高 行货 坑爹 硬件 做工 很 好
5. 1 屏幕 很 给 力 很大 气 2 按压 式 触摸屏 很 舒服 3 拍照 很 清晰 4 音质 很 干净
屏幕 大 分辨率 堪比 IP4 牌子 比较 好 拿到 大街 上 一般 不会 撞机 比 那些 街机 好多 了 通话 没 的 说 价格合理
```

re模块

- search

```
import re
comment="这个手机很好，屏幕的质量很好"
pattern="屏幕.*[好坏]"
re.search(pattern,comment)
```

```
<_sre.SRE_Match object; span=(7, 14), match='屏幕的质量很好'>
```

- 我们用正则表达式'屏幕.*[好坏]'去**search**我们的评论，得到了一个**MatchObject**，其中记录了匹配的内容'屏幕的质量很好'与匹配的位置（从第7到第14位）

re模块

- search

```
import re
comment1="这个手机很好，屏幕的质量很好"
comment2="这个手机就和屎一样，屏幕实在是太坏了！"
r1=re.compile("屏幕.*[好坏]")
r1.search(comment1)
r1.search(comment2)
```

```
<_sre.SRE_Match object; span=(7, 14), match='屏幕的质量很好'>
<_sre.SRE_Match object; span=(10, 17), match='屏幕实在是太坏'>
```

- 如前所述，需要多次复用我们可以先生成一个正则表达式对象，以提高效率。

re模块

- search
- 要注意的是，**search**进行的是贪婪匹配，即寻找最大的匹配

```
import re
comment="这个屏幕很坏，还不够我家厕所窗户好！"
pattern="屏幕.*[好坏]"
re.search(pattern,comment)
```

有两段（有包含关系）都可以匹配pattern：
'屏幕很坏'
'屏幕很坏，还不够我家厕所窗户好'
贪婪匹配选择包含关系中最大的一段

```
<_sre.SRE_Match object; span=(2, 17), match='屏幕很坏，还不够我家厕所窗户好'>
```

- 为了避免上述的错误，可以把中间的*改为**{：n}**，限制间隔的字数

re模块

- search

```
import re
comment="屏幕很好？开玩笑吧！明明屏幕很坏！"
pattern="屏幕.[好坏]"
re.search(pattern,comment)
```

Pattern代表屏幕与[好坏]只相隔一个字。
有两个独立的部分可以匹配pattern：
'屏幕很好'
'屏幕很坏'
只选择第一个匹配

```
<_sre.SRE_Match object; span=(0, 4), match='屏幕很好'>
```

- 要注意的是，**search**只是寻找从左到右第一个与正则表达式匹配，而不是找到所有匹配位置。贪婪指的是匹配的最大，而不是最多。
- 遇到这个问题应该用**findall**函数

re模块

- findall
- 刚才介绍的**search**函数只能从左到右找出第一个最大的匹配。而**findall**函数可以找出所有匹配，并把它们从左到右作为一个列表返回。如果无匹配，返回空列表。

```
import re
comment="屏幕很好？开玩笑吧！明明屏幕很坏！"
pattern="屏幕.[好坏]"
re.findall(pattern,comment)
```

注意这里'屏幕'与'好/坏'中间只隔了一个通配符'.'，所以能帮我们得到正确答案。

```
['屏幕很好', '屏幕很坏']
```

re模块

- findall
- 要注意，**re**模块的规则都是贪婪匹配的，**findall**也不例外

```
import re
comment="屏幕很好？开玩笑吧！明明屏幕很坏！"
pattern="屏幕.*[好坏]"
print re.findall(pattern)
```

注意这里'屏幕'与'好/坏'中间隔了任意个通配符'.*'，所以会导致错误。

```
['屏幕很好？开玩笑吧！明明屏幕很坏']
```

- 贪婪匹配的含义不是要找出'最多'的匹配，而是要在每一次匹配的时候找到'最大'的匹配。这提醒我们，一定要当心使用*，最好都改为由限定范围的{：n}

re模块

- finditer
- **finditer**函数与**findall**函数的作用基本是一样的。不同的是**finall**返回的是一个列表，**finditer**返回的是一个迭代器。迭代器用完之后可以自动释放内存。
- 在处理的数据较大的时候，**finditer**或许会发挥其作用。但如果数据较小、不重视内存，或者手动释放，则**finditer**的功能可以完全被**findall**取代。

re模块

- match
- 用法是**re.match**(pattern，字符串)，和**re.search**()类似，唯一不同的是**re.match**()只从头开始匹配，**re.search**()可以匹配任一位置的字符串。
- 即如果符合正则表达式的字符串如果出现在开头，那么使用**re.match**()后会返回**MatchObject**对象，但如果出现在中间位置或没有，则返回**None**。
- 也可以等价于在**search**函数的**pattern**前增加一个表示字符串开头的'**\A**'字符
- **Match**函数也是贪婪的。并且，它显然不存在匹配第一个还是多个这样的问题。因为从头开始的贪婪匹配显然只能有一种结果。

re模块

- match

```
review1 = '屏幕 很 给 力 很大 气 2 按压 式 触摸屏 很 舒服 3 拍照 很 清晰 4 音质 很 干净'
if re.match("屏幕", review1) is None:
    print "无匹配"
else:
    print "匹配成功"
```

匹配成功

```
review2 = '1 屏幕 很 给 力 很大 气 2 按压 式 触摸屏 很 舒服 3 拍照 很 清晰 4 音质 很 干净'
if re.match("屏幕", review2) is None:
    print "无匹配"
else:
    print "匹配成功"
```

这个例子说明了**match**只是从头开始匹配

无匹配

re模块

- fullmatch
- 用法和**match**类似，但它不但是'从头匹配'，而是'从头到尾'匹配
- 可以说，**fullmatch**就是用来检测**pattern**与整个**string**是不是匹配的

```
import re
string1="abcd1234"
string2="abcd1234k"
pattern="[a-z]*[0-9]*"
print re.fullmatch(pattern,string1)
print re.fullmatch(pattern,string2)
```

String1是整体与pattern匹配的
String2只有前缀与pattern匹配

```
<_sre.SRE_Match object; span=(0, 8), match='abcd1234'>
None
```

re模块

- split
- 一般的字符串也有**split**函数，用法是**string.split(sep=分隔符)**
- 则得到一个将字符串按照分隔符划分而成的表格：

```
string="a,bc,d,efg,hi,j,k,l"
lst=string.split(sep=",")
print lst
```

```
['a', 'bc', 'd', 'efg', 'hi', 'j', 'k', 'l']
```

- 但是只能用一种字符串做为**sep**来进行切分，功能不够强大

re模块

- split
- 使用**re.split(pattern，string)**可以将**string**中所有与**pattern**匹配的东西做为分割符，继而将字符串分割为一张表。

```
import re
string="我c你w大发财uu一个好k吃大餐"
pattern="[a-z]*"
re.split(pattern,string)
```

```
['我', '你', '大发财', '一个好', '吃大餐']
```

- 例如上例以所有小写字母为分隔符划分了字符串，形成列表

re模块

- split
- 要注意**re**中函数都是贪婪匹配的，**split**也不例外

```
import re
string="aaaabbdbcnoiwnoiqwxniowqndwqio"
pattern="[a-z]*"
re.split(pattern,string)
```

```
['', '']
```

- 例如上例以所有小写字母重复任意次为分割符，那么所有内容都是分割符。所以，**string**就变成了两个空字符串

re模块

- split
- **split**函数还可以指定**maxsplit**参数，规定最大被划分多少次

```
import re
string="我c你w大发财uu一个好k吃大餐"
pattern="[a-z]*"
re.split(pattern,string,maxsplit=3)
```

```
['我', '你', '大发财', '一个好k吃大餐']
```

- **maxsplit=3**表示只取前三个与**pattern**匹配的来分隔，即将字符串划分为4份。注意不要以为**maxsplit=3**就是划分为3份！

re模块

- sub

re.sub(pattern,repl,字符串)方法用于对符合正则表达式字符串的替换。

像是前面'触摸屏'、'屏幕'可认为是同义词的情况，是可以进行词替换的，这样可以减少进入训练模型的特征维度。

```
import re
pattern="(触摸屏)|(显示屏)"
comment="广告说这个触摸屏很好，推销的人也说屏幕很好，但我觉得这显示屏糟透了"
print re.sub(pattern,"屏幕",comment)
```

```
'广告说这个屏幕很好，推销的人也说屏幕很好，但我觉得这屏幕糟透了'
```

re模块

- sub

可以指定一个**count**的参数，限制最大的匹配替换次数。

```
import re
pattern="(触摸屏)|(显示屏)"
comment="广告说这个触摸屏很好，推销的人也说屏幕很好，但我觉得这显示屏糟透了"
print re.sub(pattern,"屏幕",comment,count=1)
```

```
'广告说这个屏幕很好，推销的人也说屏幕很好，但我觉得这显示屏糟透了'
```

上例中，由于指定了**count=1**，所以只有第一个匹配'触摸屏'被替换了，第二个匹配的'显示屏'并没有被替换。

re模块

- **subn**

subn与**sub**的使用方式与结果介乎是一样的，不同的是，**subn**返回的对象是一个被替换后的字符串与替换次数的两元组：

```
import re
pattern="(触摸屏)|(显示屏)"
comment="广告说这个触摸屏很好，推销的人也说屏幕很好，但我觉得这显示屏糟透了"
print re.subn(pattern,"屏幕",comment)
```

```
('广告说这个屏幕很好，推销的人也说屏幕很好，但我觉得这屏幕糟透了', 2)
```

上例返回的元组是替换后的字符串以及一共替换了两次的信息

re模块

- re**函数总结**（对于一个正则表达式与一个字符串）：
- **fullmatch**二者是否匹配，**match**检查是否与字符串前缀匹配
- **search**找出字符串第一个匹配的部分
- **findall**找出所有匹配的部分形成一个列表，**finditer**找出所有迭代部分形成一个迭代器
- **split**找出所有匹配部分，用其作为分隔符划分字符串
- **sub**将所有匹配部分替换为指定字符串，**sub**额外返回替换次数
- 要注意所有的匹配都带有贪婪匹配的性质！

目录

第一部分 手机评论数据集

第二部分 字符串的性质

第三部分 串匹配

第四部分 正则表达式

第五部分 Python的re模块

第六部分 正则表达式应用

正则表达式应用

- 去掉标点符号

通常来说，在拿到中文文本数据之后，首先把标点符号去掉，然后再进行分词。而中文数据中的标点符号不仅有用英文的还有中文，我们可以自定义标点符号的正则表达式来将标点符号替换为''，这样就可以把标点符号去掉了。

正则表达式应用

- 去掉标点符号

要注意的是：中文的标点符号大多是没有歧义的。而英文的标点符号大多数本来就是正则表达式中有特殊含义的符号。所以，必须在前面运用转义字符'\'才能将特殊含义转换为其原来的含义。

```
punctuation_e = "[\s+\.\!\/_,$%^*(+\"\']+"
```

然后使用**re.sub**函数即可以将标点符号替换为空字符''



正则表达式应用

- 重组分词

用分词工具分词之后的语句中，仍然有很多分错的情况。对于分错的解决方法有两种，第一种是将分错的词作为词典加入分词工具的词典中再重新分词；第二种是用正则表达式将这个词提取出来进行替换。第一种方法其实更常用一些，但是有时候可能仅仅需要暂时替换查看效果，那么所有句子都重新分词的成本看起来就比较大了。所以第二种方法也是需要掌握的。

正则表达式应用

- 定义一个函数，求字符串中出现的所有整数之和

```
def sumInt(fname):
    re_int = '\\b(0+|[1-9]\d*)\\b'
    inf = open(fname)
    if inf == None:
        return 0
    ilist = map(int, re.findall(re_int, inf.read())) # 可改为分行读入
    s = 0
    for n in ilist:
        s += n
    return s
```

正则表达式应用

- 刚才我们介绍过，**re.sub(pattern,repl,string)**可以把**string**中所有与**pattern**匹配的内容全部替换为**repl**。
- **re.sub**还有一个特殊的功能，如果使用**re.sub(pattern,function,string)**，其中**function**是一个函数。则可以把所有匹配的对象全部替换为作用**function**后的对象。

正则表达式应用

- 利用这个功能，我们可以实现一些更加强大的效果。
- 例如把Python中自定义的变量、函数、对象都加上前缀'**my_**'

```
from keyword import iskeyword

ident = r"\b[a-zA-Z_]\w*\b" # 标识符由字母和 _ 开头

def add_prefix(name) :
    return name if iskeyword(name) else "my_" + name

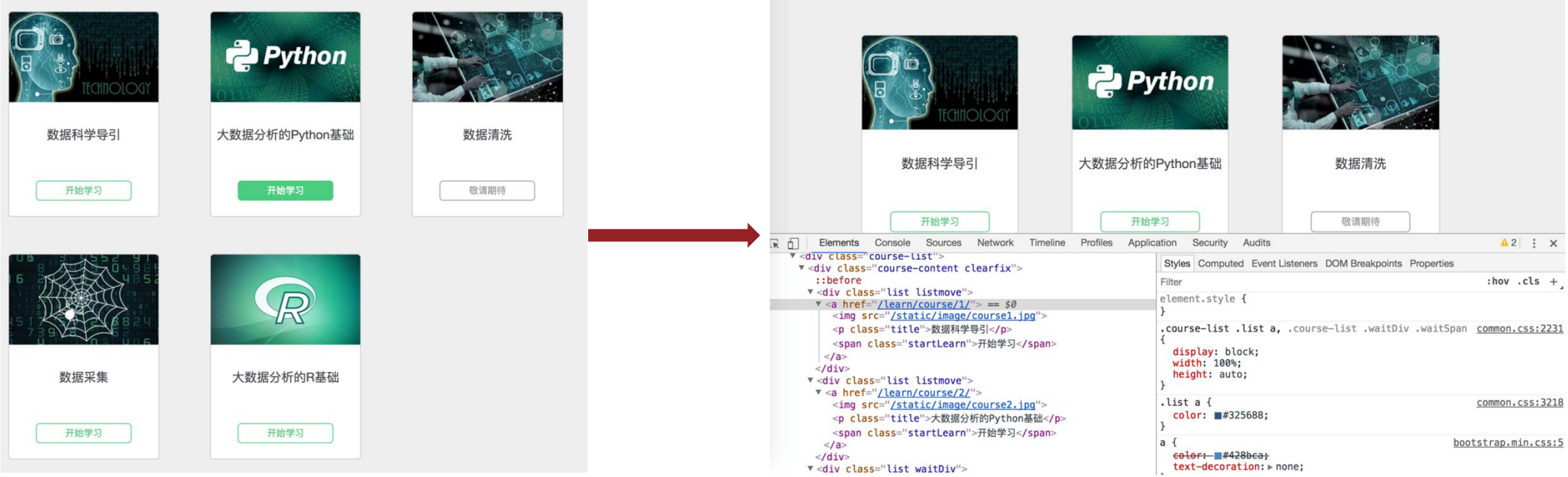
... ...

  modified = re.sub( ident, add_prefix, prog_text)

... ...
```

正则表达式解析HTML

- HTML可以看作是带有特殊标签的纯文本文件，使用正则表达式，对标签进行匹配，抽取数据
- 尝试抽取数据嗨客平台上所有课程的名称



使用正则表达式抽取数据

- 显示网页源代码

```
<div class="course-list">
    <div class="course-content clearfix">


                <div class="list listmove">
                    <a href="/learn/course/1/">
                        <img src="/static/image/course1.jpg">
                        <p class="title">数据科学导引</p>
                        <span class="startLearn">开始学习</span>
                    </a>
                </div>


                <div class="list listmove">
                    <a href="/learn/course/2/">
                        <img src="/static/image/course2.jpg">
                        <p class="title">大数据分析的Python基础</p>
                        <span class="startLearn">开始学习</span>
                    </a>
                </div>


                <div class="list waitDiv">
                    <span class="waitSpan">
                        <img src="/static/image/course3.jpg">
                        <p class="title">数据清洗</p>
                        <span class="waitLearn">敬请期待</span>
                    </span>
                </div>
```

- 名称嵌在 和 之中
- 写出对应的正则表达式

  (.+?)
- 字符 . 负责匹配除了换行字符意外的任何字符
- 字符 +? 负责字符匹配的范围
- () 区别于字符集合 []， 代表捕获组
- (.+?) 表示匹配标签之间的所有内容

使用正则表达式抽取数据

- 可以利用findall方法抽取数据

```
with open('hackdata.html','r') as f:
    content = f.read()
import re
pattern = re.compile(r'<p class=\"title\">(.+?)</p>')
course_array = pattern.findall(content)
for item in course_array:
    print item
```

```
数据科学导引
大数据分析的Python基础
数据清洗
数据采集
大数据分析的R基础
```

使用正则表达式抽取数据

- 可以利用pattern进行逐行匹配

- 请尝试抽取每个课程的课程状态

```
import re
import io

html_data = io.open('hackdata.html','r',encoding='utf8')
for line in html_data:
    target = pattern.search(line)
    if target:
        all_content = target.group(0)
        content = target.group(1)
        print all_content + "      "+ content
```

```
<p class="title">数据科学导引</p>      数据科学导引
<p class="title">大数据分析的Python基础</p>      大数据分析的Python基础
<p class="title">数据清洗</p>      数据清洗
<p class="title">数据采集</p>      数据采集
<p class="title">大数据分析的R基础</p>      大数据分析的R基础
```

使用BeautifulSoup模块抽取数据

- 网页是具有树形结构的数据，每个标签都是树形结构的一个节点
- 定位节点，找到与节点相关的数据

```
import io
from bs4 import BeautifulSoup
html_data = io.open('hackdata.html','r',encoding='utf8')
soup = BeautifulSoup(html_data)
print soup.head
```

```
<head>
<meta charset="utf-8">
<meta content="IE=edge" http-equiv="X-UA-Compatible">
<meta content="width=device-width, initial-scale=0.3, minimum-scale=0.3"
name="viewport">
<link href="/static/image/favicon.ico" rel="shortcut icon" type="image/x
-icon"/>
<link href="/static/bootstrap33/css/bootstrap.min.css" rel="stylesheet">
<link href="/static/css/common.css" rel="stylesheet">
<title>数据嗨客 全球首家大数据教育、竞赛、服务平台</title>
</link></link></meta></meta></meta></head>
```

使用BeautifulSoup模块抽取数据

- 调用 findAll() 函数，去定位标签 ，抽取所有与之相关的数据

```
alllines = soup.findAll('p')
for line in alllines:
    content = line.contents[0]
    print content
```

```
                    晏晓东


个人信息
我的练习
我的评论
我的教室
个人成就
退出
数据科学导引
大数据分析的Python基础
数据清洗
数据采集
大数据分析的R基础
© 2017 HackData - 京ICP备15065775号-2
```

使用BeautifulSoup模块抽取数据

- 进一步确定标签属性class值为title

```
alllines1 = soup.findAll('p','title')
for line in alllines1:
    content = line.contents[0]
    print content
```

```
数据科学导引
大数据分析的Python基础
数据清洗
数据采集
大数据分析的R基础
```

练习

- 尝试提取某网站书籍名称、作者、出版社、价格信息?

正则表达式解析HTML

- 解析网页到底用不用正则解析?
- 观点一：不用
  - HTML不是正则语言，无法真正地用正则表达式解析
  - 虽然直接用正则更快，但更麻烦，容易出错
  - 类似爬虫，HTML复杂度的不可预见性
  - 合法的HTML无法用正则解析，更不用说会碰到不合法但是浏览器可以支持的HTML
  - 网页是动态的，今天写的不见得明天有用

正则表达式解析HTML

- 解析网页到底用不用正则解析?
- 观点二：用
  - 获取链接建议用正则表达式
  - 解析整个HTML非常耗时，哪怕使用最快的lxml，相差四倍以上
  - 可以总结一些不正常的URL，过滤掉即可

建议：简单网页用正则、复杂网页正则搞不定的用lxml等解析器

正则表达式解析HTML

- HTML解析器
  - ElementTree是标准库的一部分
  - BeautifulSoup是一个流行的第三方库
  - lxml是一个功能齐全基于c的快速解析库

总结

- 正则表达式简介
- 正则表达式匹配过程及表示类型
- 正则表达式语法及应用
- re模块：search,findall,finditer,match,fullmatch
- re模块：split,sub,subn,pattern
- 正则表达式解析HTML



第十三部分 日期数据的处理

datatime模块
应用

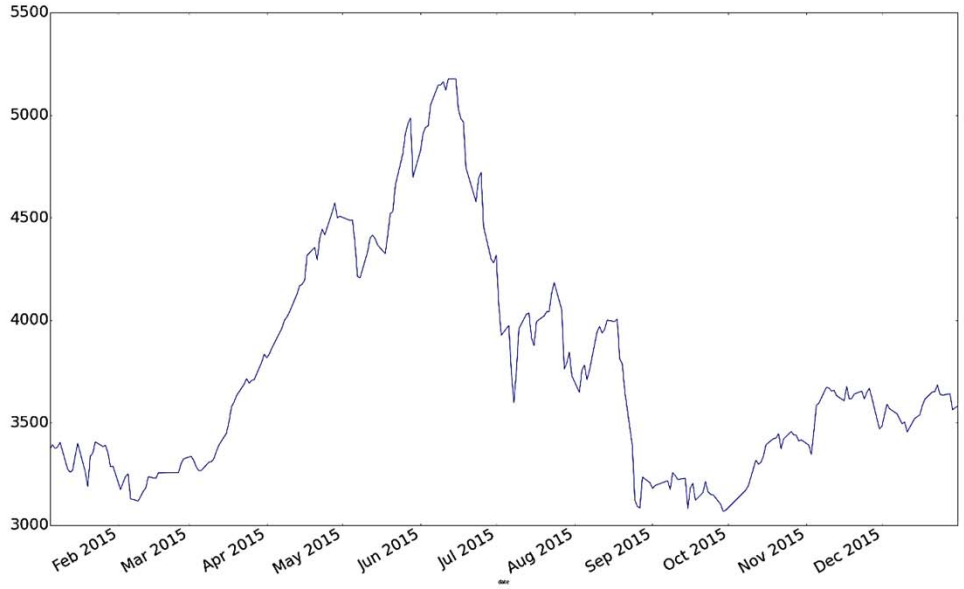
上证指数

- 上海证券综合指数，英文是：Shanghai（securities）composite index. 通常简称：“Shanghai composite index”（上证综指）。“上海证券综合指数”它是上海证券交易所编制的，以上海证券交易所挂牌上市的全部股票为计算范围，以发行量为权数综合。上证综指反映了上海证券交易市场的总体走势。



上证指数

- 数据集抽取的是2015年1月1日至2016年1月1日上证指数每日的成交量、开盘、收盘、最高、最低指数



读取数据

- 下面，我们将上证指数的数据读取进来

```
path = "sh.csv"

f = open(path, "r")
reader = csv.reader(f)
sh_content = [con for con in reader]
f.close()

print sh_content[0]
print sh_content[1]
```

```
['date', 'open', 'close', 'high', 'low', 'volume', 'code']
['2015-01-05', '3258.63', '3350.52', '3369.28', '3253.88', '531352391.0', 'sh']
```

- 每一行的数据包括日期时间与股票的open、close、high、low等等价格

datetime模块

- 可以看到，时间变量"date"下表示具体日期的值"2015-01-05"是一个字符串。其实如果所有时间都是这种"YYYY-MM-DD"比较规范的格式，那么是不需要将日期专门转为"datetime"类，普通的字符串处理方法就能够处理大多数规范格式的数据。

- 但实际上，很多日期数据读进计算机的时候是一个字符串，但是不是所有都会是一个格式的。另外，十分关键的一点是，如果用字符串处理方法，那就需要每次都输入想要的日期所在字符串的位置，这是比较容易写错的，而且并不通用。

datetime模块

- datetime是Python一个专门用于时间处理的模块，这个模块主要包含三个类：
  - Time
  - date
  - datetime
- 三个类的方法属性类似，其中time主要包含一天内的小时、分钟、秒、毫秒信息；date类包含年、月、日；datetime则是前两者的结合。数据集中的数据既需要date信息，也需要time信息，所以这章只用到datetime类。熟悉datetime类后其余的time、date都会很好上手。

datetime模块

- 转为datetime类型

datetime.strptime()是专门用于将字符串转换为"datetime"类型的方法，其用法为：

```
datetime.strptime(date_string, format)
```

- **date_string**: 指日期字符串
- **format**: 日期字符串的日期格式

datetime模块

- 转为datetime类型

日期格式"format"用具体例子来说，对于上面的"2015-01-05"的日期格式就是"%Y-%m-%d"。下面为所有表示时间的格式：

- %y 两位数的年份表示，区间为[00, 99]
- %Y 四位数的年份表示，区间为[000, 9999]
- %m 月份 [01-12]
- %d 月内中的一天 [0-31]
- %H 24小时制小时数 [0-23]
- %I 12小时制小时数 [01-12]
- %M 分钟数 [00=59]
- %S 秒 [00-59]
- %a 本地简化星期名称
- %A 本地完整星期名称v
- %b 本地简化的月份名称
- %B 本地完整的月份名称
- %c 本地相应的日期表示和时间表示
- %j 年内的一天 [001-366]
- %p 本地A.M.或P.M.的等价符
- %U 一年中的星期数 [00-53]星期天为星期的开始
- %w 星期 [0-6]，星期天为星期的开始
- %W 一年中的星期数 [00-53]星期一为星期的开始
- %x 本地相应的日期表示
- %X 本地相应的时间表示
- %Z 当前时区的名称
- %% %号本身

datetime模块

- 转为datetime类型

```
import datetime as d
date1 = "2015/01/05"
format1 = "%Y/%m/%d"
t1 = d.datetime.strptime(date1,format1)
t1
```

四位年，两位的月份，两位日期，中间用“\”分隔
所以采用format1 = "%Y/%m/%d"

```
datetime.datetime(2015, 1, 5, 0, 0)
```

datetime模块

- 转为datetime类型

```
import datetime as d
date2 = "2015-01-05 23:00:00"
format2 = "%Y-%m-%d %H:%M:%S"
t2 = d.datetime.strptime(date2,format2)
t2
```

四位年，两位的月份、日期“-”分隔
两位的小时、分钟、秒，中间用“:”分隔
format2 = "%Y-%m-%d %H:%M:%S"

```
datetime.datetime(2015, 1, 5, 23, 0)
```

datetime模块

- 转为datetime类型

```
import datetime as d
date3 = "15-01-05 23:00"
format3 = "%y-%m-%d %H:%M"
t3 = d.datetime.strptime(date3,format3)
t3
```

和上例相比，没有后面的:秒钟
所以format比上例少了后面的”:%S”
format3 = "%Y-%m-%d %H:%M"

```
datetime.datetime(2015, 1, 5, 23, 0)
```

datetime模块

我们把一个str转换为一个datetime类之后，就可以开始使用这个类

```
import datetime as d
date3 = "15-01-05 23:00"
format3 = "%y-%m-%d %H:%M"
t3 = d.datetime.strptime(date3,format3)
print t3.date
```

例如用左边的方法可以判断时间是处于哪一年。

利用这个属性，我们可以对日期数据进行批量的处理，例如把同一年的所有数据都挑出来。

```
2015
```

datetime模块

- datetime对象的属性

  “datetime”对象有各个具体时间单位的属性：

  - datetime.year 获得年份
  - datetime.month 月份
  - datetime.day 日期
  - datetime.hour 小时
  - datetime.minute 分钟
  - datetime.second 秒
  - datetime.microsecond 毫秒

datetime模块

- datetime对象的方法

datetime对象可以直接进行减法

```
import datetime as d
t1 = d.datetime(2017, 4, 11, 23, 0)
t2 = d.datetime(2015, 3, 21, 20, 0)
print(t1-t2)
```

这段代码的计算出两个日期之间相差了**752**天多三个小时
事实上，这返回了一个**timedelta**对象

```
752 days, 3:00:00
```

datetime模块

- datetime对象的方法

要注意的是datetime对象不能进行加法，因为这没有意义

```
import datetime as d
t1 = d.datetime(2017, 4, 11, 23, 0)
t2 = d.datetime(2015, 3, 21, 20, 0)
print(t1+t2)
```

```
Traceback (most recent call last):
  File "<pyshell#48>", line 1, in <module>
    t1+t2
TypeError: unsupported operand type(s) for +: 'datetime.datetime' and 'datetime.dat
```

datetime模块

- datetime对象的方法

isweekday() 是一个很方便的方法，可以直接查看这个日期对象是星期几：

```
import datetime as d
date3 = "17-04-15 23:00"
format3 = "%y-%m-%d %H:%M"
t3 = d.datetime.strptime(date3,format3)
print t3.isoweekday()
```

这段代码的含义是：
2017年4月15日是星期6

```
6
```

datetime模块

- datetime对象的方法

weekday() 是一个类似的方法，但要注意的是，这里星期一是0，星期二是1，星期天是6。这符合计算机从0开始的规则。但是不符合人的直观。

```
import datetime as d
date3 = "17-04-15 23:00"
format3 = "%y-%m-%d %H:%M"
t3 = d.datetime.strptime(date3,format3)
print t3.weekday()
```

这段代码的含义是：
2017年4月15日是星期6，而不是星期5

```
5
```

datetime模块

- datetime对象的方法

fromordinal() 接受一个整型的天数数值，返回一个天数所等价的年数、月数、日数的date对象。

```
import datetime as d
print d.datetime.fromordinal(1000)
```

这段代码的含义是：
如果现在是1年1月1日
则1000天后是3年9月27日

```
datetime.datetime(3, 9, 27, 0, 0)
```

datetime模块

- datetime对象的方法

fromtimestamp() 接收一个浮点型的时间戳，返回一个从1970年起+时间戳的秒数对应的年、月、日的date对象。

```
import datetime as d
print d.datetime.fromtimestamp(1e9)
```

这段代码的含义是：
从**1970**年**1**月**1**日**0**点开始，过了**10**亿秒钟之后，是**2001**年**9**月**9**日**9**点**46**分**40**秒。

```
datetime.datetime(2001, 9, 9, 9, 46, 40)
```

datetime模块

- datetime模块的其他方法

利用now函数，可以获得现在的时间：

```
dt = datetime.datetime.now()  #获取当前时间
print type(dt)
print dt
```

```
'datetime.datetime'
2016-11-29 15:05:38.005000
```

```
print "现在年月日为:%s.%s.%s" %(dt.year, dt.month, dt.day)
```

```
现在年月日为:2016.11.29
```

datetime模块

- 利用now函数与timedelta，可以实现计时功能

```
dt1 = datetime.datetime.now()

dt2 = datetime.datetime.now()

print dt2 - dt1
```

```
0:00:00.529000
```

两次计时之间过去了0.529秒

应用

- 事实上，股票不是每一天都开盘交易的。在节假日，股票市场一般不会交易。
- 下面我们尝试着统计一下一年间各个月份中各有多少个交易日
- 想法：将每一个出现过的日期按照月份分类，统计各个类出现的数量。

应用

- 我们创立一个字典，形式为：{月份：天数}
- 然后，我们遍历所有的日期数据，将其中的月份抽取出来，将其放进字典中。
- 当我们遍历完毕所有数据集之后，我们将字典中所有内容打印出来。

```
stat_dict = {}

for con in sh_content[1:]:

    month = con[0].month

    if str(month) not in stat_dict.keys():
        stat_dict[str(month)] = 1
    else:
        stat_dict[str(month)] += 1

for mon, stat in stat_dict.items():
    print "%s:%s" %(mon, stat)
```

```
11:21
10:17
12:23
1:20
3:22
2:15
5:20
4:21
7:23
6:21
9:20
8:21
```



第十四部分 Numpy基础知识

基本数据结构
ndarray相关操作
结构化的数据
内置操作函数

线性代数模块linalg
随机模块random

Python科学计算生态圈

- Python在科学计算方面有很多不断改良的库，结合其在通用编程方面的强大实力，使其在数据处理、交互探索性计算以及数据可视化方面深受广大编程者的喜爱

- Python有着一个强大的科学计算生态圈，已经完全可以媲美MATLAB、R等特定编程语言/工具



NumPy
Base N-dimensional array package



SciPy library
Fundamental library for scientific computing



Matplotlib
Comprehensive 2D Plotting

IP[y]: IPython

IPython
Enhanced Interactive Console



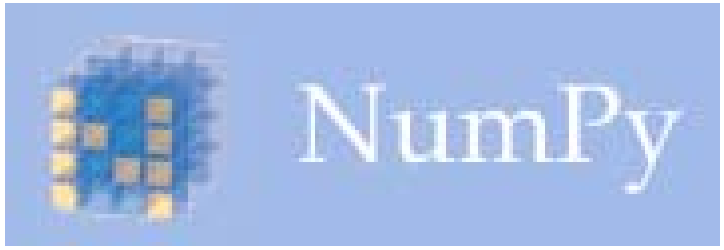
Sympy
Symbolic mathematics



pandas
Data structures & analysis

Numpy 介绍

- Numpy 是一个专门用于矩阵化运算、科学计算的开源Python
- NumPy将Python相当于变成一种免费的更强大的Matlab系统
  1. 强大的 **ndarray** 多维数组结构
  2. 成熟的函数库
  3. 用于整合C/C++和Fortran代码的工具包
  4. 实用的线性代数、傅里叶变换和随机数模块
  5. Numpy 和稀疏矩阵运算包scipy 配合使用非常方便



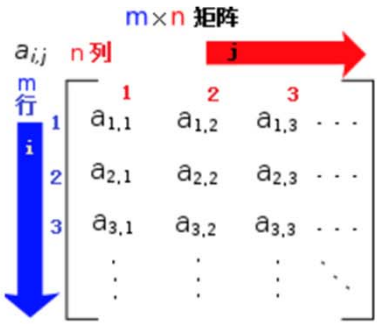
基本数据结构 ndarray

- 矩阵表示：使用Numpy，易得到二维矩阵

```
import numpy as np

np_ar = np.array([[1, 2, 3], [4, 5, 6]])  #生成简单的二维矩阵
print type(np_ar)
print np_ar
```

```
'numpy.ndarray'
[[1 2 3]
 [4 5 6]]
```



基本数据结构 ndarray

- 作为ndarray对象里的数据有时并不是所需要的，那么可以使用ndarray对象的astype() 方法转为指定的数据类型

```
a = np.array(["1", "2", "3"])  #原本其中的数据类型为字符串
print a

a_ = a.astype("float_") #转换为浮点型
print a_
```

```
['1' '2' '3']
[ 1.  2.  3.]
```

ndarray相关操作：索引

- 将数据转为ndarray对象后，会需要按某种方式来抽取数据
- ndarray对象提供了两种索引方式：
  - 切片索引：切片索引和对列表list的切片索引相似，不过由原本的一维切片变为多维

```
import numpy as np

np_ar = np.array([[1, 2, 3], [4, 5, 6]])  #生成简单的二维矩阵
print type(np_ar)
print np_ar
```

```
<type 'numpy.ndarray'>
[[1 2 3]
 [4 5 6]]
```

```
print np_ar[:, :2]
```

```
[[1 2]
 [4 5]]
```

切片索引

- 索引后还可以直接对应该位置重新赋值

```
np_ar[:, :2] = 0
np_ar
```

```
array([[0, 0, 3],
       [0, 0, 6]])
```

- 可通过shape属性得到数组的行数和列数

```
print np_ar[:, :2].shape
```

```
(2L, 2L)
```

切片索引

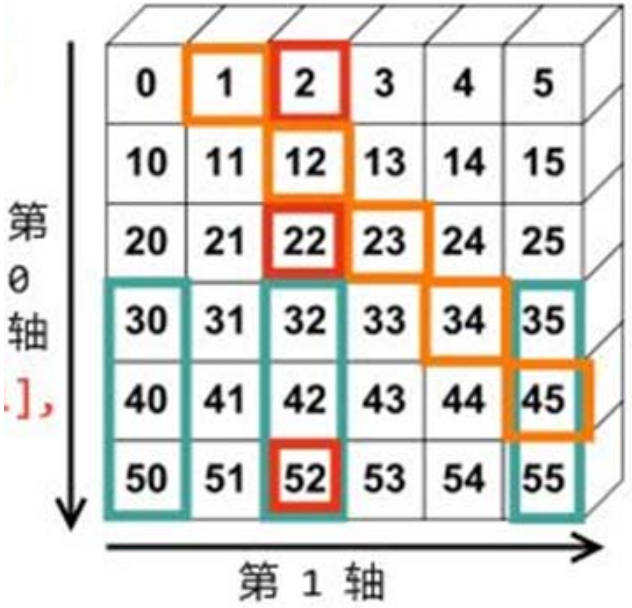
```
>>> a
array([[ 0,  1,  2,  3,  4,  5],
       [10, 11, 12, 13, 14, 15],
       [20, 21, 22, 23, 24, 25],
       [30, 31, 32, 33, 34, 35],
       [40, 41, 42, 43, 44, 45],
       [50, 51, 52, 53, 54, 55]])
```




```
>>>
a=np.array([[0,1,2,3,4,5],[10,11,12,13,14,15],[2
0,21,22,23,24,25],[30,31,32,33,34,35],[40,41,42
,43,44,45],[50,51,52,53,54,55]])
>>> a[0,3:5]
array([3, 4])
>>> a[4:,4:]
array([[44, 45],
       [54, 55]])
>>> a[:,2]
array([ 2, 12, 22, 32, 42, 52])
>>> a[2::2,::2]
array([[20, 22, 24],
       [40, 42, 44]])
```

切片索引

```
>>> a[(0,1,2,3,4),(1,2,3,4,5)]
array([ 1, 12, 23, 34, 45])
>>> a[(2,3,4),(2)]
array([22, 32, 42])
>>> a[(0,1,2,3,4),(1,2,3,4,5)]
array([ 1, 12, 23, 34, 45])
>>> a[3:,[0,2,5]]
array([[30, 32, 35],
       [40, 42, 45],
       [50, 52, 55]])
>>> mask=np.array([1,0,1,0,0,1],dtype=np.bool)
>>> a[mask,2]
array([ 2, 22, 52])
>>>
```



布尔值索引

- 通过添加条件判断数组中每个值的 **真/假** 转为布尔值再对原数组进行索引，为真 True 时会被抽取出来

```
np_ar >= 2    # 进行判断，返回数据类型是布尔值的数组
```

```
array([[False, False,  True],
       [False, False,  True]], dtype=bool)
```

```
np_ar[np_ar>=2]    # 直接将布尔值的数组对原数组进行索引，True的部分都被保留
```

```
array([3, 6])
```

ndarray相关操作：切分

- 读取上证指数数据集 sh.csv,并将数据转为ndarray对象

```
import numpy as np
import csv
import datetime

with open('./sh.csv', 'r') as f:
    reader = csv.reader(f)
    sh_content = [con for con in reader]

# 日期数据的处理#
time_format = "%Y-%m-%d"
for con in range(len(sh_content) - 1):
    sh_content[con+1][0] = datetime.datetime.strptime(sh_content[con+1][0], time_format)

# 将 list 转换为 array 格式
sh_ndarray = np.array(sh_content)
print (sh_ndarray)
```

```
[['date' 'open' 'close' ..., 'low' 'volume' 'code']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' ..., '3253.88'
  '531352391.0' 'sh']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' ..., '3303.18'
  '501661695.0' 'sh']
 ...,
 [datetime.datetime(2015, 12, 29, 0, 0) '3528.4' '3563.74' ..., '3515.52'
  '182551923.0' 'sh']
 [datetime.datetime(2015, 12, 30, 0, 0) '3566.73' '3572.88' ..., '3538.11'
  '187889609.0' 'sh']
 [datetime.datetime(2015, 12, 31, 0, 0) '3570.47' '3539.18' ..., '3538.35'
  '176963659.0' 'sh']]
```

ndarray相关操作：切分

- 取sh_ndarray前四行前四列进行ndarray操作演示

```
sh_ndarray = sh_ndarray[0:4,0:4]
print (sh_ndarray)
```

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 使用split将ndarray按照行平均分为几个ndarray

```
# 每两行作为一个新的array
np.split(sh_ndarray,2)
```

```
[array([['date', 'open', 'close', 'high'],
        [datetime.datetime(2015, 1, 5, 0, 0), '3258.63', '3350.52',
         '3369.28']], dtype=object),
 array([[datetime.datetime(2015, 1, 6, 0, 0), '3330.8', '3351.45', '3394.22'],
        [datetime.datetime(2015, 1, 7, 0, 0), '3326.65', '3373.95', '3374.9']], dtype=object)]
```

ndarray相关操作：切分

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 也可以按照下标位置进行划分

```
# 将第一行、最后一行以及剩下的行分为三个新的array
np.split(sh_ndarray,[1,-1])
```

```
[array([['date', 'open', 'close', 'high']], dtype=object),
 array([[datetime.datetime(2015, 1, 5, 0, 0), '3258.63', '3350.52',
         '3369.28'],
        [datetime.datetime(2015, 1, 6, 0, 0), '3330.8', '3351.45', '339
4.22']], dtype=object),
 array([[datetime.datetime(2015, 1, 7, 0, 0), '3326.65', '3373.95', '337
4.9']], dtype=object)]
```

ndarray相关操作：重构

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 通过 reshape 方法将所有元素按照指定行指定列进行重构

```
# 将16个元素按照2行8列方式重构array
sh_ndarray.reshape((2,8))
```

```
array([['date', 'open', 'close', 'high',
        datetime.datetime(2015, 1, 5, 0, 0), '3258.63', '3350.52',
        '3369.28'],
       [datetime.datetime(2015, 1, 6, 0, 0), '3330.8', '3351.45',
        '3394.22', datetime.datetime(2015, 1, 7, 0, 0), '3326.65',
        '3373.95', '3374.9']], dtype=object)
```

ndarray相关操作：拼接

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 通过 vstack 沿纵轴拼接
- 通过 hstack 沿横轴拼接
- 通过 concatenate 进行拼接

```
# 使用sh_ndarray第一行进行横轴拼接操作
h = np.hstack((sh_ndarray[0],sh_ndarray[0]))
# 使用sh_ndarray第一行进行纵轴拼接操作
v = np.vstack((sh_ndarray[0],sh_ndarray[0]))
#
c = np.concatenate((sh_ndarray[0],sh_ndarray[0]))
print (h)
print (h.shape)
print (v)
print (v.shape)
print (c)
```

```
['date' 'open' 'close' 'high' 'date' 'open' 'close' 'high']
(8,)
[['date' 'open' 'close' 'high']
 ['date' 'open' 'close' 'high']]
(2, 4)
['date' 'open' 'close' 'high' 'date' 'open' 'close' 'high']
```

ndarray相关操作：转置、翻转

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 转置： transpose 方法、在数组后加 .T

```
print sh_ndarray.transpose()
print sh_ndarray.T
```

```
[['date' datetime.datetime(2015, 1, 5, 0, 0)
  datetime.datetime(2015, 1, 6, 0, 0) datetime.datetime(2015, 1, 7, 0,
 0)]
 ['open' '3258.63' '3330.8' '3326.65']
 ['close' '3350.52' '3351.45' '3373.95']
 ['high' '3369.28' '3394.22' '3374.9']]
[['date' datetime.datetime(2015, 1, 5, 0, 0)
  datetime.datetime(2015, 1, 6, 0, 0) datetime.datetime(2015, 1, 7, 0,
 0)]
 ['open' '3258.63' '3330.8' '3326.65']
 ['close' '3350.52' '3351.45' '3373.95']
 ['high' '3369.28' '3394.22' '3374.9']]
```

ndarray相关操作：转置、翻转

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
```

- 翻转： fliplr 左右翻转、flipud 上下翻转

```
print sh_ndarray
# 左右翻转
print np.fliplr(sh_ndarray)
# 上线翻转
print np.flipud(sh_ndarray)
```

```
[['date' 'open' 'close' 'high']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']]
[['high' 'close' 'open' 'date']
 ['3369.28' '3350.52' '3258.63' datetime.datetime(2015, 1, 5, 0, 0)]
 ['3394.22' '3351.45' '3330.8' datetime.datetime(2015, 1, 6, 0, 0)]
 ['3374.9' '3373.95' '3326.65' datetime.datetime(2015, 1, 7, 0, 0)]]
[[datetime.datetime(2015, 1, 7, 0, 0) '3326.65' '3373.95' '3374.9']
 [datetime.datetime(2015, 1, 6, 0, 0) '3330.8' '3351.45' '3394.22']
 [datetime.datetime(2015, 1, 5, 0, 0) '3258.63' '3350.52' '3369.28']
 ['date' 'open' 'close' 'high']]
```

对位运算

- 指 ndarray 进行加减乘除运算时，使对应位置的数值进行加减乘除运算

```
a = np.array([[4,5,6],[7,8,9]])
b = np.array([[2,3,4],[5,6,7]])
print 'a + b','\n',a + b
print 'a - b','\n',a - b
print 'a * b','\n',a * b
print 'a / b','\n',a / b
```

```
a + b
[[ 6  8 10]
 [12 14 16]]
a - b
[[2 2 2]
 [2 2 2]]
a * b
[[ 8 15 24]
 [35 48 63]]
a / b
[[2 1 1]
 [1 1 1]]
```

对位运算

- 当两个 ndarray 的维度不一致时，则没有对齐的维度上会分别执行对位运算，这种机制叫做广播（broadcasting）

```
#a与c维度不一致，c将与a的每一行分别进行运算
a = np.array([[4,5,6],[7,8,9]])
c = np.array([2,3,4])
print 'a + c','\n',a + c
print 'a - c','\n',a - c
print 'a * c','\n',a * c
print 'a / c','\n',a / c
```

```
a + c
[[ 6  8 10]
 [ 9 11 13]]
a - c
[[2 2 2]
 [5 5 5]]
a * c
[[ 8 15 24]
 [14 24 36]]
a / c
[[2 1 1]
 [3 2 2]]
```

结构化的数据

- 给你的变量命名

表 1 上证指数数据

	date	open	close	high	low	volume	code
0	2015-01-06 00:00:00	3330.8	3351.45	3394.22	3303.18	501661695.0	sh
1	2015-01-07 00:00:00	3326.65	3373.95	3374.9	3312.21	391918880.0	sh
2	2015-01-08 00:00:00	3371.96	3293.46	3381.57	3285.09	371131170.0	sh
3	2015-01-09 00:00:00	3276.97	3285.41	3404.83	3267.51	410240872.0	sh
4	2015-01-12 00:00:00	3258.21	3229.32	3275.19	3191.58	322064679.0	sh

结构化的数据

- 上证指数数据中，数据sh_content的第一个元素是每个变量的命名，对其命名
- 重新设置参数dtype

```
import numpy as np

dtype = [('date', object),('open', float),('close', float),('high', float),('low', float),('volume', float),('code', str)]  #设置数据类型
sh_content_tuple = [tuple(row) for row in sh_content[1:]]  #将需要转为numpy.adarray的数据转为元组tuple类型
sh_np = np.array(sh_content_tuple, dtype=(dtype))
sh_np.dtype
```

```
dtype([('date', 'O'), ('open', 'f8'), ('close', 'f8'), ('high', 'f8'), ('low', 'f8'), ('volume', 'f8'), ('code', 'S')])
```

结构化的数据

- 按命名索引

```
sh_np["date"][:5]  #`sh_np["date"]`指抽取命名为`date`的那一列的前五个元素
```

```
array([datetime.datetime(2015, 1, 5, 0, 0),
       datetime.datetime(2015, 1, 6, 0, 0),
       datetime.datetime(2015, 1, 7, 0, 0),
       datetime.datetime(2015, 1, 8, 0, 0),
       datetime.datetime(2015, 1, 9, 0, 0)], dtype=object)
```

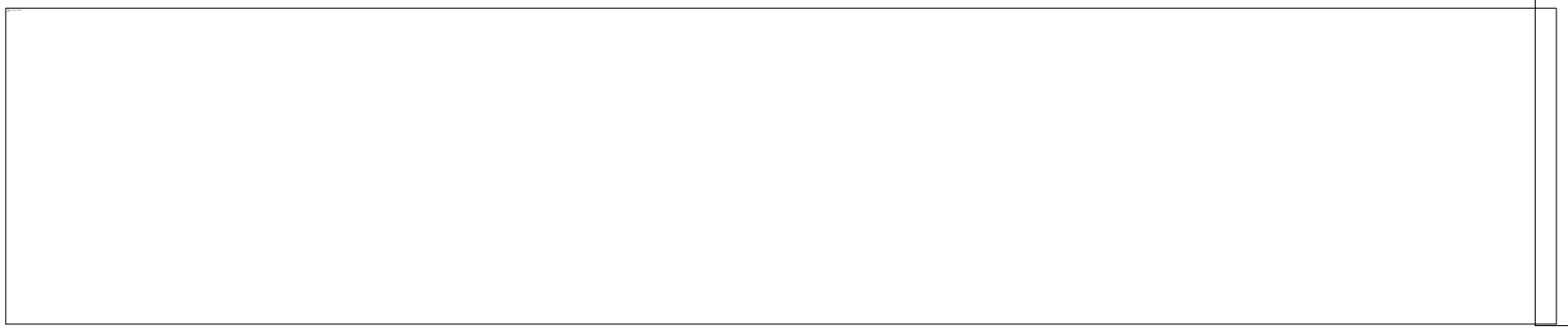
```
sh_np["volume"][:5]
```

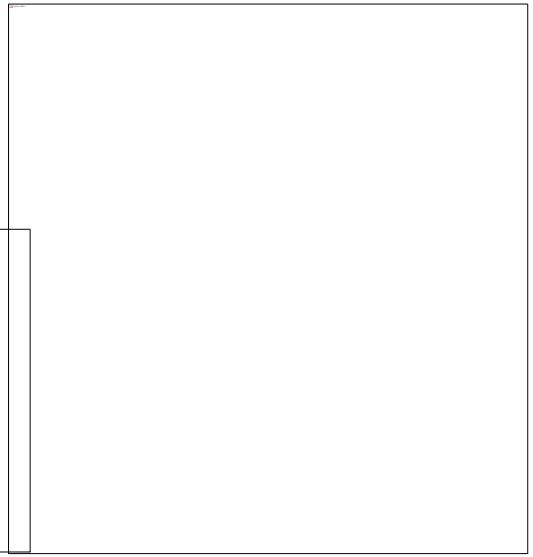
```
array([  5.31352391e+08,   5.01661695e+08,   3.91918880e+08,
         3.71131170e+08,   4.10240872e+08])
```

内置操作函数

- 数学函数
- 运算函数
- 统计函数

- numpy.log()函数是numpy模块提供的现成自然对数函数





内置操作函数

- 数学函数
- 运算函数
- 统计函数

- 在计算的时候，会用到差分、累加的情况。Numpy提供很多相关的运算函数
- 以计算差分的numpy.diff()方法为例：

```
a = np.array([1, 3, 4, 5, 8])

print np.diff(a)
```

```
[2 1 1 3]
```

`numpy.diff()` 方法的语法为：

```
numpy.diff(a, n=1, axis=-1)
```

内置操作函数

- 数学函数
- 运算函数
- 统计函数

- 对于二维矩阵，需要指明是对哪一个维度进行差分

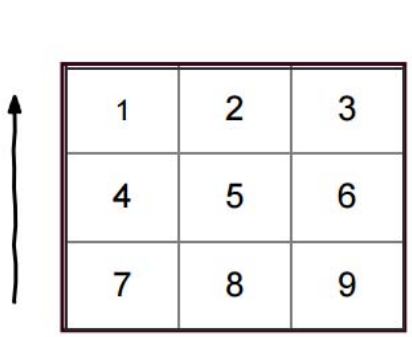
```
a = np.array([[1,2,3], [4,5,6], [7,8,9]])

print np.diff(a, axis=0)
```

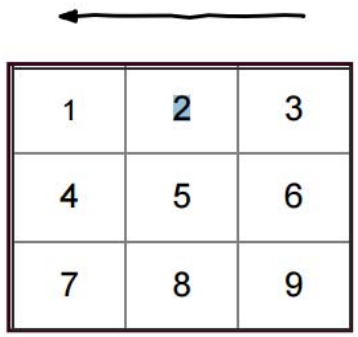
```
[[3 3 3]
 [3 3 3]]
```

```
print np.diff(a, axis=1)
```

```
[[1 1]
 [1 1]
 [1 1]]
```



axis=0



axis=1

内置操作函数

- 数学函数
- 运算函数
- 统计函数

- numpy提供了很多计算最大值、最小值、均值、中位数等统计量的函数，比如统计最大值的函数numpy.amax()

```
numpy.amax(a, axis=None, out=None, keepdims=False)
```

```
print np.amax(sh_part_ndarray)
```

```
5178.19
```

```
print np.amax(sh_part_ndarray, axis=0)
```

```
[ 5174.42  5166.35  5178.19  5103.4 ]
```

内置操作函数

- 数学函数
- 运算函数
- 统计函数

- amin(a[, axis, out, keepdims]) 返回最小值
- amax(a[, axis, out, keepdims]) 返回最大值
- percentile(a, q[, axis, out, ...]) 返回分位数
- median(a[, axis, out, overwrite_input, keepdims]) 返回中位数
- average(a[, axis, weights, returned]) 返回平均值，可以设定权重，计算权重平均
- mean(a[, axis, dtype, out, keepdims]) 返回平均值
- std(a[, axis, dtype, out, ddof, keepdims]) 计算标准差
- var(a[, axis, dtype, out, ddof, keepdims]) 计算方差

线性代数模块linalg

- Numpy 广泛应用于数值计算过程，其线性代数模块 linalg 常用于向量及矩阵的基本运算以及特征值分解、SVD分解等矩阵分解过程

```
import numpy as np
a = np.array([[1,2,3],[4,5,6],[7,8,9]]) # 矩阵
b = np.array([0,1,2])  # 向量


print np.linalg.norm(a) # norm 计算矩阵的范数
print np.dot(a,b) # 向量与矩阵的乘法
print np.trace(a) # 求矩阵的迹
print np.linalg.det(a) # 求矩阵的行列式值
print np.linalg.matrix_rank(a) # 求矩阵的秩
```

```
16.8819430161
[ 8 17 26]
15
-9.51619735393e-16
2
```

线性代数模块linalg

- 特征值分解、Cholesky分解、SVD分解

```
c = np.array([[2,1],[1,2]])
u,v = np.linalg.eig(c) # 特征值分解
l = np.linalg.cholesky(c) # Cholesky分解
U, s, V = np.linalg.svd(c) # SVD分解,计算出的s需要重建为对角矩阵
S = np.array([[s[0],0],[0,s[1]]])
print S
```

```
[[ 3.  0.]
 [ 0.  1.]]
```

线性代数模块linalg

如要求得 $Ax = b$的通解，则可以使用SVD分解方法,例如方程组为

$$\begin{aligned} 2x_1 - 2x_2 - 4x_3 &= 0 \\ -x_1 + 3x_2 + 4x_3 &= 0 \\ x_1 - 2x_2 - 3x_3 &= 0 \end{aligned}$$

```
a = np.array([[2,-2,-4],[-1,3,4],[1,-2,-3]])
b = np.array([0,0,0])
U, s, V = np.linalg.svd(a) # SVD分解
np.compress(s < 1e-10, V, axis=0) # 按照 s 对 v 进行切片
```

```
array([[-0.57735027,  0.57735027, -0.57735027]])
```

随机模块random

- 伪随机数的产生，可从离散分布和连续分布中产生
- 在蒙特卡洛方法、随机积分、随机过程模拟等很多方面都有应用
- 指定随机种子（seed）产生相同的随机数序列

```
from numpy import random

random.seed(666)
print random.rand(2,3) # 产生一个2行3列的矩阵，其中的每一个元素为[0,1)之间的浮点型随机数
print random.randint(0,10) # 产生一个[0,10)之间的整型随机数
```

```
[[ 0.70043712  0.84418664  0.67651434]
 [ 0.72785806  0.95145796  0.0127032 ]]
3
```

全局种子与局部种子

- random.seed(666) 指定全局种子，也可以使用 random.RandomState(666) 的方式指定局部种子，在指定局部种子的情况下，使用局部种子产生的随机数不改变

```
local_seed = random.RandomState(666)# 指定局部种子
print local_seed.rand(2,3) # 通过局部种子产生随机数，多次运行不改变随机数
print random.rand(2,3) # 未指定全局种子，每一次运行将会改变随机数
```

```
[[ 0.70043712  0.84418664  0.67651434]
 [ 0.72785806  0.95145796  0.0127032 ]]
[[ 0.65091808  0.51832728  0.00355049]
 [ 0.80309176  0.35932938  0.42014544]]
```

常见分布的产生方式

```
print random.binomial(n=5,p=0.5,size=5) # 二项分布：产生5个服从二项分布B(5,0.5)的样本
print random.uniform(-1,1,5) # 均匀分布：产生5个服从均匀分布U[-1,1]的样本
print random.normal(size=(2,3)) # 标准正态分布：产生2x5的标准正态分布样本
print random.normal(loc=0,scale=5,size=(2,3)) # 正态分布：产生2x5的均值为0，标准差为5的正态分布样本
```

```
[2 2 3 4 3]
[ 0.98306524 -0.93647313  0.08605806  0.01417688 -0.79867321]
[[-0.57957596 -0.53929631 -0.13996843]
 [-0.605991   -1.23679475 -0.9914476 ]]
[[ -5.06268003  -8.9900398    1.03876241]
 [  0.1611856    6.32885767 -12.79336444]]
```

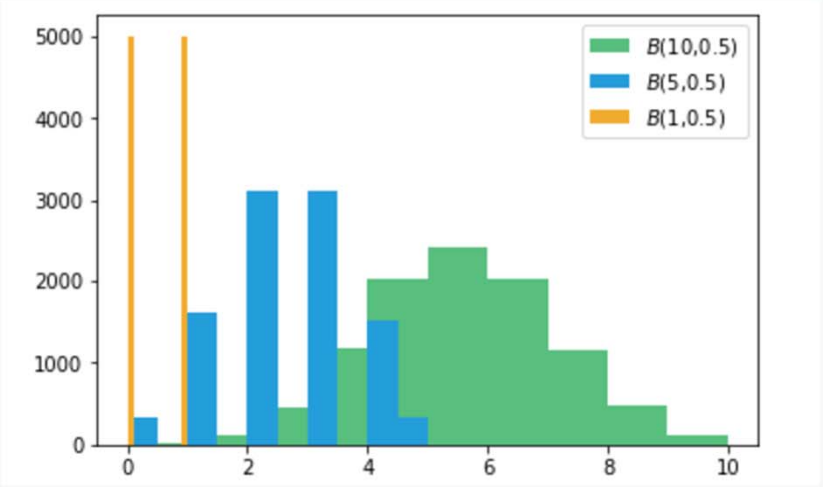
产生二项分布图练习

- 要求产生 B(10,0.5) B(5,0.5) B(1,0.5) 二项分布图（分别从二项分布B(10,0.5)、B(5,0.5)、B(1,0.5)中产生10000个样本存入数组）

```
import numpy as np
from numpy import random
import matplotlib.pyplot as plt
%matplotlib inline
random.seed(666)

data_10 = random.binomial(n=10,p=0.5,size=10000)
data_5 = random.binomial(n=5,p=0.5,size=10000)
data_1 = random.binomial(n=1,p=0.5,size=10000)

# 以下为画图部分
data = [data_10,data_5,data_1]
n_list = [10,5,1]
color = ['#5dbe80','#2d9ed8','#efab40']
fig, ax = plt.subplots()
for row,c,n in zip(data,color,n_list):
    ax.hist(row,
            color = c,
            label = '$B($'+str(n)+',0.5'+'$)$')
ax.legend(loc = 'upper right',fontsize = 10)
plt.axis('tight')
plt.grid('off')
```



模拟实验

三扇门，其中一扇门后有奖品，这扇门只有主持人知道。选手先随机选一扇门，但并不打开，主持人看到后，会打开其余两扇门中没有奖品的一扇门。然后，主持人问选手，是否要改变一开始的选择？

你会改变选择吗？

改变 2/3选对

不改变 1/3选对



模拟实验

```python
from numpy import random

random.seed(66)

# 做10000次实验
n_tests = 10000

# 生成每次实验的奖品所在的门的编号
# 0表示第一扇门，1表示第二扇门，2表示第三扇门
winning_doors = random.randint(0, 3, n_tests)

# 记录如果换门的中奖次数
change_mind_wins = 0

# 记录如果坚持的中奖次数
insist_wins = 0

# winning_door就是获胜门的编号
for winning_door in winning_doors:

    # 随机挑了一扇门
    first_try = random.randint(0, 3)

    # 其他门的编号
    remaining_choices = [i for i in range(3) if i != first_try]

    # 没有奖品的门的编号，这个信息只有主持人知道
    wrong_choices = [i for i in range(3) if i != winning_door]
```

模拟实验

```
    # 一开始选择的门主持人没法打开，所以从主持人可以打开的门中剔除
    if first_try in wrong_choices:
        wrong_choices.remove(first_try)

    # 这时wrong_choices变量就是主持人可以打开的门的编号
    # 注意此时如果一开始选择正确，则可以打开的门是两扇，主持人随便开一扇门
    # 如果一开始选到了空门，则主持人只能打开剩下一扇空门
    screened_out = random.choice(wrong_choices)
    remaining_choices.remove(screened_out)

    # 所以虽然代码写了好些行，如果策略固定的话，
    # 改变主意的获胜概率就是一开始选错的概率，是2/3
    # 而坚持选择的获胜概率就是一开始就选对的概率，是1/3

    # 现在除了一开始选择的编号，和主持人帮助剔除的错误编号，只剩下一扇门
    # 如果要改变注意则这扇门就是最终的选择
    changed_mind_try = remaining_choices[0]

    # 结果揭晓，记录下来
    change_mind_wins += 1 if changed_mind_try == winning_door else 0
    insist_wins += 1 if first_try == winning_door else 0

# 输出10000次测试的最终结果，和推导的结果差不多：
# You win 6616 out of 10000 tests if you changed your mind
# You win 3384 out of 10000 tests if you insist on the initial choice
print(
    'You win {1} out of {0} tests if you changed your mind\n'
    'You win {2} out of {0} tests if you insist on the initial choice'.form
        n_tests, change_mind_wins, insist_wins
        )
)
```

```
You win 6614 out of 10000 tests if you changed your mind
You win 3386 out of 10000 tests if you insist on the initial choice
```

总结

- Numpy基本数据结构 ndarray
- ndarray相关操作：索引、切分、重构、拼接、转置与翻转
- 对位运算原则，结构化的数据
- 内置操作函数：数学函数、运算函数、统计函数
- 线性代数模块linalg，随机模块random



第十五部分 Pandas数据分析

Python科学计算生态圈

- Python在科学计算方面有很多不断改良的库，结合其在通用编程方面的强大实力，使其在数据处理、交互探索性计算以及数据可视化方面深受广大编程者的喜爱

- Python有着一个强大的科学计算生态圈，已经完全可以媲美MATLAB、R等特定编程语言/工具



NumPy
Base N-dimensional array package



SciPy library
Fundamental library for scientific computing



Matplotlib
Comprehensive 2D Plotting

IP[y]: IPython

IPython
Enhanced Interactive Console



Sympy
Symbolic mathematics



pandas
Data structures & analysis

Python科学计算生态圈--Numpy

- NumPy是Python科学计算基础包，提供了以下功能（仅列举了几条）
  - ✓ 快速高效的多维数组对象ndarray
  - ✓ 用于对数组执行元素级计算以及直接对数组执行数学运算的函数
  - ✓ 用于读写硬盘上基于数组的数据集的工具
  - ✓ 线性代数运算、傅里叶变换以及随机数生成
  - ✓ 用于集成由C、C++、Fortran等语言编写的代码的工具
- 对于数值型数据，NumPy数组在存储和处理数据时要比内置的Python数据结构高效得多

Python科学计算生态圈--Scipy

- SciPy是一组专门解决科学计算中各种标准问题域的包的集合，跟NumPy的有机结合完全可以替代MATLAB的计算功能
  - ✓ scipy.integrate：数值积分和微分方程求解器
  - ✓ scipy.linalg：扩展了由numpy.linalg提供的线性代数和矩阵分解功能
  - ✓ scipy.optimize：函数优化器以及根查找算法
  - ✓ scipy.special：SPECFUN（实现了许多常用数学函数如伽玛函数的Fortran库）的包装器
  - ✓ scipy.stats：标准概率分布、各种统计检验方法以及更好的描述统计法
  - ✓ scipy.weave：利用内联C++代码加速数组计算的工具

Python科学计算生态圈--Pandas

- pandas提供了使我们能够快速便捷地处理结构化数据的大量数据结构和函数
- pandas兼具NumPy高性能的数组计算功能以及电子表格和关系型数据库灵活的数据处理功能
- 对于金融行业的用户，pandas提供了大量适合于金融数据的高性能时间序列功能和工具
- 学统计的人会对R语言比较熟悉，R提供的data.frame对象功能仅仅是pandas的DataFrame所提供的功能的一个子集

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

数据文件操作——读入数据

- pandas提供了一些用于将表格型数据读取为DataFrame对象的函数，常用的函数为read_csv和read_table
- 函数的选项可以划分为几个大类
  - 索引：将一个或多个列当做返回的DataFrame处理，以及是否从文件、用户获取列名
  - 类型推断和数据转换：包括用户定义值的转换、缺失值标记列表等
  - 日期解析：包括组合功能，比如将分散在多个列中的日期时间信息组合起来
  - 迭代：支持对大文件进行逐块迭代
  - 不规整数据问题：跳过一些行、页脚、注释或其他一些不重要的东西

文件读写

Pandas提供了一些用于将表格型数据读取位DataFrame对象的函数。其中最常用的为read_csv和read_table。read_csv 从文件、URL、文件型对象中加载带分隔符的数据。默认分隔符为逗号。read_table从文件、URL、文件型对象中加载带分隔符的数据。默认分隔符为制表符（“\t”）

```
In[19]:df = pd.read_csv('iris.csv')
df.head()
Out[19]:
   sepal_len  sepal_wh  petal_len  petal_wh  target
0        5.1       3.5        1.4       0.2       0
1        4.9       3.0        1.4       0.2       0
2        4.7       3.2        1.3       0.2       0
3        4.6       3.1        1.5       0.2       0
4        5.0       3.6        1.4       0.2       0
```

文件读写

使用read_table从文件、URL、文件型对象中加载带分隔符的数据。
默认分隔符为制表符（“\t”）

```
In[20]:df = pd.read_table('./iris.csv',sep=',')
df.head()
Out[20]:
   sepal_len  sepal_wh  petal_len  petal_wh  target
0        5.1       3.5        1.4       0.2       0
1        4.9       3.0        1.4       0.2       0
2        4.7       3.2        1.3       0.2       0
3        4.6       3.1        1.5       0.2       0
4        5.0       3.6        1.4       0.2       0
```

文件读写

read_csv/read_table常用参数介绍：

- path ： 表示文件系统位置、URL、文件型对象的字符串
- sep/delimiter：用于对行中个字段进行拆分的字符序列或正则表达式
- header：用做列名的行号。默认为0（第一行），若无header行，设为None
- names：用于结果的列名列表，结合header=None
- skiprows: 需要忽略的行数
- na_values：一组用于替换NA的值
- nrows：需要读取的行数（从文件开始处算起）
- verbose：打印各种解析器信息，比如“非数值列中缺失值的数量”
- encoding：用于unicode的文本格式编码。例如，“utf-8”表示用UTF-8 编码的文本

文件读写

在处理不规则文件时，skiprows可以帮助跳过文件中的若干行。

```
df = pd.read_csv('iris.csv',skiprows=[0,2,3])
df.head()
Out[57]:
   5.1  3.5  1.4  0.2  0
0  4.6  3.1  1.5  0.2  0
1  5.0  3.6  1.4  0.2  0
2  5.4  3.9  1.7  0.4  0
3  4.6  3.4  1.4  0.3  0
4  5.0  3.4  1.5  0.2  0
```

文件读写

可以在读取文件时处理数据中的缺失值，要用到参数na_values。

```
tips = pd.read_csv('tips.csv')
tips.head()
Out[70]:
   Unnamed: 0  total_bill   tip     sex smoker  day    time  size
0           0       16.99  1.01  Female     No  Sun  Dinner     2
1           1       10.34  1.66    Male     No  Sun  Dinner     3
2           2       21.01  3.50    Male     No  Sun  Dinner     3
3           3       23.68  3.31    Male     No  Sun  Dinner     2
4           4       24.59  3.61  Female     No  Sun  Dinner     4
```

文件读写

na_values可以接受一组表示缺失值的字符串，用字典将列的特定值指定为NaN。

```
map_id = {'sex':['Female'],'size':[3]}
tips = pd.read_csv('tips.csv',na_values=map_id)
tips.head()
Out[71]:
   Unnamed: 0  total_bill   tip   sex smoker  day    time  size
0           0       16.99  1.01   NaN     No  Sun  Dinner     2
1           1       10.34  1.66  Male     No  Sun  Dinner   NaN
2           2       21.01  3.50  Male     No  Sun  Dinner   NaN
3           3       23.68  3.31  Male     No  Sun  Dinner     2
4           4       24.59  3.61   NaN     No  Sun  Dinner     4
```

文件读写

在处理较大文件时，有两种方式：通过nrows指定读取的行；
通过设置chunksize，逐块读取。

```
In[66]:chunker = pd.read_csv('iris.csv', chunksize=50)
#输出每一块的长度
for piece in chunker:
    print len(piece)
Out[66]:
50
50
50
```

```
In[79]:pd.read_csv('iris.csv',nrows=5)
Out[79]:
   sepal_len  sepal_wh  petal_len  petal_wh  target
0        5.1       3.5        1.4       0.2       0
1        4.9       3.0        1.4       0.2       0
2        4.7       3.2        1.3       0.2       0
3        4.6       3.1        1.5       0.2       0
4        5.0       3.6        1.4       0.2       0
```

数据文件操作——写出数据

使用to_csv写出数据到文件，默认会写出行和列的标签，

可以设置 index和header选项进行选择。

```
In[10]:df = pd.read_csv('iris.csv')
df.to_csv('out.csv')
!type out.csv

Out[10]:,a,b,c,d
Sepal_len,sepal_wh,petal_len,petal wh,target
5.1,3.5,1.4,0.2,0
4.9,3.0,1.4,0.2,0
4.7,3.2,1.3,0.2,0
4.6,3.1,1.5,0.2,0
```

```
In[11]:df.to_csv('out.csv',index=False,header=False)
!type out.csv
5.1,3.5,1.4,0.2,0
4.9,3.0,1.4,0.2,0
4.7,3.2,1.3,0.2,0
4.6,3.1,1.5,0.2,0
```

文件读写小结

导入数据

- pd.read_csv(filename)：从CSV文件导入数据
- pd.read_table(filename)：从限定分隔符的文本文件导入数据
- pd.read_excel(filename)：从Excel文件导入数据
- pd.read_sql(query, connection_object)：从SQL表/库导入数据
- pd.read_json(json_string)：从JSON格式的字符串导入数据
- pd.read_html(url)：解析URL、字符串或者HTML文件，抽取其中的tables表格
- pd.read_clipboard()：从你的粘贴板获取内容，并传给read_table()
- pd.DataFrame(dict)：从字典对象导入数据，Key是列名，Value是数据

导出数据

- df.to_csv(filename)：导出数据到CSV文件
- df.to_excel(filename)：导出数据到Excel文件
- df.to_sql(table_name, connection_object)：导出数据到SQL表
- df.to_json(filename)：以Json格式导出数据到文本文件

文件读写小结

创建测试对象

- pd.DataFrame(np.random.rand(20,5))：创建20行5列的随机数组成的DataFrame对象
- pd.Series(my_list)：从可迭代对象my_list创建一个Series对象
- df.index = pd.date_range('1900/1/30', periods=df.shape[0])：增加一个日期索引

查看、检查数据

- df.head(n)：查看DataFrame对象的前n行
- df.tail(n)：查看DataFrame对象的最后n行
- df.shape()：查看行数和列数
- http://df.info()：查看索引、数据类型和内存信息
- df.describe()：查看数值型列的汇总统计
- s.value_counts(dropna=False)：查看Series对象的唯一值和计数
- df.apply(pd.Series.value_counts)：查看DataFrame对象中每一列的唯一值和计数

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

变量离散化

为了便于分析，连续数据常常被离散化或拆分为“bin”。要实现该功能，需要使用pandas的cut函数。pandas返回的是一个特殊的Categorical对象，可以将其看做一组表示“bin”名称的字符串。实际上，它含有一个表示不同分类名称的levels数组以及一个为要划分特征数据进行标号的labels属性：

```
sl = df[u'sepal_len']
bins = [4,5,6,7,8]
cats = pd.cut(sl, bins)
cats[:5]
Out[19]:
0    (5, 6]
1    (4, 5]
2    (4, 5]
3    (4, 5]
4    (4, 5]
Name: sepal length (cm), dtype: category
Categories (4, object): [(4, 5] < (5, 6] < (6, 7] < (7, 8]]
```

变量离散化

与“区间”的数学符号一样，圆括号表示开区间，二方括号表示闭区间，可以通过right参数修改区间。

```
pd.cut(sl,bins,right=False)
Out[53]:
0    [5, 6)
1    [4, 5)
2    [4, 5)
3    [4, 5)
4    [5, 6)
5    [5, 6)
```

变量离散化

qcut是一个非常类似于cut的函数，它可以根据样本分位数对数据进bin划分。相比cut，qcut使用样本分位数，可以得到大小基本相等的bin。

```
cats = pd.qcut(sl,4)
cats.head()
Out[55]:
0    [4.3, 5.1]
1    [4.3, 5.1]
2    [4.3, 5.1]
3    [4.3, 5.1]
4    [4.3, 5.1]
Name: sepal length (cm), dtype: category
Categories (4, object): [[4.3, 5.1] < (5.1, 5.8] < (5.8, 6.4] < (6.4, 7.9]]
```

```
pd.value_counts(cats)
Out[56]:
[4.3, 5.1]    41
(5.1, 5.8]    39
(6.4, 7.9]    35
(5.8, 6.4]    35
dtype: int64
```

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

处理缺失数据

pandas使用浮点值NaN（Not a Number）表示浮点和非浮点数组中的缺失数据，它只是一个便于被检测出来的标记而已；Python内置的None值也会被当做NA处理

```
In [2]: string_data=Series([None,'b',np.nan,'d'])
In [3]: string_data
Out[3]:
0    None
1       b
2     NaN
3       d
dtype: object
```

```
In [4]: string_data.isnull()
Out[4]:
0     True
1    False
2     True
3    False
dtype: bool
```

处理缺失值

对于DataFrame对象，dropna默认丢弃任何含有缺失值的行，how='all'只丢弃全为NA的那些行，axis=1指定丢弃列

```
teenager = pd.read_csv('teenager_sns.csv')
teenager = teenager.ix[:,[0,1,2,3,4]]
teenager.head()
Out[9]:
  gradyear gender     age  friends
basketball
0     2006      M  18.980       2         0
1     2006      F  18.801       0         0
2      NaN    NaN     NaN      NaN       NaN
3     2006      F  18.875       0         0
4     2006    NaN  18.995      NaN         0
```

```
teenager.dropna().head()
Out[10]:
  gradyear gender     age  friends  basketball
0     2006      M  18.980       2         0
1     2006      F  18.801       0         0
3     2006      F  18.875       0         0
6     2006      F  18.930      72         0
7     2006      M  18.322      17         0
```

处理缺失值

丢弃数据中全部为缺失值的行。

```
In[11]:teenager.dropna(how='all').head()
Out[11]:
   gradyear gender     age  friends  basketball
0      2006      M  18.980        2           0
1      2006      F  18.801        0           0
3      2006      F  18.875        0           0
4      2006    NaN  18.995      NaN           0
5      2006      F     NaN      142           0
```

处理缺失值

丢弃数据中全部为缺失值的列。

```
In[13]:teenager['newcol'] = np.NaN
teenager.head()
Out[13]:
  gradyear gender     age  friends  basketball
newcol
0     2006      M  18.980       2         0     NaN
1     2006      F  18.801       0         0     NaN
2      NaN    NaN     NaN     NaN       NaN
NaN
3     2006      F  18.875       0         0     NaN
4     2006    NaN  18.995     NaN         0     NaN
```

```
In[14]:teenager =
teenager.dropna(how='all',axis=1)
teenager.head()
Out[14]:
  gradyear gender     age  friends
basketball
0     2006      M  18.980       2         0
1     2006      F  18.801       0         0
2      NaN    NaN     NaN     NaN       NaN
3     2006      F  18.875       0         0
4     2006    NaN  18.995     NaN         0
```

处理缺失数据

使用fillna方法可以填充缺失数据，并且可以增加不同的选项

```
In[15]:teenager.fillna(0).head()
Out[15]:
   gradyear gender     age  friends  basketball
0      2006      M  18.980        2           0
1      2006      F  18.801        0           0
2         0      0   0.000        0           0
3      2006      F  18.875        0           0
4      2006      0  18.995        0           0
```

处理缺失数据

使用字典和method方式填充，method=‘ffill’或‘bfill’(前向或后向填充)，
limit可以调节最大填充数量

```
In[17]:teenager.fillna({'gender':'M','friend':0}).head()
Out[17]:
   gradyear gender     age  friends  basketball
0      2006      M  18.980        2           0
1      2006      F  18.801        0           0
2       NaN      M     NaN      NaN         NaN
3      2006      F  18.875        0           0
4      2006      M  18.995      NaN           0
```

```
In[18]:teenager.fillna(method='ffill').head()
Out[18]:
   gradyear gender     age  friends  basketball
0      2006      M  18.980        2           0
1      2006      F  18.801        0           0
2      2006      F  18.801        0           0
3      2006      F  18.875        0           0
4      2006      F  18.995        0           0
```

目录

第一部分 文件读写
第二部分 变量离散化
第三部分 缺失值填补
第四部分 数据标准化
第五部分 数据合并
第六部分 数据组合
第七部分 数字编码
第八部分 OneHot编码

数据标准化

- Pandas中并没有提供直接进行数据标准化的方式，可以使用scikit-learn中preprocessing的方法进行数据的标准化操作。

- 标准化有两种处理方式：Z-score标准化和min-max标准化。在一些机器学习算法中（如，支持向量机的RBF核，线性模型中的L1和L2正则化），算法要求输入的数据特征均值为0，并且方差在相同数量级。这一操作可以调用preprocessing的scale方法和StandardScaler类实现。

- 在线性模型的计算过程中，要求输入数据特征在同一个数量级上，为了实现这一操作，可以调用preprocessing的MinMaxScaler方法。

数据标准化

```
df = pd.read_csv('iris.csv')
x  = preprocessing.scale(df)
x[:5]
Out[93]:
array([[-0.90068117,  1.03205722, -1.3412724 , -1.31297673, -1.22474487],
       [-1.14301691, -0.1249576 , -1.3412724 , -1.31297673, -1.22474487],
       [-1.38535265,  0.33784833, -1.39813811, -1.31297673, -1.22474487],
       [-1.50652052,  0.10644536, -1.2844067 , -1.31297673, -1.22474487],
       [-1.02184904,  1.26346019, -1.3412724 , -1.31297673, -1.22474487]])

#标准化数据的均值
In[94]:x.mean(axis=0)
Out[94]: array([ -2.77555756e-16,  -5.14033260e-16,   1.15463195e-16,
         9.25185854e-16,   1.18423789e-17])
#标准化数据的方差
In[95]:x.std(axis=0)
Out[95]: array([ 1.,  1.,  1.,  1.,  1.])
```

数据标准化

preprocessing模块提供了StandardScaler工具类，包含名为Transformer的API，可以在训练集上计算均值和标准差，并且能够在之后将同样的转换应用于测试集。

```
scaler = preprocessing.StandardScaler().fit(df)
#应用于新数据
scaler.transform([[-1.,  1., 0.,4.,3.]])
Out[97]: array([[-8.29192127, -4.7530169 , -2.13739226,  3.68299613,
2.44948974]])
```

数据标准化

preprocessing中提供了MinMaxScalar方法对数据进行[0,1]标准化

```
min_max_scaler = preprocessing.MinMaxScaler()
scaler = min_max_scaler.fit_transform(df)
scaler
Out[98]:
array([[ 0.22222222,  0.625     ,  0.06779661,  0.04166667,  0.        ],
       [ 0.16666667,  0.41666667,  0.06779661,  0.04166667,  0.        ],
       [ 0.11111111,  0.5       ,  0.05084746,  0.04166667,  0.        ],
       [ 0.08333333,  0.45833333,  0.08474576,  0.04166667,  0.        ],
       [ 0.19444444,  0.66666667,  0.06779661,  0.04166667,  0.        ],
       [ 0.30555556,  0.79166667,  0.11864407,  0.125     ,  0.

X_test = np.array([[ -3., -1.,  4., 0., 7.]])
min_max_scaler.transform(X_test)
Out[99]: array([[-2.02777778, -1.25      ,  0.50847458, -0.04166667,  3.5       ]])
```

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

数据库风格的DataFrame合并

数据集的合并或连接运算通过一个或多个键将行连接起来，pandas中主要使用merge函数

```
In [55]: df1=DataFrame({'lkey':['b','b','a','c'],
    ...:                'data1':range(4)})
In [56]: df2=DataFrame({'rkey':['a','b','b','d'],
    ...:                'data2':range(4)})
```

数据库风格的DataFrame合并

默认情况下merge会将重叠列的列名当做键，可以显示地用left_on和right_on指定；连接的时候产生的是行的笛卡尔积，如下面合并结果所示有4个“b”行。

```
In [59]: pd.merge(df1,df2,left_on='lkey',right_on='rkey')
Out[59]:
   data1 lkey  data2 rkey
0     0    b     1    b
1     0    b     2    b
2     1    b     1    b
3     1    b     2    b
4     2    a     0    a
```

数据库风格的DataFrame合并

默认情况下，merge做的是“inner”连接，结果中的键是交集，其他方式还有“left”、“right”以及“outer”

```
In [60]: pd.merge(df1,df2,left_on='lkey',right_on='rkey',how='outer')
Out[60]:
   data1 lkey  data2 rkey
0     0    b     1    b
1     0    b     2    b
2     1    b     1    b
3     1    b     2    b
4     2    a     0    a
5     3    c   NaN  NaN
6   NaN  NaN     3    d
```

索引上的合并

有时候，DataFrame中的连接键位于其索引上，此时可以传入left_index=True或right_index=True（或两个都传）以说明索引应该被用作连接键

```
In [63]: left1=DataFrame({'key':['a','b','a','a','b','c'],'value':range(6)})
In [64]: right1=DataFrame({'group_val':[3.5,7]},index=['a','b'])
In [65]: pd.merge(left1,right1,left_on='key',right_index=True,how='outer')
Out[65]:
  key  value  group_val
0   a      0        3.5
2   a      2        3.5
3   a      3        3.5
1   b      1        7.0
4   b      4        7.0
5   c      5        NaN
```

轴向连接

使用concat函数可以将不同的Series或DataFrame进行轴向连接；

传入axis=1可以在列上进行轴向连接

```
In [2]: s1=Series([0,1],index=['a','b'])

In [3]: s2=Series([3,4],index=['c','d'])

In [4]: pd.concat([s1,s2])
Out[4]:
a    0
b    1
c    3
d    4
dtype: int64
```

```
In [5]: pd.concat([s1,s2],axis=1)
Out[5]:
     0   1
a   0 NaN
b   1 NaN
c NaN   3
d NaN   4
```

轴向连接

在列上进行轴向连接得到的是基于索引外连接的结果，使用join='inner'可得到基于索引的内连接结果。

```
In [6]: s3=pd.concat([s1*5,s2])
```

```
In [7]: pd.concat([s1,s3],axis=1)
Out[7]:
    0  1
a   0  0
b   1  5
c NaN  3
d NaN  4
```

```
In [8]: pd.concat([s1,s3],axis=1,join='inner')
Out[8]:
   0  1
a  0  0
b  1  5
```

轴向连接

对DataFrame进行轴向连接也是同样的效果，有时候行索引是没有意义的，可以传入ignore_index=True

```
In [11]: df1=DataFrame(np.arange(4).reshape((2,2)),columns=['a','b'])

In [12]: df2=DataFrame(np.arange(4).reshape((2,2)),columns=['a','c'])

In [13]: pd.concat([df1,df2],ignore_index=True)
Out[13]:
   a  b   c
0  0  1 NaN
1  2  3 NaN
2  0 NaN   1
3  2 NaN   3
```

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

数据组合

对数据集进行分组并对各组应用一个函数，这是数据分析过程中的一个重要环节。在数据集准备好之后，通常的任务就是计算分组统计或生成透视表。Pandas提供了一个灵活高效的groupby功能，它使你能以一种自然的方式对数据集进行切片、切块、摘要等操作。

数据组合

对数据分组并对各组应用一个函数，是数据分析工作的一个主要环节。

Pandas中对象可以在行（axis=0）或列（axis=1）上进行分组，然后将一个函数应用（apply）到个分组并产生一个新值。最后所有这些函数的执行结果会被合并（combine）到最终结果对象中。

数据组合

下面的例子按照sex进行分组，并计算tip列的平均值。

```
df = pd.read_csv('tips.csv')
#按照sex进行分组，并计算tip列的平均值
grouped = df['tip'].groupby(df['sex'])
grouped

grouped.mean()
Out[100]:
sex
Female    2.833448
Male      3.089618
Name: tip, dtype: float64
```

数据组合

一次传入多个数组，通过两个键对进行分组，得到的Series具有一个层次化索引

```
means = df['tip'].groupby([df['sex'],df['time']]).mean()
means
Out[101]:
sex     time
Female  Dinner    3.002115
        Lunch     2.582857
Male    Dinner    3.144839
        Lunch     2.882121
Name: tip, dtype: float64
```

```
means.unstack()
Out[102]:
time      Dinner     Lunch
sex
Female  3.002115  2.582857
Male    3.144839  2.882121
```

数据组合

利用agg()使用自定义聚合函数，一次传入多个聚合函数。

```
def peak_to_peak(arr):
    return arr.max() - arr.min()

grouped.agg(peak_to_peak)
Out[107]:
sex
Female    5.5
Male      9.0
Name: tip, dtype: float64
```

```
grouped.agg(['mean','std', peak_to_peak])
Out[108]:
            mean      std  peak_to_peak
sex
Female  2.833448  1.159495           5.5
Male    3.089618  1.489102           9.0
```

数据处理小结

数据处理：Filter 、Sort 和 GroupBy

- df[df[col] > 0.5]：选择col列的值大于0.5的行
- df.sort_values(col1)：按照列col1排序数据，默认升序排列
- df.sort_values(col2, ascending=False)：按照列col1降序排列数据
- df.sort_values([col1,col2], ascending=[True,False])：先按列col1升序排列，后按col2降序排列数据
- df.groupby(col)：返回一个按列col进行分组的Groupby对象
- df.groupby([col1,col2])：返回一个按多列进行分组的Groupby对象
- df.groupby(col1)[col2]：返回按列col1进行分组后，列col2的均值
- df.pivot_table(index=col1, values=[col2,col3], aggfunc=max)：创建一个按列col1进行分组，并计算col2和col3的最大值的数据透视表
- df.groupby(col1).agg(np.mean)：返回按列col1分组的所有列的均值
- data.apply(np.mean)：对DataFrame中的每一列应用函数np.mean
- data.apply(np.max,axis=1)：对DataFrame中的每一行应用函数np.max

数据合并

- df1.append(df2)：将df2中的行添加到df1的尾部
- df.concat([df1, df2],axis=1)：将df2中的列添加到df1的尾部
- df1.join(df2,on=col1,how='inner')：对df1的列和df2的列执行SQL形式的join

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

数字编码

使用map函数和replace函数可以将Series或DataFrame中的值进行替换，

```
In[109]:df = pd.read_csv('tips.csv')
df['time'].value_counts()
Out[109]:
Dinner    176
Lunch      68
Name: time, dtype: int64

In[110]:df['time'].replace(['Dinner','Lunch'],[0,1]).value_
counts()
Out[110]:
0    176
1     68
Name: time, dtype: int64
```

数字编码

```
In[110]:df['time'].replace(['Dinner','Lunch'],[0,1]).value_counts()
Out[110]:
0    176
1     68
Name: time, dtype: int64

In[111]:map_id = {'Dinner':0,'Lunch':1}
df['time'].map(map_id).value_counts()
Out[111]:
0    176
1     68
Name: time, dtype: int64
```

数据清理小结

数据清理

- df.columns = ['a','b','c']：重命名列名
- pd.isnull()：检查DataFrame对象中的空值，并返回一个Boolean数组
- pd.notnull()：检查DataFrame对象中的非空值，并返回一个Boolean数组
- df.dropna()：删除所有包含空值的行
- df.dropna(axis=1)：删除所有包含空值的列
- df.dropna(axis=1,thresh=n)：删除所有小于n个非空值的行
- df.fillna(x)：用x替换DataFrame对象中所有的空值
- s.astype(float)：将Series中的数据类型更改为float类型
- s.replace(1,'one')：用‘one’代替所有等于1的值
- s.replace([1,3],['one','three'])：用'one'代替1，用'three'代替3
- df.rename(columns=lambda x: x + 1)：批量更改列名
- df.rename(columns={'old_name': 'new_ name'})：选择性更改列名
- df.set_index('column_one')：更改索引列
- df.rename(index=lambda x: x + 1)：批量重命名索引

目录

第一部分 文件读写

第二部分 变量离散化

第三部分 缺失值填补

第四部分 数据标准化

第五部分 数据合并

第六部分 数据组合

第七部分 数字编码

第八部分 OneHot编码

OneHot编码

在机器学习中，一种常用的分类变量转换方式为将分类变量转换为“哑变量矩阵”或“指标矩阵”。如果DataFrame的某一列含有k个不同的值，则可以派生出一个k列矩阵（其值全为1或0）。pandas有一个get_dummies函数可以实现该功能。

```
dummies = pd.get_dummies(df['sex'])
dummies.head()

Out[116]:
   Female  Male
0       1     0
1       0     1
2       0     1
3       0     1
4       1     0
```

OneHot编码

有时候，你可能想给指标DataFrame的列加上一个前缀，以便能够跟其他数据进行合并。get_dummies的prefix参数可以实现这个功能：

```
dummies = pd.get_dummies(df['sex'],prefix='key')
df_with_dummy = df[['sex']].join(dummies)
df_with_dummy.head()
Out[117]:
     sex  key_Female  key_Male
0  Female          1         0
1    Male          0         1
2    Male          0         1
3    Male          0         1
4  Female          1         0
```

OneHot编码

一个有用的秘诀是：结合get_dummies和诸如cut之类的离散化函数。

```
bins = [1,3,6,10]
pd.get_dummies(pd.cut(df['tip'],bins)).head()
Out[118]:
   (1, 3]  (3, 6]  (6, 10]
0       1       0        0
1       1       0        0
2       0       1        0
3       0       1        0
4       0       1        0
```